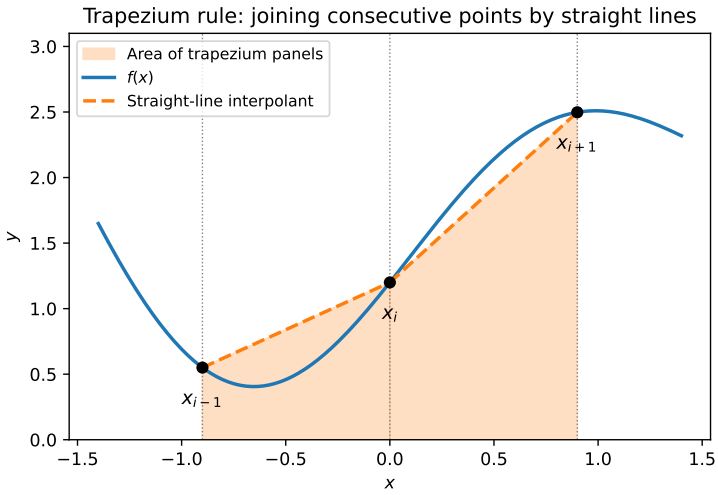
MTH2014, Numerical Modelling



Dr Stephen Thomson

Contents

1 Introduction **1**
- 1.1 Relationship between MTH2014 and MTH1003 . . . 1
- 1.2 What are numerical methods? . . . 2
- 1.3 Applications of numerics and optimisation . . . 2
  - 1.3.1 Fluid flows . . . 2
  - 1.3.2 Google . . . 2
  - 1.3.3 Financial modelling . . . 2
- 1.4 Module structure . . . 2
  - 1.4.1 Tutorials and Practicals . . . 3
- 1.5 How to learn about numerical methods . . . 3
- 1.6 Example problem requiring numerical solution . . . 3
- 1.7 Using python to understand numerics and optimisation . . . 4
- 1.8 Iteration and convergence . . . 6
- 1.9 Taylor series . . . 6
- 1.10 Notation of discretisation . . . 6
- 1.11 Books . . . 7

2 Root finding **8**
- 2.1 Finding roots of nonlinear equations . . . 8
- 2.2 Bracketing methods . . . 9
- 2.3 Bisection ♢ . . . 10
  - 2.3.1 Convergence analysis of bisection method . . . 11
- 2.4 Fixed-point iteration ♢ . . . 12
  - 2.4.1 Python example of fixed point method . . . 12
  - 2.4.2 Convergence analysis of fixed-point method ♢ . . . 13
- 2.5 Newton-Raphson method . . . 16
  - 2.5.1 Multiple roots and the modified Newton's method . . . 19
  - 2.5.2 Higher dimensional Newton-Raphson . . . 21
  - 2.5.3 2D Newton Raphson example . . . 23

3 Ordinary differential equations **26**
- 3.1 Numerical accuracy and the truncation error . . . 26
- 3.2 Round-off error . . . 27
- 3.3 Finite difference schemes ♢ . . . 28
  - 3.3.1 First derivative ♢ . . . 28

3.3.2 Second derivative ◊ . . . 29
3.4 Timestepping problems . . . 29
3.5 The concept of numerical stability . . . 29
3.5.1 Recursive example . . . 31
3.6 Forward Euler scheme ◊ . . . 32
3.6.1 Exponential decay ◊ . . . 32
3.6.2 Numerical instability example . . . 33
3.6.3 Oscillation ◊ . . . 34
3.7 Implicit schemes . . . 36
3.8 Leapfrog scheme . . . 36
3.8.1 Decay case . . . 37
3.8.2 Oscillation case . . . 38
3.9 Runge-Kutta method . . . 39
3.9.1 Worked example . . . 41
3.9.2 4th order Runge-Kutta (RK4) . . . 42
3.10 Multi-step methods . . . 43
3.10.1 Worked example . . . 44
3.10.2 Third order AB method . . . 44
3.10.3 Adams-Moulton methods . . . 45
3.10.4 Predictor-corrector methods . . . 45
3.11 Stiff differential equations . . . 49
3.12 Boundary value problems . . . 51

4 Numerical integration **54**
4.1 Newton-Cotes . . . 55
4.1.1 Trapezium Rule . . . 55
4.1.2 Simpson's Rule . . . 58
4.1.3 Generalisation to higher order polynomials . . . 60
4.1.4 Comparison with timestepping . . . 61
4.2 Gaussian Quadrature . . . 62

5 Matrix problems **65**
5.1 Example application . . . 66
5.2 Revision of simultaneous equations . . . 66
5.3 Gaussian elimination revisited ◊ . . . 66
5.4 LU decomposition . . . 68
5.4.1 The cost of Gaussian elimination versus LU . . . 69
5.5 Iterative matrix methods . . . 70
5.5.1 Jacobi method . . . 70
5.5.2 Example of rearrangement . . . 70
5.5.3 Python example . . . 71
5.5.4 Gauss-Seidel method . . . 72
5.6 Decomposition for analysis of Jacobi and Gauss-Seidel . . . 73
5.6.1 Jacobi method . . . 73
5.6.2 Gauss-Seidel method . . . 73

5.7 Norms: tools for convergence analysis . . . 74
5.7.1 Vector norms . . . 74
5.7.2 Matrix norms . . . 76
5.8 Convergence analysis of Jacobi method: method of norms . . . 76
5.8.1 Examples of diagonal dominance . . . 78
5.9 Convergence analysis using eigenvalues . . . 78
5.9.1 Worked example of spectral radius . . . 79
5.10 Calculation of eigenvalues: power method . . . 80
5.10.1 Python example . . . 81
5.10.2 Power method for smallest absolute eigenvalue . . . 82
5.11 The condition number . . . 82
5.12 Matrices and partial differential equations . . . 83

6 Topics in mathematical modelling **85**
6.1 Electronic networks: a matrix problem . . . 85
6.2 Chemical kinetics: an ODE/matrix problem . . . 87
6.3 Predictability and chaos: nonlinear ODEs . . . 88
6.3.1 Predictability ideas . . . 88
6.3.2 Parameters for chaos . . . 89
6.3.3 Runge-Kutta schemes . . . 90
6.4 Image compression: a matrix decomposition problem . . . 90

7 Partial differential equations **93**
7.1 The linear diffusion equation . . . 93
7.2 von Neumann stability analysis . . . 94

Chapter 1

Introduction

Historical figures

Charles Babbage (1791–1871) and **Ada Lovelace** (1815–1852) worked together on one of the earliest visions of a general-purpose computer. Babbage's Difference Engine, designed from 1819, was a mechanical calculator that tabulates polynomial functions using only repeated addition — the method of finite differences that underlies much of numerical computation — and he went on to design a more ambitious successor, the Analytical Engine. It was never built, but in 1843 Lovelace published notes on it that are now recognised as containing the first algorithm intended for execution by a machine: a step-by-step iterative procedure for computing Bernoulli numbers. The idea of repeating a simple step to converge on an answer, which she was among the first to describe precisely, is the thread running through almost every method in this module. Babbage and Lovelace on Wikipedia.

1.1 Relationship between MTH2014 and MTH1003

MTH1003 (Mathematical Modelling) covers a huge range of topics in applied mathematics. Some of these are very much applied physics problems, with numerical methods discussed as a tool for solving them. In this course, we will take a similar approach, but with less of a focus on the problems and more of a focus on the numerical methods themselves.

Many of the topic areas covered in MTH2014 build heavily on the material you have seen in MTH1003. In order to do this building, it will be necessary to revisit material from MTH1003. In these lecture notes I have taken the approach of including material that overlaps with MTH1003, and those subsections that are essentially revision of MTH1003 will be marked with a diamond symbol ($\Diamond$). These sections will not be discussed extensively in the lectures, but are included here to help set the scene for the more advanced topics that follow.

I am indebted to Prof. Bob Beare, who wrote the majority of these notes for the the old MTH2005 module, which MTH2014 has replaced.

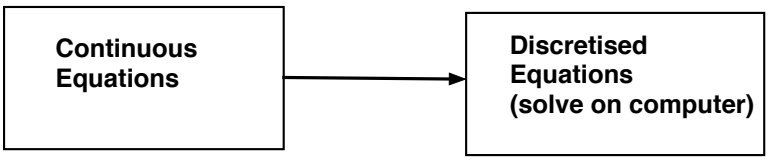


Figure 1.1: A schematic illustrating the process of numerical methods

1.2 What are numerical methods?

Numerical methods involve transforming a continuous set of mathematical equations such as differential equations into a discrete, algebraic set of equations (see Figure 1.1). These algebraic equations can then be solved on a computer. Numerics is thus an applied mathematics topic, but nonetheless with a rigorous mathematical basis.

1.3 Applications of numerics and optimisation

1.3.1 Fluid flows

- Formula 1 cars. Modelling the fluid flow around a Formula 1 car is an application of computational fluid dynamics. Typically, the air surrounding the car will be divided into a mesh of grid boxes and the fluid equations solved for each of these grid boxes.
- Weather and climate prediction. The fluid equations are solved for the global atmospheric shell (as well as the oceans). These are discretised on a grid.
- Modelling other planets. Similar methods to solving the fluid equations for weather and climate are used to model the atmospheres of other planets.

1.3.2 Google

When you use the Google search engine, it requires a ranking of web pages. Part of the PageRank algorithm solves an eigenvalue problem in order to do this. The matrices involved are massive, so the power method (studied under the matrices section of the module) is used to find the leading eigenvalues

1.3.3 Financial modelling

Aspects of the time evolution of the stock market can be modelled by differential equations such as the Black-Scholes equation. Solving such equations can involve numerical computation, for example to generate random numbers.

1.4 Module structure

The module has four parts:

1. Root finding
2. Ordinary differential equations
3. Matrices
4. Optimisation

These sections provide a useful introduction to numerics and optimisation.

1.4.1 Tutorials and Practicals

In addition to the lectures, we run four tutorials and five practicals during the term on alternate weeks. The tutorials will be classes going through problem sheets on each of the four sections. They will tend to be exam questions. The practicals will be in computing labs and give you the chance to try out the numerical methods on a computer using python. The first practical will include a refresher on the basics of python.

1.5 How to learn about numerical methods

In this module we pursue several different approaches to learning about numerical methods.

- Analytically (lectures and problem classes)
- Solving numerical methods by hand (lectures and problems classes)
- At a computer using python (practicals)
- Intuitively using diagrams (lectures and problems classes)

I thoroughly recommend you pursuing all these approaches in order to develop a full understanding of the subject.

1.6 Example problem requiring numerical solution

We can integrate the following analytically

$$\int_0^1 x^2 dx = \left[\frac{x^3}{3}\right]_0^1 = \frac{1}{3},$$

but some integrals cannot be solved in this way, for example

$$\int_1^2 x^x dx.$$

The integral exists as we can identify the area under the curve as in the Fig. 1.2. Instead we need to discretise the integral and solve it numerically.

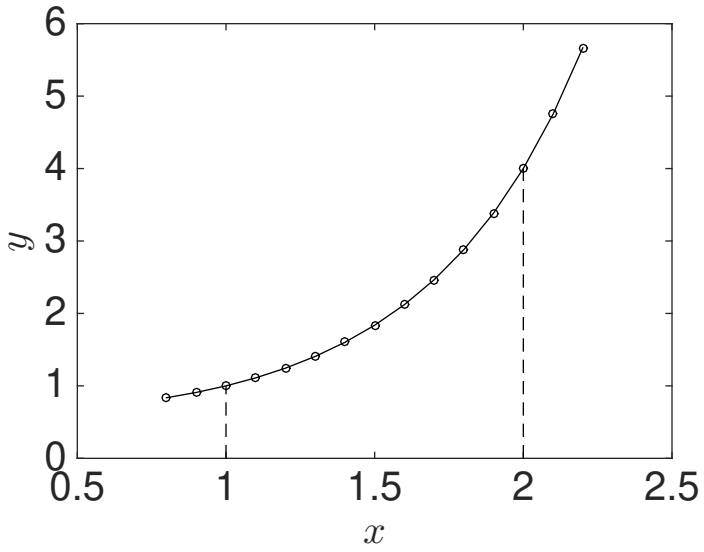


Figure 1.2: The function $y = x^x$ including the limits marked by vertical dashed lines.

1.7 Using python to understand numerics and optimisation

Here we will provide some brief examples of using python. There will be more chance to practice python in the practicals and I will use python examples in the lectures. This module is not another programming course, but the aim is to use python codes to understand the algorithms in the lectures.

Matrices are relatively easy to manipulate in python. There is also more than one way to define one. Since we will be using the numpy libary in this module, here is an example of defining a 3x3 matrix:

$$A = \begin{pmatrix} 1 & 3 & 2 \\ 5 & 7 & -1 \\ 2 & 5 & 4 \end{pmatrix}$$

In python input

```
import numpy as np

A = np.array([[1, 3, 2],
              [5, 7, -1],
              [2, 5, 4]])

print(A)
```

Note the use of square brackets to define the matrix, with each row defined with in a pair of square brackets. A column vector

$$\mathbf{x} = \begin{pmatrix} 5 \\ 7 \\ 8 \end{pmatrix}$$

is defined using numpy in a similar way to matrix but now each row only contains a single element

```
import numpy as np

x=np.array([[5],
            [7],
            [8]])

print(x)
```

We will revise basic python in the first practical.

1.8 Iteration and convergence

A key aspect of numerics is transforming the underlying mathematical problem into a series of iterations that can be solved by computer. Whilst iterations are tedious to us, they are no problem for a sufficiently large computer! A key aspect of the mathematics of numerical methods is analysing the conditions for convergence of numerical methods. By this we mean that the numerical solution should converge to the true solution with many iterations.

As mathematicians, we shouldn't just use algorithms, but understand the conditions under which they work or don't work. Throughout the module, we will accumulate a 'tool kit' of algorithms. If you have a good understanding of the module, you will also be able to analyse these algorithms and thus understand the conditions for their convergence.

1.9 Taylor series

The Taylor series is very useful in defining numerical methods as it approximates a function at a discrete displacement from a starting point. We briefly revise it here. If the function f is infinitely differentiable at x then values of f in some neighbourhood of x are given by

$$\begin{aligned} f(x+h) &= f(x) + hf'(x) + \frac{h^2}{2}f''(x) + ... \\ &= \sum_{k=0}^{\infty} \frac{h^k}{k!} f^k(x) \end{aligned} \tag{1.1}$$

where h is a displacement from x.

1.10 Notation of discretisation

The number of the iteration step will be typically represented here by the integers n or i in the subscript. So

$$x_n$$

indicates the value of x at the step n.

1.11 Books

The module is self contained meaning that all you should need will be in these lecture notes or on the ele page. However, if you are interested in reading further you can look at the following books

- Kharab A. and Guenther R. G.,"An introduction to numerical methods: A MATLAB approach, Fifth Edition", Chapman and Hall/CRC press. This book follows the module most closely for the algorithms except that we now use python instead of MATLAB.
-

Chapter 2

Root finding

Historical figures

Every method in this chapter shares a single idea: repeat a simple step until the answer converges. **Ada Lovelace** (1815–1852) gave one of the earliest precise descriptions of this idea in 1843, publishing notes on Babbage's proposed Analytical Engine that are now recognised as the first algorithm intended for execution by a machine — a step-by-step iterative procedure for computing Bernoulli numbers. The same idea, applied specifically to finding roots, goes back further still: **Isaac Newton** (1643–1727) described a method for approximating roots of polynomials in 1669, and **Joseph Raphson** (c. 1668–c. 1715) published a simpler, more general version in his 1690 book *Analysis Aequationum Universalis*; the modern iterative formula is a synthesis of the two. Lovelace (more on "Note G"), Newton, Raphson on Wikipedia.

In this chapter, we will revisit some of the root finding methods you covered in year 1, with the aim of carefully observing the assumptions made and thoroughly understanding the proofs of convergence. A key concept that we will focus on will be the rate of convergence of the numerical solution to the true one. This process will then prepare you for new areas of the module such as iterative matrix methods.

In lectures we will go through these sections rather quickly, given that you've studied both bisection and fixed-point iteration in MTH1003. However, the full explanations are given below to help you solidify your understanding. The main new material in this section is on the Newton–Raphson method, which is covered in section 2.5.

2.1 Finding roots of nonlinear equations

Suppose we wish to solve

$$f(x) = 0 \tag{2.1}$$

for some nonlinear scalar function f of a scalar variable x. Equation (2.1) is solvable analytically for some special cases, for example if $f(x)$ is quadratic. However, if this is not the case, then Eq. (2.1) needs to be solved numerically. A sequence of approximate solutions to

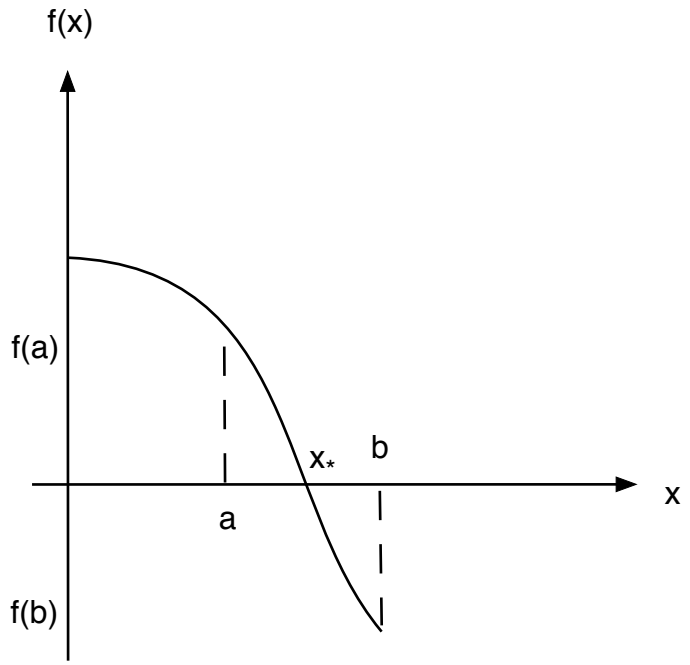


Figure 2.1: An example of a and b bracketing the root x_*. $f(a)$ and $f(b)$ have opposite signs

x are given by

$$x_n : n \in \mathbb{N},$$

where n is the iteration step, and we take $\mathbb{N} = \{0, 1, 2, \ldots\}$ throughout this chapter (x_n is the same as $x^{(n)}$, sometimes used in other courses). For our numerical method to be successful, we require that x_n *converges* to the true solution (x_*) meaning

$$x_n \to x_* \quad \text{as} \quad n \to \infty. \tag{2.2}$$

This is our first numerical method, and with it we find some new terminology and concepts:

- *A discretised system of equations* We have replaced an analytic problem (e.g. solving a quadratic) with a sequence of iteration steps. Computers do not complain about doing many iterations, and lots of numerical methods are of a similar form.
- *The true solution* Sometimes called the exact solution, denoted by x_*. We assume that the true solution exists. In the examples we encounter in this module, we know the true solution. This is not always the case in real-world problems.
- *Convergence* For our numerical method to be useful, we require that the numerical solution converges to the true solution as stated in Eq. (2.2).

2.2 Bracketing methods

A robust way of ensuring that the numerical method converges is to ensure that the root is 'bracketed' by two points a_n and b_n throughout the iteration. By this we mean that a_n and

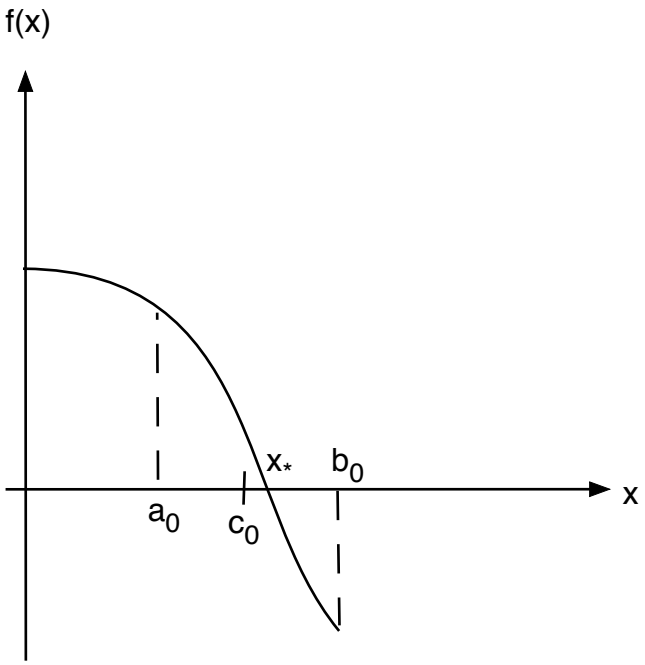


Figure 2.2: The initial step of the bracketing algorithm

b_n are always placed either side of the root during the iterations. The existence of the root within the bracket can be determined using the Intermediate Value Theorem (IVT).

Theorem 1 (Intermediate Value Theorem (IVT))**.** *If f is continuous on $[a, b]$ and if $f(a)f(b) \leq 0$, then there exists a root $x_* \in [a, b]$ such that $f(x_*) = 0$.*

The proof of this theorem is in the Analysis module. The IVT only applies in one dimension. The fact that the true solution (x_*) exists underpins our root finding methods. Fig. 2.1 provides and illustration of the IVT, where $f(a)$ and $f(b)$ having opposite signs. We will now consider one type of bracketing method, bisection.

2.3 Bisection ♢

Bisection is the simplest and most robust bracketing method. It is robust because bracketing, by definition, always ensures convergence to the root. The steps of the bisection algorithm are

1. *Set inital bracket*
$$a_0 = a; \quad b_0 = b$$
2. *Bisect the bracket to find the new point c_n*
$$x_n = c_n = \frac{1}{2}(a_n + b_n) \quad n = 0, 1, 2 \ldots$$
3. *Select new bracket using IVT*:

If $f(a_n)f(c_n) < 0$ use the left bracket $[a_n, c_n]$, such that

$$a_{n+1} = a_n; \quad b_{n+1} = c_n$$

If $f(c_n)f(b_n) < 0$ use the right bracket $[c_n, b_n]$, such that

$$a_{n+1} = c_n; \quad b_{n+1} = b_n$$

4. *Repeat steps 2-3* until $f(c_n) = 0$ within a certain tolerance.

Bisection is robust (it will always converge) but slow to converge as the bracket size is only halved at each iteration.

2.3.1 Convergence analysis of bisection method

An important part of numerical analysis is understanding:

1. if a method will converge to the true solution
2. if point 1 applies, then at what *rate* will it converge with iteration step

We now consider the convergence of the bisection method. The IVT has ensured point 1 is satisfied. For point 2, we consider the width of the bracket (w_n):

$$w_n = b_n - a_n = \frac{1}{2^n}(b_0 - a_0) \tag{2.3}$$

as the width is halved with each iteration. The *error* in the solution, e_n, at any point in the iteration is defined as the difference between the numerical and true solutions

$$e_n = x_n - x_*. \tag{2.4}$$

A comment on the notation here. Throughout the module, we will refer to Eq. (2.4) as the "error" and

$$|e_n| = |x_n - x_*| \tag{2.5}$$

as the *absolute* error. Sometime the absolute error is called the error elsewhere in the numerics literature.

Given x_n is the bisector, we know that the true solution will be somewhere between the bisector and the end of the bracket; thus we know that $|e_n|$ will be less than half the width of the bracket

$$\max(|e_n|) = \frac{w_n}{2}. \tag{2.6}$$

We know that this width will be the maximum possible difference between the numerical and true solutions. Substituting Eq. (2.3) into Eq. (2.6) gives the maximum error

$$\max(|e_n|) = \frac{w_n}{2} = \frac{b_n - a_n}{2} = \frac{1}{2^{n+1}}(b_0 - a_0)$$

meaning the maximum error halves with each step

$$\max(|e_{n+1}|) = \frac{\max(|e_n|)}{2}$$

Such rates of the convergence of the error are very useful in assessing numerical methods. Here, we refer to the convergence rate of the bisection method as linear. The general formula for convergence of the error is

$$|e_{n+1}| = A|e_n|^q. \tag{2.7}$$

Here we have also been restricted to look at the maximum error, due to the nature of the problem (we can see from the geometry that the maximum error is going to be bounded by the half width). We require $A < 1$ for convergence to happen at all. Assuming that the numerical solution is sufficiently close to the true one that $|e_{n+1}| < 1$, the rate of convergence is determined by the power q. The greater q, the faster the error converges to zero as $n \to \infty$. For $q = 1$, as in this bisection case, the convergence rate is first-order or linear. So, although bisection is robust (it will always converge if the root exists), it converges relatively slowly. We will compare this rate of convergence with other methods later. Other bracketing methods (not part of the module) include the Secant and Regula Falsi methods.

2.4 Fixed-point iteration ♢

The fixed-point method involves rearranging our original root problem into a different form. This rewriting sometimes presents the biggest challenges conceptually, so is illustrated in Figure 2.3. Starting with $f(x) = 0$ (intersecting at the x axis) we rewrite the problem as

$$g(x) = x \Leftrightarrow f(x) = g(x) - x = 0$$

The root has not changed. The method has changed by moving the intersection point in the $x - y$ plane. Having completed the rewriting stage and, given a form of $g(x)$ that is sufficiently well behaved (see proof later), we have the algorithm:

1. We rewrite $f(x) = 0$ in the form $g(x) = x$.
2. Iterate: $x_{n+1} = g(x_n)$.

This is one of the simplest root-finding methods to compute as it only involves evaluting $g(x)$ at each stage of the iteration.

2.4.1 Python example of fixed point method

Find the roots of

$$f(x) = \cos(x) - x = 0.$$

First rewrite the problem as

$$g(x) = \cos(x) = x$$

giving a fixed-point iteration of

$$x_{n+1} = g(x_n) = \cos(x_n)$$

In python we can solve it using the following lines

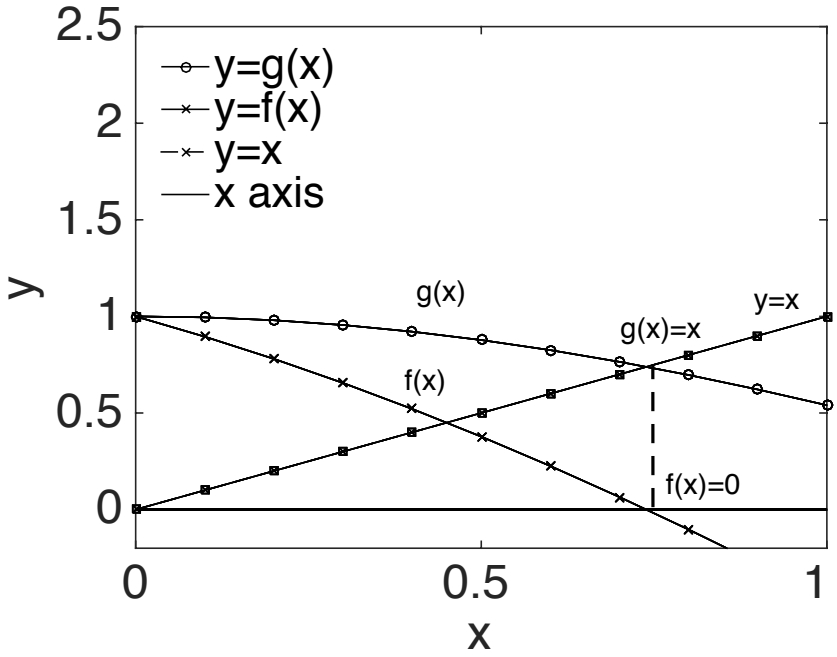


Figure 2.3: Illustration of the rewriting of $f(x) = 0$ as $g(x) = x$. The root is the same in both, but the intersections occur at different locations in the x-y plane.

```
import numpy as np # Import NumPy library
x=0.4 # Initial value of x

for n in range(20): # Perform 20 fixed-point iterations
    x_new=np.cos(x) #Fixed point iteration
    x=x_new # Transfer variables for next step

print('%.3f' %x) # Print x. Verify that x=cos(c) to 3dp

print('%.3f' %np.cos(x)) # Print cos(x)
```

The last two lines validate that x is approximately $g(x)$.

2.4.2 Convergence analysis of fixed-point method ◊

We will state the mean value theorem without proof.

Theorem 2. *If the function f is continuous on the closed interval $[a, b]$ and differentiable on the open interval (a, b), then there exists a real number $\xi \in (a, b)$ such that*

$$f'(\xi) = \frac{f(b) - f(a)}{b - a}.$$

This theorem can be proved using Taylor series. Note the use of the dependent variable ξ instead of x. The variable ξ can be interpreted in words as 'anywhere in the interval

(a, b) without the need of specifying exactly where'. The mean value theorem is then stating that somewhere in the interval (a, b) the gradient of f will be the same as the straight line connecting $f(a)$ and $f(b)$.

Definition 1. *When $f(x) = 0$ is rearranged as $x = g(x)$, the fixed point iteration is given by*

$$x_{n+1} = g(x_n). \tag{2.8}$$

Definition 2. *The true solution (x_*) is given by*

$$x_* = g(x_*). \tag{2.9}$$

Theorem 3. *The fixed-point iteration converges to the root if $|g'(\xi)| < 1$ where $\xi \in (a, b)$ and (a, b) brackets the root.*

Proof Substituting from Eqs. (2.8) and (2.9), the error at a step n is given by

$$e_n = x_n - x_* = g(x_{n-1}) - g(x_*). \tag{2.10}$$

For a given step, we have the point $\xi_n \in (a, b)$ that satisfies the Mean Value Theorem so that

$$g'(\xi_{n-1}) = \frac{g(x_{n-1}) - g(x_*)}{x_{n-1} - x_*}$$

Substituting from Eq. (2.10).

$$\Rightarrow e_n = g'(\xi_{n-1})(x_{n-1} - x_*) = g'(\xi_{n-1})e_{n-1}.$$

Taking the modulus and using the "multiplicativity" identity ($|ab| = |a||b|$ where a and b are real) gives

$$\Rightarrow |e_n| = |g'(\xi_{n-1})||e_{n-1}|.$$

Comparing with the general convergence formula, Eq. 2.7, we see that $q = 1$ and $A = |g'(\xi_{n-1})|$. So, the convergence is still first order, but the rate of convergence is also controlled by $|g'(\xi_{n-1})|$, compared to the constant factor of 0.5 in bisection. For e_n to decrease at each iteration step ($A < 1$) requires

$$\Rightarrow |g'(\xi_n)| < 1 \quad \forall n \in \mathbb{N},$$

where we have substituted $n-1$ for n (as if the convergence criterion applies for step $n-1$, then it also applies at step n). The method improves on the rate of convergence of bisection if $|g'(\xi_n)| < 1/2$. This proof shows that, although there can be several apparently equivalent forms of $x = g(x)$, it is the gradient of $g(x)$ that determines whether the fixed-point method will converge or not. Figures 2.4 and 2.5 illustrate this result. In Figure 2.4, the absolute value of the gradient of $g(x)$ is smaller than one so that the fixed point method "cobwebs" into the solution. In contrast, Fig. 2.5 shows the method spiralling away from the solution when the absolute value of the gradient of $g(x)$ is greater than one.

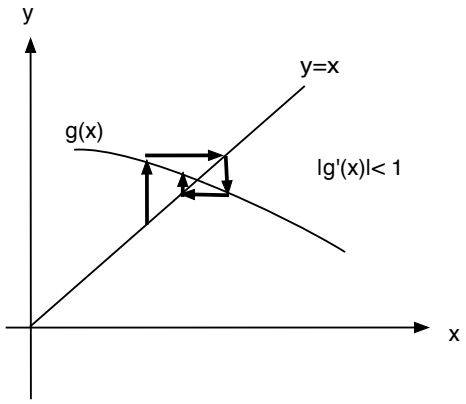


Figure 2.4: An example of 'cobwebbing' into the solution with $g'(x) < 0$ and $|g'(x)| < 1$ for all x within the initial bracket.

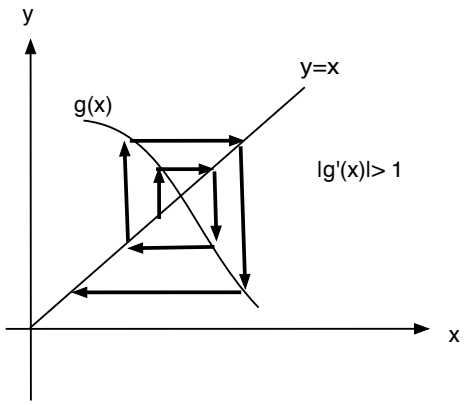


Figure 2.5: An example of 'spiralling' out and diverging when $g'(x) < 0$ and $|g'(x)| > 1$ within the initial bracket.

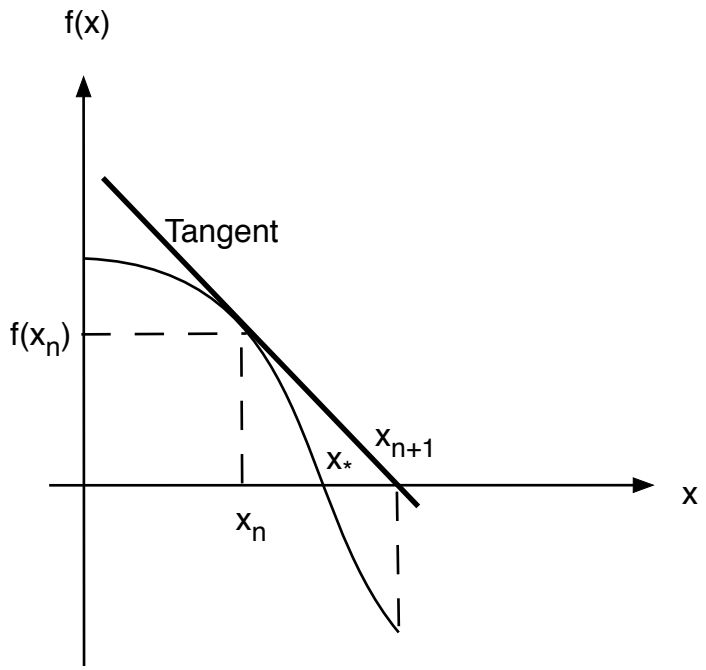


Figure 2.6: The Newton-Raphson method

2.5 Newton-Raphson method

In cases where the first derivative $f'(x)$ is *well behaved* in the region of the root (we will give a theorem later that will define precisely what is meant by this), we can used the Newton-Raphson method to compute the root.

Figure 2.6 shows the geometric construction of the method. We equate the first derivative at x_n with the the gradient of the constructed tangent line giving

$$f'(x_n) = -\frac{f(x_n)}{x_{n+1} - x_n},$$

which can be rearranged to give the Newton-Raphson algorithm

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad n = 0, 1, 2 \ldots \tag{2.11}$$

Rate of convergence

Using the definition of e_n with Eq. 2.11 gives

$$\begin{aligned} e_{n+1} &= x_{n+1} - x_* \\ &= x_n - x_* - \frac{f(x_n)}{f'(x_n)} \\ &= e_n - \frac{f(x_n)}{f'(x_n)} \end{aligned} \tag{2.12}$$

We now apply the Taylor series expanding about x_n to get x_*

$$\begin{aligned} f(x_*) = 0 &= f(x_n) + (x_* - x_n)f'(x_n) + \frac{(x_* - x_n)^2}{2}f''(\xi_n) \\ &= f(x_n) - e_n f'(x_n) + \frac{e_n^2}{2}f''(\xi_n) \end{aligned} \tag{2.13}$$

where the last term is the *Lagrange remainder* term. We use the Lagrange remainder theorem here without proof (proofs are available in some calculus courses and involve a form of the mean value theorem). Essentially, the Lagrange remainder theorem states that *all* the remaining terms in a Taylor series (in this case second order and up) can be replaced by a remainder term if the variable $\xi_n \in (x_*, x_n)$ replaces x_n (in this case in the second order term). Here the Lagrange remainder is equivalent to all the terms in the Taylor series above and including the second derivative. We are also assuming x_n sufficiently close to x_* such that the leading order terms dominate the Taylor series. Eliminating $f(x_n)$ between Eqs. (2.13) and (2.12) gives

$$\begin{aligned} 0 &= -(e_{n+1} - e_n)f'(x_n) - e_n f'(x_n) + \frac{e_n^2}{2}f''(\xi_n) \\ \Rightarrow 0 &= -e_{n+1} + \frac{e_n^2}{2}\frac{f''(\xi_n)}{f'(x_n)} \\ \Rightarrow e_{n+1} &= \frac{e_n^2}{2}\frac{f''(\xi_n)}{f'(x_n)} \\ \Rightarrow |e_{n+1}| &\propto |e_n^2| \end{aligned}$$

So the Newton-Raphson method is second order, i.e. faster than the linear bisection rate. However, Newton-Raphson is not a bracketing method, so we are now dependent on the gradient being well behaved near the root to ensure convergence. For example, if the gradient tends to infinity at the root, or is zero at some point of the iteration,the Newton Raphson method will not find the root.

We now provide a proof of the behaviour of the gradient for convergence.

Theorem 4.

$$\frac{|f(\xi_n)f''(\xi_n)|}{f'(\xi_n)^2} < 1$$

over the range of ξ_n around the root that Newton-Raphson is being applied for convergence

Proof We first state the Newton Raphson method as a fixed-point problem. The Newton Rapshon method has converged when $\xi_n = x_{n+1} = x_n$ and so

$$-\frac{f(\xi_n)}{f'(\xi_n)} = 0$$

Adding ξ_n to each side, changes the problem to a fixed point one

$$g(\xi_n) = \xi_n - \frac{f(\xi_n)}{f'(\xi_n)} = \xi_n$$

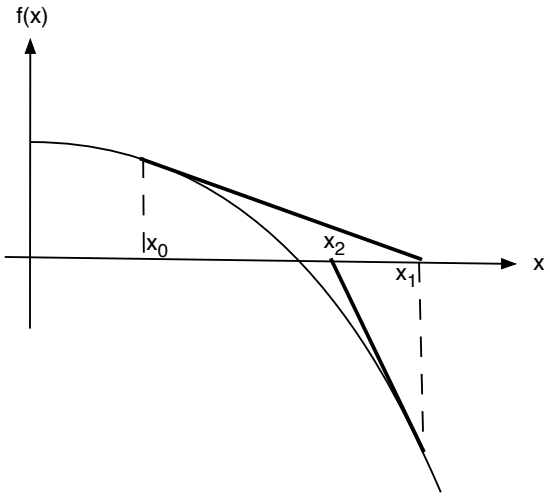


Figure 2.7: An example of the gradient being well behaved in the region of the root and Newton-Raphson converging to the root.

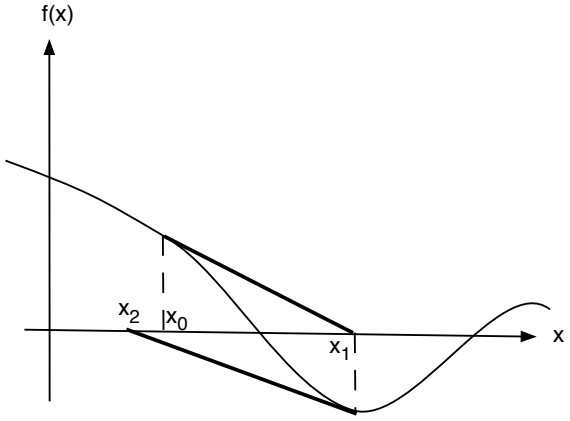


Figure 2.8: An example of the gradient being poorly behaved in the region of the root. The local minimum to the right of the root, cause the algorithm to diverge.

Differentiating using the quotient rule gives

$$g'(\xi_n) = 1 - \frac{f'(\xi_n)f'(\xi_n) - f(\xi_n)f''(\xi_n)}{f'(\xi_n)^2} = \frac{f(\xi_n)f''(\xi_n)}{f'(\xi_n)^2}$$

and using the result for the convergence of the fixed point method ($|g'(\xi_n)| < 1$) gives

$$\frac{|f(\xi_n)f''(\xi_n)|}{f'(\xi_n)^2} < 1$$

for convergence

For example, in Fig. 2.8, $f'(x)$ becomes small at x_1, thus violating this condition. In contrast, Fig. 2.7 has a well-behaved gradient in the region of the root and thus converges.

2.5.1 Multiple roots and the modified Newton's method

The convergence theorem above implicitly assumes that x_* is a *simple* root, i.e. $f'(x_*) \neq 0$. What happens when $f'(x_*) = 0$ too?

We say that x_* is a root of *multiplicity* m of f if

$$f(x_*) = f'(x_*) = \cdots = f^{(m-1)}(x_*) = 0, \qquad f^{(m)}(x_*) \neq 0 \tag{2.14}$$

(so that a simple root, as assumed above, is the case $m = 1$). Using Taylor's theorem about x_*, any such f can be written in the form

$$f(x) = (x - x_*)^m h(x), \qquad h(x) = \frac{f^{(m)}(x_*)}{m!} + \frac{f^{(m+1)}(x_*)}{(m+1)!}(x - x_*) + \ldots \neq 0. \tag{2.15}$$

Worked example. The function $f(x) = x^3 - 7x^2 + 11x - 5$ satisfies $f(1) = 0$ and $f'(1) = 3 - 14 + 11 = 0$, but $f''(1) = 6 - 14 = -8 \neq 0$, so $x_* = 1$ is a root of multiplicity $m = 2$.

Newton-Raphson at a multiple root

We first check numerically what happens if we apply the ordinary Newton-Raphson iteration (2.11) to this example, starting from $x_0 = 0$.

```
def f(x):
    return x**3 - 7*x**2 + 11*x - 5

def fprime(x):
    return 3*x**2 - 14*x + 11

def newton(x0, n_iter):
    xs = [x0]
    x = x0
    for _ in range(n_iter):
        x = x - f(x) / fprime(x)
        xs.append(x)
    return xs
```

Running `newton(0.0, 30)` and comparing each iterate to the true root $x_* = 1$ gives Table 2.1. The pattern in the last column is no coincidence: it can be shown that at a root of

n	x_n	$\lvert x_n - x_* \rvert$	$\lvert e_{n+1} \rvert / \lvert e_n \rvert$
0	0.00000000	1.0000×10^{0}	0.545455
1	0.45454545	5.4545×10^{-1}	0.528302
2	0.71183533	2.8816×10^{-1}	0.516254
3	0.85123388	1.4877×10^{-1}	0.508807
4	0.92430682	7.5693×10^{-2}	0.504600
5	0.96180520	3.8195×10^{-2}	0.502353
6	0.98081271	1.9187×10^{-2}	0.501191
7	0.99038351	9.6165×10^{-3}	0.500599
8	0.99518600	4.8140×10^{-3}	0.500300
9	0.99759155	2.4084×10^{-3}	0.500150

Table 2.1: Standard Newton-Raphson applied to $f(x) = x^3 - 7x^2 + 11x - 5$ from $x_0 = 0$. The ratio in the final column is converging towards 0.5, not decaying as $|e_n|^2$: the method has degraded to *linear* convergence. Continuing the iteration, it takes 27 further steps to reach $|x_n - x_*| < 10^{-8}$.

multiplicity m, plain Newton-Raphson converges linearly with asymptotic ratio $|e_{n+1}|/|e_n| \to (m-1)/m$ – here $(m-1)/m = 1/2$, matching Table 2.1 closely.

The modified Newton's method

Differentiating Eq. (2.15),

$$f'(x) = (x - x_*)^{m-1}\Big[m\,h(x) + (x - x_*)h'(x)\Big], \tag{2.16}$$

so the function

$$u(x) = \frac{f(x)}{f'(x)} = (x - x_*)\,\Psi(x), \qquad \Psi(x) = \frac{h(x)}{m\,h(x) + (x - x_*)h'(x)}, \tag{2.17}$$

has a *simple* zero at x_*, since $\Psi(x_*) = 1/m \neq 0$. Applying Newton-Raphson to u instead of f should therefore restore quadratic convergence. Rather than differentiate u itself (which would require f''), we use the limiting value $u'(x_*) = \Psi(x_*) = 1/m$ in place of $u'(x_n)$ for x_n close to x_*, giving

$$x_{n+1} = x_n - \frac{u(x_n)}{u'(x_n)} \approx x_n - m\,u(x_n) = x_n - m\frac{f(x_n)}{f'(x_n)}. \tag{2.18}$$

This is the *modified Newton's method*. It needs only f, f' and the multiplicity m – no second derivative is required.

```
def modified_newton(x0, m, n_iter):
```

```
    xs = [x0]
    x = x0
    for _ in range(n_iter):
        x = x - m * f(x) / fprime(x)
        xs.append(x)
    return xs
```

Running `modified_newton(0.0, 2, 5)` on the same example gives Table 2.2.

n	x_n	$\lvert x_n - x_* \rvert$	$\lvert e_{n+1} \rvert / \lvert e_n \rvert^2$
0	0.00000000	1.0000×10^{0}	0.090909
1	0.90909091	9.0909×10^{-2}	0.120879
2	0.99900100	9.9900×10^{-4}	0.124953
3	0.99999988	1.2470×10^{-7}	–
4	1.00000000	1.7179×10^{-9}	–

Table 2.2: Modified Newton's method ($m = 2$) applied to the same problem. The ratio in the final column is converging towards a constant (≈ 0.12), consistent with quadratic convergence, $|e_{n+1}| \propto |e_n|^2$. The ratio is omitted for $n = 3, 4$: once $|x_n - x_*|$ drops below about $\sqrt{\epsilon_{\text{mach}}} \approx 10^{-8}$, both $f(x_n)$ and $f'(x_n)$ are themselves computed to only a handful of significant figures (since both vanish at x_*), so the ratio is dominated by floating-point rounding noise rather than the true convergence behaviour.

Figure 2.9 compares the two methods directly: modified Newton reaches $|x_n - x_*| < 10^{-8}$ after only 4 iterations, versus 27 for plain Newton-Raphson on the same problem.

Practical note. The modified method requires knowing the multiplicity m in advance. In practice m is often known from the context of the problem (as here), but if not, the linear convergence ratio of plain Newton-Raphson (Table 2.1) can be used to estimate it, since that ratio tends to $(m-1)/m$.

2.5.2 Higher dimensional Newton-Raphson

Consider a two dimensional vector, consisting of two functions f and g evaluated at step n

$$\mathbf{F}(\mathbf{x}_n) = \begin{pmatrix} f(x_n, y_n) \\ g(x_n, y_n) \end{pmatrix}$$

We now aim to use Newton-Raphson to find the roots of *both* f and g. Taylor expanding both f and g at the root gives

$$\begin{pmatrix} f(x_{n+1}, y_{n+1}) \\ g(x_{n+1}, y_{n+1}) \end{pmatrix} = \overbrace{\begin{pmatrix} f(x_n, y_n) \\ g(x_n, y_n) \end{pmatrix}}^{\text{vector } \mathbf{F}(\mathbf{x}_n)} + \overbrace{\begin{pmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{pmatrix}}^{\text{Jacobian } J(\mathbf{x}_n)} \begin{pmatrix} x_{n+1} - x_n \\ y_{n+1} - y_n \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \tag{2.19}$$

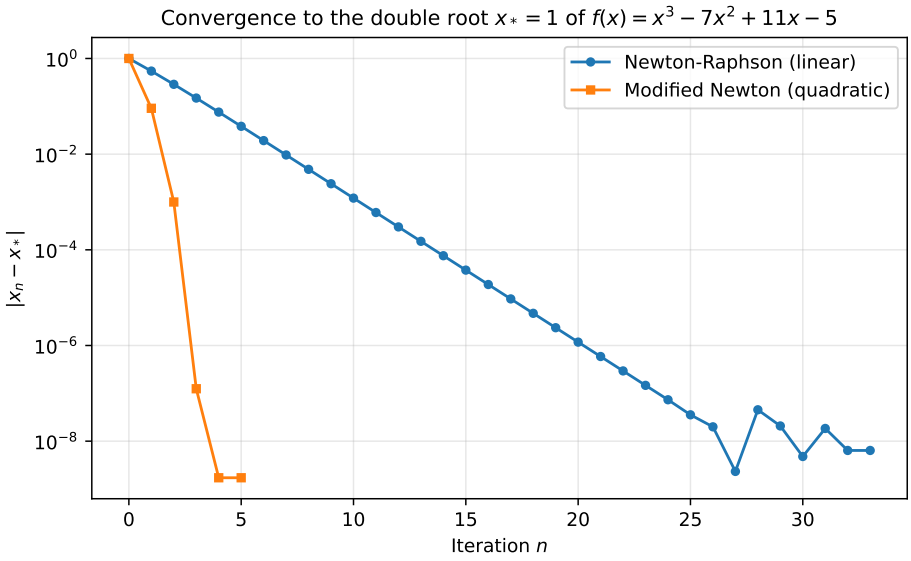


Figure 2.9: Convergence of standard Newton-Raphson (linear) and the modified Newton's method (quadratic) to the double root $x_* = 1$ of $f(x) = x^3 - 7x^2 + 11x - 5$, both started from $x_0 = 0$. Note the logarithmic vertical axis: a straight line indicates linear convergence, while the sharply curving modified-Newton line indicates quadratic convergence. Both curves flatten out once they reach the floating-point noise floor.

Equation 2.19 can be considered a two component version of the following expansion of a function $f(x, y)$ (a similar expression would apply for $g(x, y)$):

$$f(x + \Delta x, y + \Delta y) = f(x, y) + \frac{\partial f}{\partial x}\Delta x + \frac{\partial f}{\partial y}\Delta y, \tag{2.20}$$

which is just a 2D version of

$$f(x + \Delta x) = f(x) + \frac{df}{dx}\Delta x + \dots$$

in 1D.

The last part of Eq. 2.19 arises as we are considering the root, where the changes in $\mathbf{x}_n$ are zero. The Jacobian matrix, J, plays the same role as the gradient in 1D. Rearranging gives the 2D (or higher) Newton Raphson method

$$\mathbf{x}_{n+1} = \mathbf{x}_n - J^{-1}(\mathbf{x}_n)\mathbf{F}(\mathbf{x}_n) \tag{2.21}$$

This generalises directly to N equations $f_1, \dots, f_N$ in N unknowns $x_1, \dots, x_N$. Write $\mathbf{x}_n = (x_1, \dots, x_N)$ for the unknowns at iteration n, and $\mathbf{F}(\mathbf{x}_n) = \big(f_1(\mathbf{x}_n), \dots, f_N(\mathbf{x}_n)\big)^T$. The Jacobian becomes the general $N \times N$ matrix of partial derivatives

$$J(\mathbf{x}_n) = \begin{pmatrix} \dfrac{\partial f_1}{\partial x_1} & \cdots & \dfrac{\partial f_1}{\partial x_N} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial f_N}{\partial x_1} & \cdots & \dfrac{\partial f_N}{\partial x_N} \end{pmatrix}, \tag{2.22}$$

and the update rule

$$\mathbf{x}_{n+1} = \mathbf{x}_n - J^{-1}(\mathbf{x}_n)\mathbf{F}(\mathbf{x}_n) \tag{2.23}$$

derived above for two equations is unchanged in form for any N.

As in the 1D case, Newton's method for systems converges quadratically provided the initial estimate is sufficiently close to the root and the Jacobian J is *non-singular* there – the direct multivariate analogue of the 1D convergence theorem in Section 2.5.

2.5.3 2D Newton Raphson example

Consider finding the roots of the two functions

$$f(x, y) = 9 - x^4 - y^4$$
$$g(x, y) = x^2 - 2y + 2$$

starting at

$$\mathbf{x}_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

The Jacobian is

$$\begin{pmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{pmatrix} = \begin{pmatrix} -4x^3 & -4y^3 \\ 2x & -2 \end{pmatrix}$$

The first step of the 2D Newton-Raphson method is:

$$\mathbf{x}_1 = \mathbf{x}_0 - J^{-1}(\mathbf{x}_0)F(\mathbf{x}_0)$$
$$\begin{pmatrix} x_1 \\ y_1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} - \frac{1}{8}\begin{pmatrix} -1 & 2 \\ -1 & -2 \end{pmatrix}\begin{pmatrix} 7 \\ 1 \end{pmatrix} = \begin{pmatrix} 1.625 \\ 2.125 \end{pmatrix}$$

In python this can be calculated via

```
import numpy as np # Import NumPy library

x0 = np.array([[1], [1]]) # Define x0 as a column vector. Note the
                          # arrangement of square brackets, as it
                          # uses similar syntax to the matrix
J = np.array([[-4, -4], [2, -2]]) # Define J as a 2 by 2 matrix

# Calculate inverse of J using the linalg inverse function
x1 = x0 - np.linalg.inv(J) @ np.array([[7], [1]])

np.set_printoptions(precision=4)
print(x1) #Print result to 4 dp
```

Continuation of interation can be done in python by defining the functions and first dervatives as inline (also called anonymous) functions (using the python lambda command)

```
dfdx = lambda x: -4 * x**3 # df/dx (x) = -4 x^3
dfdy = lambda y: -4 * y**3
dgdx = lambda x: 2 * x
f = lambda x, y: 9 - x**4 - y**4 # f(x,y) = 9 - x^4 - y^4
g = lambda x, y: x**2 - 2*y + 2
```

so that they can be evaluated easily at each iteration step. The 2D Newton Raphson becomes

```
import numpy as np
# Initial guess
x0 = np.array([[1], [1]])

# Iteration loop
for n in range(5):
    J = np.array([[dfdx(x0[0, 0]), dfdy(x0[1, 0])],
                  [dgdx(x0[0, 0]), -2]])

    F = np.array([[f(x0[0, 0], x0[1, 0])],
                  [g(x0[0, 0], x0[1, 0])]])

    x1 = x0 - np.linalg.inv(J) @ F # Newton-Raphson update
    x0 = x1 # Shift for the next step

np.set_printoptions(precision=4)
print(x0) #Print result to 4 dp
```

Note that x and y appear as $x0[0], x0[1]$ in the code. The method returns the root

$$\begin{pmatrix} 1.1363 \\ 1.6456 \end{pmatrix}$$

This can be checked in python

```
print(f(x0[0],x0[1]))
print(g(x0[0],x0[1]))
```

where both are close to zero.

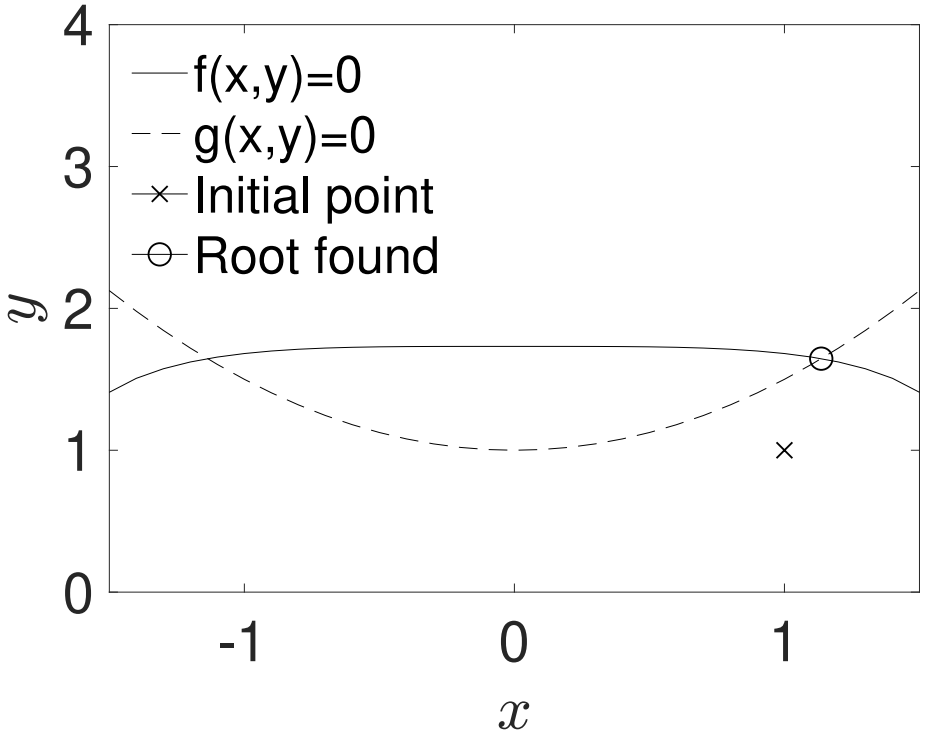


Figure 2.10: The zero lines of the functions f and g in the 2D Newton Raphson example. Also shown are the initial point and the root found in the example.

Chapter 3

Ordinary differential equations

Historical figures

When John Glenn was preparing to become the first American to orbit the Earth in 1962, he personally requested that **Katherine Johnson** (1918–2020) re-verify by hand the trajectory that NASA's electronic computers had calculated, before he would trust it with his life. Johnson, a mathematician at NASA and its predecessor NACA, spent her career hand-calculating spacecraft trajectories by numerically integrating the equations of motion — including with Euler's method, the oldest scheme in this chapter, dating back to **Leonhard Euler** (1707–1783) in 1768. The higher-order schemes she and NASA also relied on were developed later: **Carl Runge** (1856–1927) and **Martin Kutta** (1867–1944) independently extended Euler's idea around 1900 into the family of methods that bears their names, including the classical RK4 scheme, while **John Couch Adams** (1819–1892, better known for co-predicting the existence of Neptune) developed the multi-step methods later refined into the Adams–Bashforth and Adams–Moulton families. Johnson, Euler, Runge, Kutta, Adams on Wikipedia.

The key aim of this chapter is to deepen your understanding of numerical methods for ordinary differential equations. As with the root finding chapter, there will be some revision of year 1 material, but with overall aim of deepening your understanding. Since differential equations form a significant part of applied mathematics problems, developing a sound understanding of these methods will equip you well for future projects and modules. The key concepts that I want to emphasise are the *accuracy* and *stability*; for example you may have a very accurate scheme that has poor stability properties, and thus of little use as a solver. Before we consider the numerical solution of ordinary differential equations (ODEs), we will identify the key types of error that affect their solution.

3.1 Numerical accuracy and the truncation error

The accuracy of a numerical scheme is often evaluated in the context of the derivative/s being evaluated for the problem. For example, discretising in x with an interval of h gives

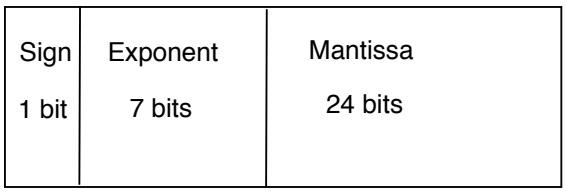


Figure 3.1: An illustration of the storage for a 32 bit computer

for the first derivative

$$f'(x) = \mathcal{N} + \overbrace{O(h^m)}^{\text{Truncation error}},$$

where $\mathcal{N}$ symbolises the numerical approximation and the $O(h^m)$ is 'order' notation for the truncation error (sometimes also called the global truncation error). The 'order' notation enables us to state the functional dependence with h without the specific coefficients. It also identifies the *smallest* power in the remainder, as it this will dominate when h becomes small. The exponent m is often given as an integer and represents the degree of the scheme. This formula can also be viewed as giving the rate of convergence of the numerical to the true derivative; the higher m means faster convergence as h becomes small. So a first order scheme for $f'(x)$ has $m = 1$ and would be

$$f'(x) = \mathcal{N} + O(h),$$

A second order scheme has $m = 2$ and would be

$$f'(x) = \mathcal{N} + O(h^2).$$

The order of the scheme control the rate at which $f'(x)$ converges to $\mathcal{N}$ with as $h \to 0$. Thus a first-order scheme converges more slowly than a second-order scheme and is thus considered less accurate. It is important that the derivative is in the above form to get the correct truncation error. If we had instead

$$hf'(x) = \mathcal{N} + O(h)$$

this would give the incorrect truncation error and order of accuracy. Here, we have introduced the truncation error in general terms. Specific examples will follow in this chapter that will help clarify your understanding of its definition. For example, in time integration problems, $h = \Delta t$, the timestep, and for integration with respect to x, $h = \Delta x$, the grid length. It is important to be familiar with the terms timestep and grid length as we will use them frequently in this module as they are more specific than simply using h.

3.2 Round-off error

The other important type of error is round-off error. For simplicity we will consider an (old) 32 bit computer. Figure 3.1 illustrates the storage. An example of a number in this form is

$$\overbrace{-}^{\text{Sign}}\ \overbrace{0.8125}^{\text{Mantissa}} \times 2^{\overbrace{2}^{\text{Exponent}}}$$

Since the mantissa is limited to 24 bits, there are certain numbers smaller than a certain size ϵ that the computer cannot represent. ϵ is referred to as the round-off error. We can find the specific value of ϵ in python. For my computer I find

```
import numpy as np
epsilon=np.spacing(1)
epsilon_format=format(epsilon,".2e")
print(epsilon_format)
2.22e-16
```

Whilst this seems a very small number, it can become important for cases of numerical instability. Numerical instability can cause the round off error to grow to such a size that it could corrupt the solution. This effect also applies to truncation errors.

3.3 Finite difference schemes ♢

We first construct Taylor series for $f(x+h)$ and $f(x-h)$ for deviations h from x

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + O(h^4) \tag{3.1}$$

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(x) + O(h^4) \tag{3.2}$$

Note that we have bundled all the terms that have been left out into the $O(h^4)$; only the h^4 term is given as this will dominate all higher powers as h becomes small.

3.3.1 First derivative ♢

By rearranging the Taylor series for $f(x+h)$, Eq. 3.1, to obtain the Forward Euler formula

$$f'(x) = \frac{f(x+h) - f(x)}{h} + O(h). \tag{3.3}$$

Similarly, by rearranging the Taylor series for $f(x-h)$, Eq. 3.2, we obtain the Backward Euler formula

$$f'(x) = \frac{f(x) - f(x-h)}{h} + O(h). \tag{3.4}$$

Both equations are first order accurate i.e.the leading truncation error scales like h to the power 1 as $h \to 0$.

By taking Eq. 3.1 minus Eq. 3.2 we obtain the centred difference formula

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + O(h^2). \tag{3.5}$$

This scheme is second order accurate. By combining the Taylor series for $f(x+2h)$, $f(x+h)$, $f(x-h)$, and $f(x-2h)$, we obtain the fourth-order centred difference formula

$$f'(x) = \frac{-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)}{12h} + O(h^4). \tag{3.6}$$

Note the trade off here: the greater accuracy is achieved by large 'stencils' of function evaluation.

3.3.2 Second derivative ◊

By taking Eq. 3.1 plus Eq. 3.2 we obtain the centred difference formula we can obtain a formula for the second derivative

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + O(h^2). \tag{3.7}$$

The coefficients 1, -2 and 1 are common in numerics and form the components of a tridiagonal matrix. See boundary-value problems later.

3.4 Timestepping problems

A timestepping scheme is a method of integrating an ODE in time in discrete steps. We can use some of the finite difference formulas derived previously as the basis for timestepping schemes. Suppose we want to integrate in time the ODE

$$\frac{dx}{dt} = F(x,t)$$

from some initial condition $x(0) = x_0$. Here, we will sometimes use

$$F(x,t) = F(x)$$

or at other times the full $F(x,t)$. We want to find the solution values at regularly spaced time steps $t_n = n\Delta t$, $n = 0,\ 1,\ 2,\ \ldots$ with Δt a constant.

3.5 The concept of numerical stability

A very important concept in our study of ODEs (and many other numerical methods) is numerical stability. Together with numerical accuracy, these form important 'pillars' of numerical methods. For example, if a numerical method is very accurate yet numerically unstable, it is of no use to use in practical integration problems. For very big numerical problems, for example forecasting the weather and climate, solving numerical stability problems can save significant time and money.

Figures 3.2 and 3.3 compare numerically stable and unstable time-stepping scenarios. It is important to note that our assessment of stability is relative to the true solution (marked with red dots). Other scenarios may have a different true solution. When the numerical solution "shadows" the true solution (Fig. 3.2), then it is considered numerically stable (even if it is not completely accurate). By shadowing we mean that the numerical solution decreases in the same way as the true solution; this doesn't mean that the numerical and true solutions are the same. In contrast, when the numerical solution diverges significantly from the true solution, it is numerically unstable. For example, in Fig. 3.3 the numerical

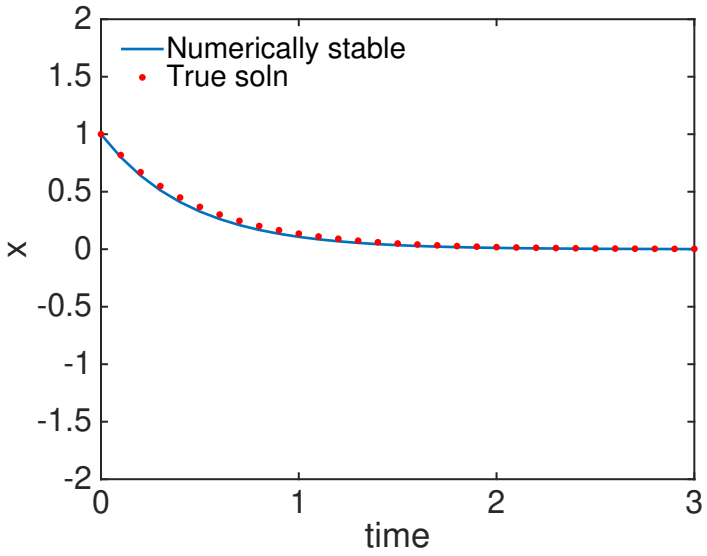


Figure 3.2: An example of a numerically *stable* solution (blue) compared to the true solution (red dots)

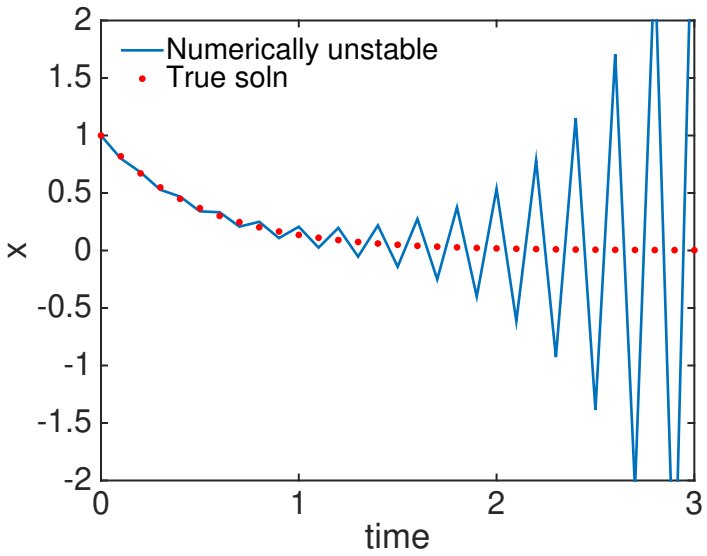


Figure 3.3: An example of a numerically *unstable* solution (blue) compared to the true solution (red dots)

solution both oscillates with time step and grow unbounded in magnitude. It can no longer be considered to shadow the true solution.

The numerical instability 'breeds' on very small round-off and trunctation errors, until such a point when these errors have corrupted the solution. Numerical instability is often the number one thing people want to fix in industrial applications of numerical methods.

3.5.1 Recursive example

We consider a simple recursive example of numerical instability to understand how small initial errors can corrupt a solution when instability is present. Consider the recursion relation

$$3x_n = 10x_{n-1} - 3x_{n-2} \tag{3.8}$$

with initial points $x_0 = 1$, $x_1 = \frac{1}{3}$. There is a solution

$$x_n = 3^{-n}$$

which decays with increasing n. Lets check by substitution into the right hand side of Eq. 3.8 to get

$$10.3^{-n+1} - 3.3^{-n+2} = (30 - 27)3^{-n} = 3.3^{-n} = 3x_n$$

which is the same as the left hand side of Eq. 3.8. There is also a growing solution

$$x_n = 3^n.$$

Lets check again

$$10.3^{n-1} - 3.3^{n-2} = (10/3 - 3/9)3^n = 3.3^n = 3x_n$$

The general solution is the superposition of the two previous solutions

$$x_n = A3^{-n} + B3^n$$

where A and B are constants set by the initial conditions. Now our chosen initial conditions mean that

$$A = 1, \quad B = 0, \quad x_n = 3^{-n}$$

so the *true* solution has no projection on the growing solution. However, if we are to include a round-off error ϵ in the initial conditions.

$$x_0 = 1 + \epsilon, \quad x_1 = \frac{1}{3} + 3\epsilon.$$

then

$$A = 1, \quad B = \epsilon, \quad x_n = 3^{-n} + \overbrace{\epsilon 3^n}^{\text{Parasitic solution}}$$

There is now a small numerical projection onto a growing mode. Although initially small, the parasitic solution (or computational mode) stimulated by the round-off error will grow unbounded and eventually corrupt the decaying solution. We will come across the computational mode again when we consider leapfrog timestepping schemes. We can solve this problem in python

```
a=1
b=1/3 # initial conditions
n=2
while n<40: # run for 40 iterations
   c=(10*b-3*a)/3 # recursion relation
   print( n, c, 3**(-n)) # display n, x_n and true solution

   a=b # shift for next step
   b=c # shift for next step
   n=n+1 # increase n
```

where a is x_{n-2}, b is x_{n-1} and c is x_n. You will see that the numerical and true solutions will diverge after about 15 steps. This is due to the intrinsic round-off error in your computer.

3.6 Forward Euler scheme ♢

If we use the forward Euler formula (using Δt in the place of h) to approximate the time derivative and confine ourselves to $F(x,t) = F(x)$ in Eq. 3.4, then we obtain

$$\frac{dx}{dt} = \frac{x_{n+1} - x_n}{\Delta t} + O(\Delta t) = F\left(x_n\right)$$

where the subscript n means at timestep n. Note that the subscript has a different meaning from when it is used in root-finding methods. This scheme is first order accurate, meaning that if we substitute the true solution into the finite difference equation the residual is order Δt to the power 1.

However, a given order of accuracy does not necessarily imply that the scheme will work well in practice; it must also be *stable*. There are many definitions of stability used in numerical analysis. Here we will adopt a simple and practical one: the numerical solution shadows (or follows) the true solution. One example of shadowing is when the numerical solution decays if the true solution also decays.

3.6.1 Exponential decay ♢

Consider a simple example in which $F(x) = -\lambda x$ with λ real and positive. Then the true solution is

$$x_* = x_0 \exp(-\lambda t)$$

where x_0 is a constant set by the initial conditions. The true solution decays with time. How does the numerical solution behave? The numerical scheme becomes

$$\frac{x_{n+1} - x_n}{\Delta t} = -\lambda x_n,$$

or rearranging to

$$x_{n+1} = (1 - \lambda \Delta t)\, x_n.$$

Thus the numerical solution changes by a factor $A = (1 - \lambda \Delta t)$ at each step. A is called the *amplification factor*.

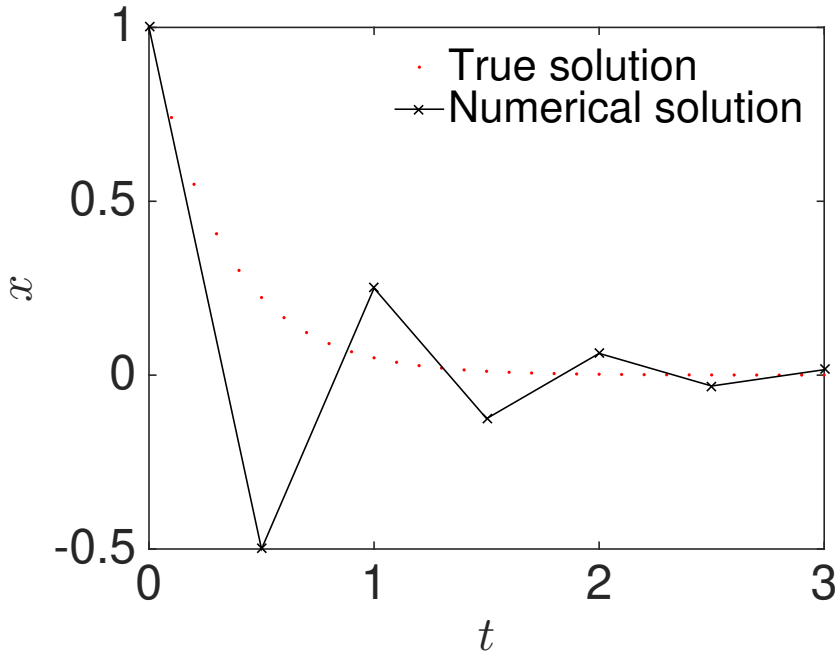


Figure 3.4: A comparison of numerical and true solutions for the case $1 < \lambda\Delta t < 2$

- If $\lambda\Delta t < 1$ then $0 < A < 1$; the numerical solution is multiplied by the amplification factor A at each step, giving a decaying numerical solution, so the scheme is stable.

- If $1 < \lambda\Delta t < 2$ then $-1 < A < 0$; the numerical solution still decays in amplitude but oscillates in sign. See Fig. 3.4 for an example. The scheme is still stable, but not very accurate.

- If $2 < \lambda\Delta t$ then $A < -1$; the numerical solution oscillates and grows. The scheme is now unstable. See Fig. 3.5 for an example.

For this problem the forward Euler scheme is *conditionally stable*: its stability depends on the size of Δt.

3.6.2 Numerical instability example

Let's solve the problem using some specific numbers in order to understand further the concept of numerical instability. If we consider the case where $\lambda = 100$ and $\Delta t = 0.1$, this should be the case where $\lambda\Delta t > 2$ in the above analysis.

$$\dot{x} = -\lambda x = -100x$$

Assuming $x_0 = 1$, the true solution is

$$x_* = e^{-100t}$$

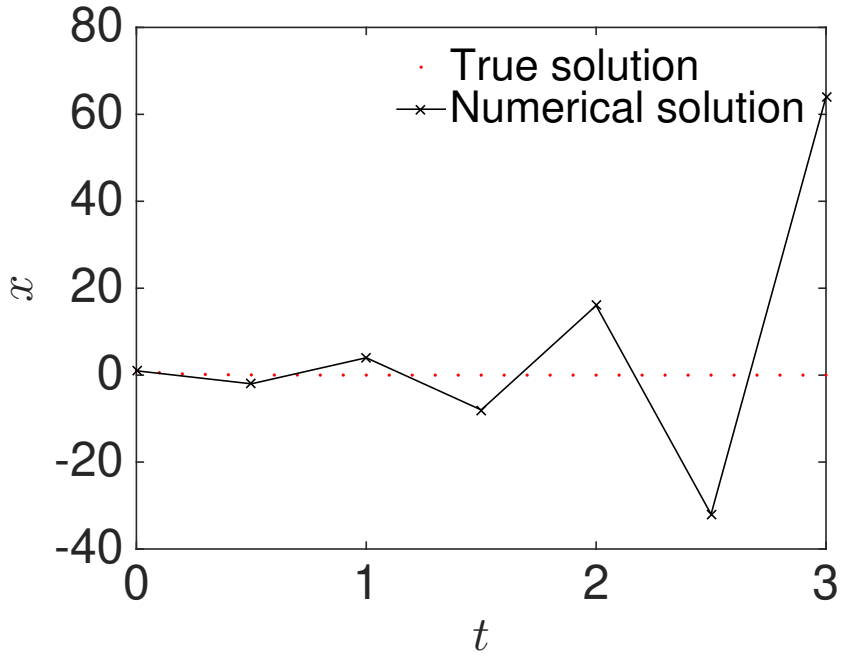


Figure 3.5: A comparison of numerical and true solutions for the case $\lambda\Delta t > 2$. Note the expanded y axis compared to Fig. 3.4.

so when $t = 1$, $x_* = e^{-100}$. Now to the numerical solution. We know that $x_0 = 1$, so from Forward-Euler formula

$$\begin{aligned} x_1 &= (1 - \lambda\Delta t)x_0 \\ &= (1 - 100.0.1).1 \\ &= -9 \end{aligned}$$

If we repeat this another 9 times (for example in python or with a calculator) we find

$$x_{10} = 3.5 \times 10^9$$

which has diverged significantly from the true solution of $x_* = e^{-100}$. By comparing with the true solution, we see that the numerical solution is definitely unstable. The problem arises from the timestep not being sufficiently small for the given value of λ. The numerical method is thus unable to capture the timescale in the true solution.

3.6.3 Oscillation ♢

Equations with oscillating solutions are also of interest. When $F(x) = i\omega x$ with ω real, the ODE becomes

$$Re\left\{\frac{dx}{dt}\right\} = Re\{i\omega x\} \tag{3.9}$$

and the true solution now is

$$Re\{x_*\} = Re\{x_0 \exp(i\omega t)\} \tag{3.10}$$

where x, x_0 and x_* are complex. Often the $Re\{\quad\}$ notation is dropped for ease of calculation. In general, x_0 is defined as

$$x_0 = a + ib,$$

where a and b are constants. The true solution can be expanded as

$$Re\{x_*\} = Re\{(a + ib)(\cos\omega t + i\sin\omega t)\} = a\cos\omega t - b\sin\omega t.$$

Equations (3.9) and (3.10) are thus a compact way of representing cosine and sine solutions. The values of a and b can be determined from the initial conditions. For example, consider the initial condition $Re\{x_*(t=0)\} = 1$, $Im\{x_*(t=0)\} = 0$, then this is satisfied by $a = 1$ and $b = 0$ giving the solution $x_* = \cos\omega t$. The key point is that the complex modulus of the true solution in Eq. (3.10) remains constant,

$$|x_*| = |x_0||\exp(i\omega t)| = |x_0| = (a^2 + b^2)^{1/2}$$

meaning that the true solution is sinusoidal (a combination of sin and cos) in time with constant amplitude. The constant amplitude is a key feature, against which we will compare the numerical solution. Recall that the complex modulus of a complex number $z = c + id$ is

$$|z| = (c^2 + d^2)^{1/2}$$

Given the complex amplification factor takes the numerical solution across each timestep, the ideal situation will be when the complex modulus of the amplification factor is unity. When the complex modulus of the amplification factor is unity, this means that oscillating numerical solution maintains constant amplitude in time. The numerical solution in terms of the amplification factor is

$$x_n = A^n x_0.$$

Taking the complex modulus gives

$$|x_n| = |A|^n |x_0|.$$

Thus, if the complex modulus of A is unity ($|A| = 1$) then the complex modulus of the numerical solution is the same as the true solution. Note that the criterion for stability has changed from the decaying case as we now have a different true solution. The amplification factor for the forward Euler scheme is

$$A = (1 + i\omega\Delta t)\,,$$

where we have substituted $i\omega$ for $-\lambda$ in the previous formula. The complex modulus is

$$|A| = (1 + \omega^2\Delta t^2)^{1/2}.$$

Thus, $|A| > 1$, so the forward Euler scheme is *unconditionally unstable* for this type of equation even though it was stable for the decaying case.

3.7 Implicit schemes

Improved stability can often be obtained by using implicit schemes like the *backward Euler scheme*

$$\frac{x_{n+1} - x_n}{\Delta t} = F\left(x_{n+1}\right).$$

The 'implicit' aspect is introduced by the change from evaluating F using x_{n+1} instead of x_n. The implicit aspect means the equations need to be rearranged so that x_{n+1} is the subject. Let us return to the exponential decay problem $\dot{x} = -\lambda x$. We can now derive the amplification factor.

$$\begin{aligned} x_{n+1} &= x_n - \lambda \Delta t x_{n+1} \\ (1 + \lambda \Delta t) x_{n+1} &= x_n \\ x_{n+1} &= \frac{x_n}{1 + \lambda \Delta t} \\ A = \frac{x_{n+1}}{x_n} &= \frac{1}{1 + \lambda \Delta t} \end{aligned} \tag{3.11}$$

Compared to Forward Euler, where $|A| > 1$ for certain values of Δt, here $|A| < 1$ for any Δt . The implicit method is *unconditionally* stable. Applying the amplification factor recursively N times gives

$$x_N = \frac{x_0}{(1 + \lambda \Delta t)^N}$$

for the Backward Euler method. As $N \to \infty$ $x_N \to 0$ in precisely the same way as the true solution. In contrast, the forward Euler method would give

$$x_N = x_0 (1 - \lambda \Delta t)^N$$

and $x_N \to 0$ as $N \to \infty$ only for suitably small values of Δt.

The backward Euler method is still only first-order accurate. In order to improve the accuracy, whilst retaining some implicit properties, the *trapezoidal implicit scheme* or *Crank-Nicolson scheme* is used

$$\frac{x_{n+1} - x_n}{\Delta t} = \frac{1}{2}\left[F(x_n) + F(x_{n+1})\right],$$

which is second order accurate (see Adams-Moulton method later). Schemes similar to these are often used in modelling fast process which would require unfeasibly short time steps with an explicit scheme.

So far we have considered first-order accurate time stepping schemes. We now consider schemes that improve on this degree of accuracy.

3.8 Leapfrog scheme

If we approximate the time derivative by a centred difference scheme we get leapfrog scheme

$$\frac{dx}{dt} = \frac{x_{n+1} - x_{n-1}}{2\Delta t} + O(\Delta t^2) = F(x_n).$$

The leapfrog scheme is second order accurate, and is so named as the scheme "leapfrogs" over step n in the differencing. We will now consider the numerical stability of the leapfrog scheme for the exponential decay case. The amplification factor is a factor that transforms the numerical solution from one timestep to the next and so

$$A = \frac{x_{n+1}}{x_n} = \frac{x_n}{x_{n-1}}.$$

3.8.1 Decay case

Consider the exponential decay case, $F(x) = -\lambda x$. The equation for the amplification factor is then

$$\begin{aligned}
&\frac{x_{n+1} - x_{n-1}}{2\Delta t} = -\lambda x_n \\
\Rightarrow &\frac{x_{n+1}}{x_n} - \frac{x_{n-1}}{x_n} + 2\lambda\Delta t = 0 \\
\Rightarrow &A - \frac{1}{A} + 2\lambda\Delta t = 0 \\
\Rightarrow &A^2 + 2\lambda\Delta t A - 1 = 0 \\
\Rightarrow &A = -\lambda\Delta t \pm \sqrt{\lambda^2\Delta t^2 + 1}
\end{aligned}$$

The two roots of the quadratic equation are A_+ and A_- corresponding to the positive and negative sign before the square root. Taylor expanding the square root term in the expression for A^+ (using the Taylor series for $(1+x)^n$ using $x = \lambda^2\Delta t^2$ and $n = 1/2$) gives

$$\begin{aligned}
A_+ &= -\lambda\Delta t + \sqrt{\lambda^2\Delta t^2 + 1} \\
&= -\lambda\Delta t + 1 + \frac{\lambda^2\Delta t^2}{2} + O(\Delta t^4) \\
&= 1 - \lambda\Delta t + \frac{\lambda^2\Delta t^2}{2} + O(\Delta t^4) \\
&= e^{-\lambda\Delta t} + O(\Delta t^3) + O(\Delta t^4) \\
&= e^{-\lambda\Delta t} + O(\Delta t^3)
\end{aligned}$$

Applying the same process to A^- gives

$$\begin{aligned}
A_- &= -\lambda\Delta t - \sqrt{\lambda^2\Delta t^2 + 1} \\
&= -1 - \lambda\Delta t - \frac{\lambda^2\Delta t^2}{2} + O(\Delta t^4) \\
&= -e^{\lambda\Delta t} + O(\Delta t^3)
\end{aligned}$$

The general form of the numerical solution is

$$\begin{aligned}
x_n &= c_1 A_+^n + c_2 A_-^n \\
&\simeq c_1 e^{-\lambda n\Delta t} + (-1)^n c_2 e^{\lambda n\Delta t} \\
&= c_1 e^{-\lambda t_n} + (-1)^n c_2 e^{\lambda t_n}
\end{aligned}$$

for some constants c_1 and c_2 determined by the initial conditions. The first term is exponentially decay and shadows the true solution. The second term is an artefact of the numerical method and bears no relation to the true solution: it both oscillates in sign with each time step and grows exponentially; this term is called the parasitic mode or computational mode. For this exponential decay case, a small amount of numerical error in the initial conditions will project onto the computational mode, and it will grow unbounded and corrupt the numerical solution. Thus, for the exponential decay case (leapfrog behaves differently for other forms of $F(x)$), the leapfrog scheme is numerically unstable. So, whilst the forward Euler is less accurate than leapfrog for exponential decay, it is numerically stable whilst the leapfrog is not.

3.8.2 Oscillation case

The solution for the oscillation case involves substituting $i\omega$ for $-\lambda$ (or equivalently $-i\omega$ for λ) in the quadratic for A

$$\begin{aligned}
&A^2 - 2i\omega\Delta t A - 1 = 0 \\
\Rightarrow &A = \frac{2i\omega\Delta t}{2} \pm \frac{\sqrt{4i^2\omega^2\Delta t^2 + 4}}{2} \\
\Rightarrow &A = \frac{2i\omega\Delta t}{2} \pm \frac{\sqrt{-4\omega^2\Delta t^2 + 4}}{2} \\
\Rightarrow &A = i\omega\Delta t \pm \sqrt{-\omega^2\Delta t^2 + 1} \\
\Rightarrow &A = i\omega\Delta t \pm \sqrt{1 - \omega^2\Delta t^2}
\end{aligned}$$

Stable case

For the case of $\omega\Delta t \leq 1$, the square root is a real term, meaning the complex modulus squared involves summing the squares of the imaginary and real components

$$\begin{aligned}
&|A|^2 = \omega^2\Delta t^2 + 1 - \omega^2\Delta t^2 \\
\Rightarrow &|A|^2 = 1 \\
\Rightarrow &|A| = 1
\end{aligned}$$

Thus the leapfrog scheme is conditionally stable with the condition of $\omega\Delta t \leq 1$.

Unstable case

For the case of $\omega\Delta t > 1$, the square root is now imaginary giving the following calculation of the complex modulus

$$\begin{aligned}
&A = i\omega\Delta t \pm \sqrt{1 - \omega^2\Delta t^2} \\
\Rightarrow &A = i\omega\Delta t \pm \sqrt{-1}\sqrt{\omega^2\Delta t^2 - 1} \\
\Rightarrow &A = i(\omega\Delta t \pm \sqrt{\omega^2\Delta t^2 - 1})
\end{aligned}$$

The positive root gives the worst case in terms of stability

$$\begin{aligned} A_+ &= i(\omega\Delta t + \sqrt{\omega^2\Delta t^2 - 1}) \\ \Rightarrow |A_+|^2 &= (\omega\Delta t + \sqrt{\omega^2\Delta t^2 - 1})^2 \\ \Rightarrow |A_+| &= \omega\Delta t + \sqrt{\omega^2\Delta t^2 - 1} \end{aligned}$$

Since $\omega\Delta t > 1$, $\sqrt{\omega^2\Delta t^2 - 1} > 0$ and so

$$|A_+| > 1,$$

and is a parasitic growing mode.

3.9 Runge-Kutta method

Developed in the early 1900s by the German mathematicians Runge and Kutta (note this was well before computers as numerical methods were still important then for solving mathematical problems), this method improves on the accuracy of the forward Euler scheme by evaluating the forcing function ($F(x,t)$) at several intermediate points *within* the timestep. Here we will derive the second order Runge-Kutta method (RK2). The method involves two sets of Taylor expansions. The first is of the general numerical method (with undetermined constants). The second expansion is of the ODE or the true solution. Comparing the two Taylor expansions will help determine the constants in the numerical method.

We will first present the RK2 algorithm with free constants that need to be determined.

1. Compute $k_1 = F(x_n, t_n)$ at the start of the time interval.
2. Evaluate
$$x_n + a\Delta t k_1; \quad t_n + b\Delta t$$
where a and b are parameters to be determined.
3. Compute k_2, approximating $\Delta x = k_1\Delta t$
$$\begin{aligned} k_2 &= F(x_n + a\Delta x, t_n + b\Delta t) \\ &= F(x_n + a\Delta t k_1, t_n + b\Delta t) \end{aligned} \tag{3.12}$$
4. Compute new point
$$x_{n+1} = x_n + \Delta t(ck_1 + dk_2)$$
where c and d are to be determined. Note that the special case of $c = 1$, $d = 0$ simply becomes the Forward-Euler scheme.

Substituting for k_1 and k_2, and Taylor-expanding $F(x_n + a\Delta t k_1, t_n + b\Delta t)$ adapting the standard 2D Taylor series $f(x + \Delta x, t + \Delta t) = f + \frac{\partial f}{\partial x}\Delta x + \frac{\partial f}{\partial t}\Delta t$, gives

$$\begin{aligned} x_{n+1} &= x_n + \Delta t(cF + dF(x_n + a\Delta t F, t_n + b\Delta t)) \\ &= x_n + \Delta t\left(cF + dF + da\Delta t F\frac{\partial F}{\partial x} + db\Delta t\frac{\partial F}{\partial t}\right) + O(\Delta t^3) \\ &= x_n + (c+d)\Delta t F + d\Delta t^2\left(aF\frac{\partial F}{\partial x} + b\frac{\partial F}{\partial t}\right) + O(\Delta t^3) \end{aligned} \tag{3.13}$$

We will now determine a, b, c and d. The derivation involves a second Taylor expansion, this time of the true solution, using expressions for $\dot{x}$ and $\ddot{x}$ to get an equation in the same form as Equation (3.13) but without the free constants. Comparison will allow the determination of the constants.

Taylor expand the true solution in time

$$x_{n+1} = x_n + \Delta t\dot{x} + \frac{\Delta t^2}{2}\ddot{x} + O(\Delta t^3) \tag{3.14}$$

We next need to derive an expression for $\ddot{x}$. Recall that

$$\dot{x} = F(x,t) \tag{3.15}$$

Differentiating with respect to time and applying the (partial derivative) chain rule gives

$$\ddot{x} = \frac{d}{dt}F(x,t) = \frac{\partial F}{\partial x}\dot{x} + \frac{\partial F}{\partial t}$$

Substituting from the the original ODE ($\dot{x} = F$) gives

$$\ddot{x} = \frac{\partial F}{\partial x}F + \frac{\partial F}{\partial t} \tag{3.16}$$

Substituting Eq. (3.15) and Eq. (3.16) into Eq. (3.14) gives

$$x_{n+1} = x_n + \Delta tF + \frac{\Delta t^2}{2}\left(\frac{\partial F}{\partial x}F + \frac{\partial F}{\partial t}\right) + O(\Delta t^3) \tag{3.17}$$

Note that although the remaining term is $O(\Delta t^3)$, the truncation error applies to the time derivative $\frac{x_{n+1}-x_n}{\Delta t}$ so involves dividing through by Δt giving an $O(\Delta t^2)$ truncation error. Comparing Equations (3.13) and (3.17) gives the coefficients

$$c + d = 1; \quad da = db = \frac{1}{2}$$

We still have the freedom to choose one constant. The most common choice is

$$c = d = \frac{1}{2}; \quad a = b = 1$$

giving the standard 'RK2' scheme, sometimes also called Heun's scheme or the improved Euler method:

$$\begin{aligned} k_1 &= F(x_n, t_n), \\ k_2 &= F(x_n + \Delta t k_1, t_n + \Delta t), \\ x_{n+1} &= x_n + \frac{\Delta t}{2}(k_1 + k_2). \end{aligned}$$

An alternative is the mid-point method where

$$c = 0; \quad d = 1; \quad a = b = \frac{1}{2}$$

$$\begin{aligned} k_1 &= F(x_n, t_n), \\ k_2 &= F(x_n + \frac{\Delta t}{2}k_1, t_n + \frac{\Delta t}{2}), \\ x_{n+1} &= x_n + \Delta t k_2. \end{aligned}$$

3.9.1 Worked example

In order to demonstrate the Runge-Kutta method, we will now introduce a new ODE example, where the right-hand side is a function of both x and t

$$\frac{dx}{dt} = F(x,t) = 2t - x \tag{3.18}$$

or for convenience of finding the true solution

$$\frac{dx}{dt} + x = 2t, \tag{3.19}$$

with initial conditions $x(0) = -1$. Using a method from the differential equations module, we multiply both sides of Eq. (3.19) by an integrating factor, I, where

$$I = e^t$$

Equation (3.19) becomes

$$\begin{aligned} I\frac{dx}{dt} + Ix &= I2t \\ \Rightarrow e^t\frac{dx}{dt} + e^t x &= e^t 2t \end{aligned}$$

The left hand side is now the derivative of $e^t x$. Integrating both sides and using integration by parts for the right hand side gives

$$\begin{aligned} e^t x &= 2\int te^t dt + A \\ \Rightarrow e^t x &= 2te^t - 2\int e^t dt + A \\ \Rightarrow e^t x &= 2te^t - 2e^t + A \\ \Rightarrow x &= Ae^{-t} + 2t - 2 \end{aligned}$$

where A is a constant of integration.

Applying the initial condition gives $A = 1$ and so the true solution is

$$x_* = e^{-t} + 2t - 2$$

When $t = 1$, $x_* = e^{-1}$. Now, let's see how the mid-point form of the Runge-Kutta method performs against this true solution. $x_0 = -1$, $\Delta t = 0.1$.

$$\begin{aligned} k_1 &= F(x_0, 0) = F(-1, 0) = 1 \\ k_2 &= F(x_0 + k_1\Delta t/2, \Delta t/2) = F(-1 + 0.05, 0.05) = F(-0.95, 0.05) = 0.1 + 0.95 = 1.05 \\ x_1 &= x_0 + k_2\Delta t \\ &= -1 + 1.05 \times 0.1 = -0.895 \end{aligned}$$

Method	Trunction error	Function evaluations per timestep
Forward Euler	Δt	1
RK2	Δt^2	2
RK4	Δt^4	4

Table 3.1: A comparison of truncation error and number of function evaluations for three of the methods discussed.

Iterating a further 9 times gives at $t = 1$

$$x_{10} = 0.3685$$

where the true solution is $e^{-1} = 0.3678$. For reference the Forward Euler method gives:

$$\begin{aligned} x_1 &= x_0 + k_1 \Delta t \\ &= -1 + 0.1 \times 0.1 = -0.9 \\ x_{10} &= 0.3487, \end{aligned} \tag{3.20}$$

significantly further from the true solution compared to the mid-point method due to its first-order accuracy.

3.9.2 4th order Runge-Kutta (RK4)

Here RK2 is extended to four coefficients by expanding the Taylor series for x_{n+1} up to $O(\Delta t^5)$ instead of $O(\Delta t^3)$,

$$\begin{aligned} k_1 &= F(x_n, t_n), \\ k_2 &= F\left(x_n + \frac{\Delta t}{2} k_1, t_n + \frac{\Delta t}{2}\right), \\ k_3 &= F\left(x_n + \frac{\Delta t}{2} k_2, t_n + \frac{\Delta t}{2}\right), \\ k_4 &= F\left(x_n + \Delta t k_3, t_n + \Delta t, \right). \end{aligned}$$

Another name for Runge-Kutta methods is a type of 'multi-stage' method, where the evaluation of k_1, k_2 etc is a different stage. The timestepping formula then combines these stages as

$$x_{n+1} = x_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) + O(\Delta t^5).$$

The coefficients are chosen so that the timestepping method now has a $O(\Delta t^4)$ truncation error (when this formula is divided through by Δt to form the time derivative of x_n) i.e. is fourth-order accurate. The extra accuracy comes at the expense of the additional function evaluations as indicated in Table 3.1.

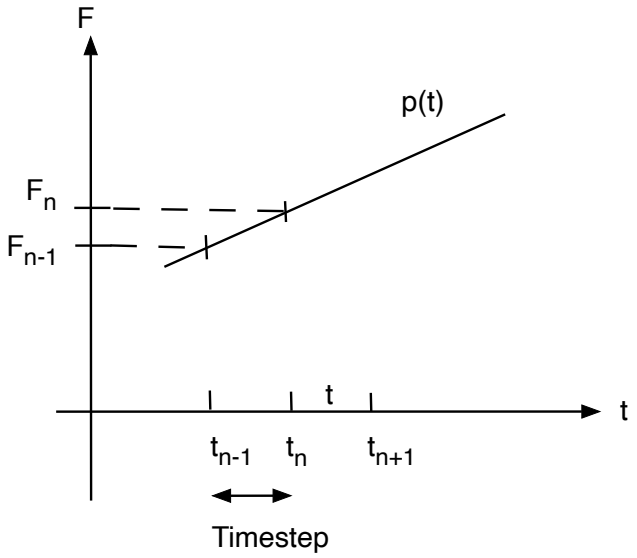


Figure 3.6: The polynomial approximation for the second-order Adams-Bashforth method.

3.10 Multi-step methods

A disadvantage of the Runge-Kutta method is that extra function evaluations need to be calculated within the timestep. The different stages of function evaluations means that Runga-Kutta methods are termed a *multi-stage* method. In contrast, multi-step methods evaluate the forcing function F at existing time steps.

The second order Adams-Bashforth method (AB2) is

$$x_{n+1} = x_n + \frac{\Delta t}{2}(3F_n - F_{n-1}),$$

where we have used the contraction/shorthand $F_n = F(x_n, t_n)$ (given n defines both x_n and t_n, we can simply use n to identify them). The derivation of this method involves using a polynomial approximation for the forcing function F. For AB2, we use a straight line approximation (p) of F, illustrated in Fig. 3.1. Higher order methods use successively higher order polynomials. The polynomial is defined as

$$p = F_{n-1} + \frac{(F_n - F_{n-1})}{\Delta t}(t - t_{n-1})$$

Integrating the ODE over one timestep gives

$$\begin{aligned} x_{n+1} - x_n &= \int_{t_n}^{t_{n+1}} F(x,t)dt \simeq \int_{t_n}^{t_{n+1}} p dt \\ &= \int_{t_n}^{t_{n+1}} F_{n-1} + \frac{(F_n - F_{n-1})}{\Delta t}(t - t_{n-1})dt. \end{aligned} \tag{3.21}$$

Using the dummy variable $s = t - t_{n-1}$, so that $ds = dt$ and $t_n - t_{n-1} = \Delta t$ gives

$$\begin{aligned} x_{n+1} - x_n &= \int_{t_n - t_{n-1}}^{t_{n+1} - t_{n-1}} F_{n-1} + \frac{(F_n - F_{n-1})}{\Delta t} s ds \\ &= \int_{\Delta t}^{2\Delta t} F_{n-1} + \frac{(F_n - F_{n-1})}{\Delta t} s ds \\ &= \left[F_{n-1} s + \frac{(F_n - F_{n-1})}{\Delta t} \frac{s^2}{2} \right]_{\Delta t}^{2\Delta t} \\ &= F_{n-1} \Delta t + \frac{(F_n - F_{n-1})}{\Delta t} \frac{3\Delta t^2}{2} \\ &= \frac{\Delta t}{2} (3F_n - F_{n-1}) \end{aligned} \tag{3.22}$$

3.10.1 Worked example

We repeat the problem in Equation 3.18 using the AB2 method. Given that these methods require more than one step, at initial time we use an Euler or RK method to compute the first set of steps. The first step we take from the Forward-Euler method

$$x_1 = x_0 + \Delta t F(x_0, 0) = x_0 + \Delta t F_0 = -1 + 0.1 = -0.9$$

For the next step we use the AB2

$$\begin{aligned} &x_2 = x_1 + \frac{\Delta t}{2}(3F_1 - F_0) \\ &F_1 = F(x_1, t_1) = F(x_1, \Delta t) = 2\Delta t - x_1 = 2 \times 0.1 + 0.9 = 1.1 \quad F_0 = 1 \\ &x_2 = -0.9 + 0.05(3 \times 1.1 - 1) = -0.785 \end{aligned} \tag{3.23}$$

Repeat a further 9 times gives:

$$x_{10} = 0.3675$$

This result is comparable to the RK2 method which is also second order accurate.

3.10.2 Third order AB method

A third-order accurate AB method, AB3, can be derived using a quadratic curve instead of a straight line for the polynomial in the derivation. The resulting formula is

$$x_{n+1} = x_n + \frac{\Delta t}{12}(23F_n - 16F_{n-1} + 5F_{n-2}).$$

The resulting truncation error is $O(\Delta t^3)$. The extra accuracy compared with AB2 comes with the trade off of now having to evaluate an extra function value, F_{n-2}.

3.10.3 Adams-Moulton methods

The Adams-Moulton (AM) method is an implicit extension of the Adams-Bashforth method. The second order AM2 method is given by

$$x_{n+1} = x_n + \frac{\Delta t}{2}(F_n + F_{n+1}),$$

where function evaluations are now made at step $n+1$ on the right hand side. We will now derive the AM2 method. Justs as with AB2, we define a polynomial approximation to F but using the $n+1$ values instead

$$p = F_n + \frac{(F_{n+1} - F_n)}{\Delta t}(t - t_n)$$

Integrating with respect to time using $s = t - t_n$ give s

$$\begin{aligned}
x_{n+1} - x_n &= \int_{t_n}^{t_{n+1}} F_n + \frac{(F_{n+1} - F_n)}{\Delta t}(t - t_n)dt \\
&= \int_0^{\Delta t} F_n + \frac{(F_{n+1} - F_n)}{\Delta t} s ds \\
&= \left[F_n s + \frac{(F_{n+1} - F_n)}{\Delta t} \frac{s^2}{2} \right]_0^{\Delta t} \\
&= F_n \Delta t + \frac{(F_{n+1} - F_n)}{\Delta t} \frac{\Delta t^2}{2} \\
&= \frac{\Delta t}{2}(F_{n+1} + F_n) \qquad (3.24)
\end{aligned}$$

which is the AM2 formula. We also see that this scheme is identical to the trapezoidal implicit method. The polynomial derivation here gives a reason for the second-order accuracy of both schemes.

3.10.4 Predictor-corrector methods

The AM2 formula just derived is *implicit*: $F_{n+1} = F(x_{n+1}, t_{n+1})$ depends on the very unknown x_{n+1} we are trying to find. How do we actually evaluate it in practice?

The standard solution is to pair the implicit (AM) formula with an explicit (AB) one. First use AB2 to *predict* a provisional value $x_{n+1}^{(0)}$; then substitute this into the AM2 formula to *correct* it,

$$x_{n+1}^{(k+1)} = x_n + \frac{\Delta t}{2}\left(F_n + F\left(x_{n+1}^{(k)}, t_{n+1}\right)\right), \quad k = 0, 1, 2, \ldots \qquad (3.25)$$

This can, in principle, be iterated repeatedly (re-evaluating F at each new corrected value) until successive estimates agree to within a chosen tolerance, but in practice a single correction is normal: this is called *PECE* (Predict, Evaluate, Correct, Evaluate) after its four steps – predict $x_{n+1}^{(0)}$, evaluate F there, correct to get x_{n+1}, then evaluate F once more at x_{n+1} so that the result is cached and ready for the *next* step's predictor.

The pairing above uses AB2 and AM2, both second order, to match the earlier derivations in this chapter. For a fair comparison against RK4 (fourth order), we pair same-order predictor and corrector instead: AB4 and AM4. Following exactly the same polynomial-fitting derivation as before, but fitting a cubic through the four points $F_{n-3}, \ldots, F_n$ instead of the straight line used for AB2, gives the fourth-order Adams-Bashforth formula

$$x_{n+1}^{(0)} = x_n + \frac{\Delta t}{24}\left(55F_n - 59F_{n-1} + 37F_{n-2} - 9F_{n-3}\right), \tag{3.26}$$

and fitting the analogous cubic through $F_{n-2}, \ldots, F_{n+1}$ (as for AM2, but with one more point) gives the fourth-order Adams-Moulton formula

$$x_{n+1} = x_n + \frac{\Delta t}{24}\left(9F\left(x_{n+1}^{(0)}, t_{n+1}\right) + 19F_n - 5F_{n-1} + F_{n-2}\right). \tag{3.27}$$

Both AB4 and AM4 need four previous points, so – like AB3 – they are not self-starting: the first three steps must come from a single-step method such as RK4.

Worked example. We repeat the running example of this chapter, $\dot{x} = F(x,t) = 2t - x$, $x(0) = -1$ (exact solution $x_*(t) = e^{-t} + 2t - 2$), using RK4 to bootstrap the first three steps and the AB4/AM4 PECE pair (Eqs. (3.26)–(3.27)) for the rest.

```
def F(x, t):
    return 2*t - x

def rk4_solve(x0, t0, dt, N):
    """Classical RK4, N steps. Also caches F at every accepted point (Fs),
    so this doubles as the bootstrap for the multistep method below."""
    ts, xs, Fs = [t0], [x0], [F(x0, t0)]
    x, t = x0, t0
    for _ in range(N):
        k1 = Fs[-1]
        k2 = F(x + dt/2*k1, t + dt/2)
        k3 = F(x + dt/2*k2, t + dt/2)
        k4 = F(x + dt*k3, t + dt)
        x = x + dt/6*(k1 + 2*k2 + 2*k3 + k4)
        t = t + dt
        xs.append(x); ts.append(t); Fs.append(F(x, t))
    return ts, xs, Fs

def predictor_corrector_solve(x0, t0, dt, N):
    """AB4 predictor / AM4 corrector, single correction per step (PECE)."""
    n_boot = min(3, N)
    ts, xs, Fs = rk4_solve(x0, t0, dt, n_boot)
    x, t = xs[-1], ts[-1]
    for _ in range(N - n_boot):
        Fn, Fn1, Fn2, Fn3 = Fs[-1], Fs[-2], Fs[-3], Fs[-4]
        t_new = t + dt
```

```
        x_pred = x + dt/24*(55*Fn - 59*Fn1 + 37*Fn2 - 9*Fn3) # Predict
        F_pred = F(x_pred, t_new) # Evaluate
        x_new = x + dt/24*(9*F_pred + 19*Fn - 5*Fn1 + Fn2) # Correct
        F_new = F(x_new, t_new) # Evaluate
        xs.append(x_new); ts.append(t_new); Fs.append(F_new)
        x, t = x_new, t_new
    return ts, xs
```

Running both with $x_0 = -1$, $\Delta t = 0.1$, $N = 10$ gives Table 3.2.

n	t_n	RK4 x_n	PC x_n	exact
0	0.0	−1.00000000	−1.00000000	−1.00000000
1	0.1	−0.89516250	−0.89516250	−0.89516258
2	0.2	−0.78126910	−0.78126910	−0.78126925
3	0.3	−0.65918158	−0.65918158	−0.65918178
4	0.4	−0.52967971	−0.52968008	−0.52967995
5	0.5	−0.39346907	−0.39346973	−0.39346934
6	0.6	−0.25118807	−0.25118897	−0.25118836
7	0.7	−0.10341438	−0.10341547	−0.10341470
8	0.8	0.04932929	0.04932806	0.04932896
9	0.9	0.20656999	0.20656866	0.20656966
10	1.0	0.36787977	0.36787837	0.36787944

Table 3.2: RK4 versus the AB4/AM4 predictor-corrector pair on $\dot{x} = 2t - x$, $x(0) = -1$. Rows $n = 0$–3 are identical because both methods use RK4 for the bootstrap. RK4's error at $t = 1$ is 3.3×10^{-7}; the predictor-corrector's is 1.1×10^{-6} – both are $O(10^{-6})$, confirming both methods are genuinely fourth order, even though RK4 is somewhat more accurate in absolute terms.

Cost comparison. Counting function evaluations: after the RK4 bootstrap (which needs the usual 4 evaluations per step), each further PECE step needs only *two* evaluations of F – one at the predicted point, one at the corrected point (to cache F_{n+1} for the next step's predictor) – rather than RK4's four stage evaluations. So the predictor-corrector pair should cost roughly half as much as RK4 for the same accuracy, once the fixed cost of the three-step bootstrap becomes negligible.

To test this, F was given an artificial 2 ms delay on every call, standing in for a right-hand side that is genuinely expensive to evaluate (e.g. a call to an external simulation or device). This isolates the cost that scales with the *number* of evaluations from Python's own (essentially fixed, and here negligible) interpreter overhead, so that the predicted scaling shows up cleanly.

```
import time

def make_F(sleep):
    calls = [0]
```

```
    def F(x, t):
        calls[0] += 1
        time.sleep(sleep)
        return 2*t - x
    return F, calls
```

Table 3.3 shows the results of solving the same problem over $[0, 1]$ with $\Delta t = 1/N$, for a range of N, timed with `time.perf_counter()`.

N	evals (RK4)	evals (PC)	time RK4 (s)	time PC (s)	speedup
10	40	27	0.100	0.068	1.48
20	80	47	0.211	0.129	1.64
50	200	107	0.515	0.278	1.85
100	400	207	1.029	0.578	1.78
200	800	407	2.117	1.057	2.00
400	1600	807	4.138	2.074	2.00

Table 3.3: Wall-clock runtime with a 2 ms artificial delay per evaluation of F. The evaluation count ratio (PC/RK4) falls from $27/40 = 0.68$ at $N = 10$ towards 0.5 as N grows and the fixed bootstrap cost becomes negligible, and the measured speedup converges to the predicted factor of 2.

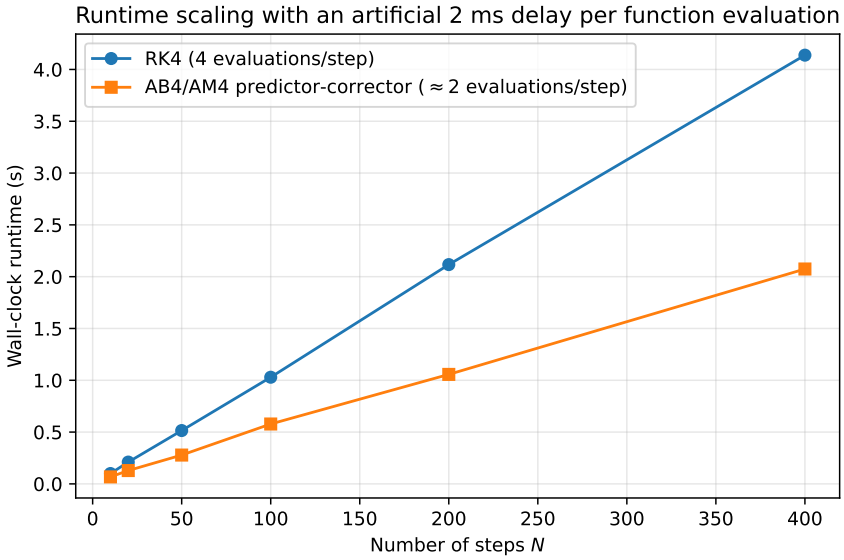


Figure 3.7: Wall-clock runtime against number of steps N, with a 2 ms artificial delay per evaluation of F. Both lines are linear in N, as expected since both methods do $O(N)$ evaluations of F, but the predictor-corrector line's slope is essentially half that of RK4's – exactly the 2:1 evaluation-count ratio derived above.

This is the practical payoff of predictor-corrector methods: for the same order of accuracy as RK4, the AB4/AM4 pair needs about half as many evaluations of F per step, translating directly into a real wall-clock speedup whenever F is expensive to evaluate.

3.11 Stiff differential equations

Consider the linear system (Kharab & Guenther, §12.11)

$$\dot{x} = 131x - 931y, \quad \dot{y} = 133x - 933y, \qquad x(0) = 6,\ y(0) = -6. \tag{3.28}$$

Its exact solution is

$$x(t) = 14e^{-2t} - 8e^{-800t}, \qquad y(t) = 2e^{-2t} - 8e^{-800t}, \tag{3.29}$$

a mix of a slowly decaying mode (e^{-2t}) and a very rapidly decaying one (e^{-800t}) – rates 400 times apart. Except very close to $t = 0$, the fast mode is utterly negligible and the solution is, to all intents and purposes, the smooth, slowly decaying curve $x(t) \approx 14e^{-2t}$.

Forward versus backward Euler. We solve Eq. (3.28) with both Forward Euler and the backward Euler scheme of Section 3.7, writing it as $\dot{\mathbf{y}} = \Lambda\mathbf{y}$ with $\mathbf{y} = (x, y)^T$. For backward Euler, $\mathbf{y}_{n+1} = \mathbf{y}_n + \Delta t\,\Lambda\mathbf{y}_{n+1}$ rearranges to the linear system $(I - \Delta t\Lambda)\mathbf{y}_{n+1} = \mathbf{y}_n$, which we solve directly rather than forming a matrix inverse – the same practice recommended in Section 2.5.

```
import numpy as np

Lambda = np.array([[131.0, -931.0],
                   [133.0, -933.0]])

def forward_euler(y0, dt, N):
    ys = [y0]
    y = y0
    for _ in range(N):
        y = y + dt * (Lambda @ y)
        ys.append(y)
    return np.array(ys)

def backward_euler(y0, dt, N):
    A = np.eye(2) - dt * Lambda
    ys = [y0]
    y = y0
    for _ in range(N):
        y = np.linalg.solve(A, y) # solve (I - dt*Lambda) y_new = y_old
        ys.append(y)
    return np.array(ys)
```

Running both from $\mathbf{y}_0 = (6, -6)$ with $\Delta t = 0.1$ gives Table 3.4.

What makes a system stiff? Precisely defining stiffness turns out to be surprisingly difficult. As Iserles puts it,

n	t_n	Forward Euler x_n	Backward Euler x_n	exact $x(t_n)$
0	0.0	6.0000	6.00000000	6.00000000
1	0.1	643.2000	11.56790123	11.46223054
2	0.2	-49919.04	9.72100290	9.38448064
3	0.3	3.944×10^{6}	8.10183680	7.68336291
5	0.5	2.462×10^{10}	5.62628601	5.15031218
10	1.0	-7.575×10^{19}	2.26107816	1.89469397

Table 3.4: Forward versus backward Euler on Eq. (3.28), $\Delta t = 0.1$. Forward Euler's error already exceeds the true value by a factor of 50 after one step, and grows without bound; backward Euler stays close to the (slowly decaying) exact solution throughout, despite using the same, relatively large, step size.

> *"Several attempts at a rigorous definition of stiffness appear in the literature, but it is perhaps more informative to adopt an operative (and slightly vague) designation. Thus, we say that an ODE system $y' = f(t, y)$, $t \geq t_0$, $y(t_0) = y_0$, is stiff if its numerical solution by some methods requires (perhaps in a portion of the solution interval) a significant depression of the step size to avoid instability. Needless to say this is not a proper mathematical definition, but then we are not aiming to prove theorems of the sort 'if a system is stiff then ...'."*
>
> — Iserles, *A First Course in the Numerical Analysis of Differential Equations*, p. 56.

Working informally, then: a system is *stiff* when it contains widely separated timescales – equivalently, when the eigenvalues of the (linearised) system have real parts of very different magnitude, as in Eq. (3.28) above. The step size needed for stability with an explicit method is dictated by the *fastest* decaying mode, even long after that mode has become numerically negligible to the solution itself.

It is tempting to think that, once the fast transient has visibly died away, it would be safe to increase Δt again. It would not: the instability in Table 3.4 and Figure 3.8 comes entirely from the (now utterly negligible) fast mode reasserting itself numerically. Iserles again, rather memorably:

> *"Faced with an equation [...] (with an arbitrary initial condition) we are likely to employ a small step size in the initial transient regime, in which the contribution of the 'unstable' eigenvector is still significant. However, as soon as this has disappeared and the solution is completely described by the 'stable' eigenvector, it is tempting to increase h. This must be resisted: like a malign version of the Cheshire cat, the rogue eigenvector might seem to have disappeared, but its hideous grin stays and is bound to thwart our endeavours."*
>
> — Iserles, *A First Course in the Numerical Analysis of Differential Equations*, p. 55.

Connection to implicit schemes. This is exactly the situation already flagged in Section 3.7: implicit schemes are "used in modelling fast processes which would require unfeasibly short time steps with an explicit scheme". Backward Euler and Crank-Nicolson, derived there, are precisely the fix – as confirmed numerically above.

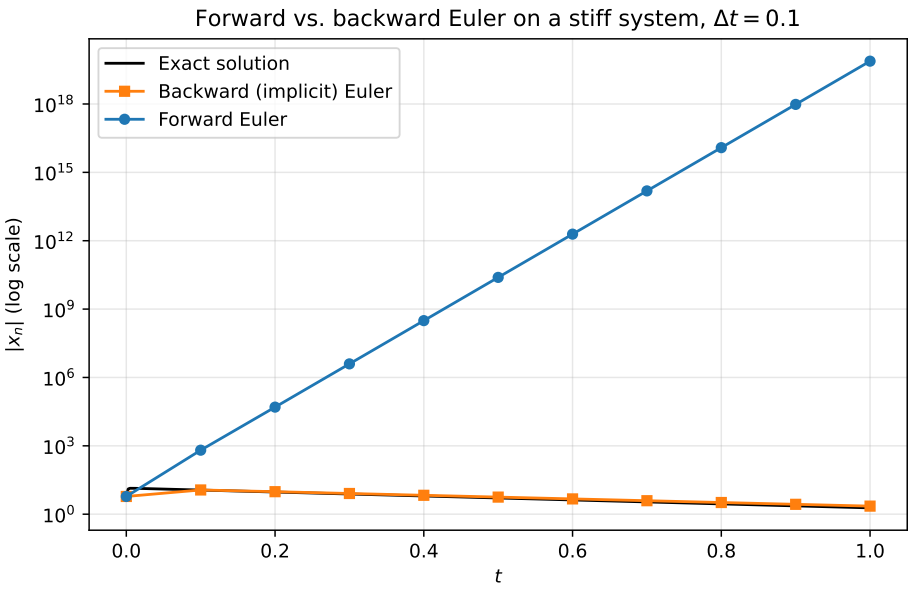


Figure 3.8: $|x_n|$ on a logarithmic scale: Forward Euler diverges geometrically (note the alternating sign each step, from the negative amplification factor $1 + 131\Delta t \approx -79$ along the stiff eigendirection), while backward Euler tracks the true, smoothly decaying solution closely throughout.

A-stability. One line of formal vocabulary, without proof: a method that is stable for *any* step size when applied to the decay equation $\dot{x} = \lambda x$ with $\mathrm{Re}(\lambda) < 0$ is called *A-stable*. Backward Euler is A-stable; Forward Euler, is not. A full treatment of stability regions and A-stability (Padé approximants, order barriers, and so on) is a much larger topic than we need here – see Iserles, *A First Course in the Numerical Analysis of Differential Equations*, Chapter 4, for a complete account.

3.12 Boundary value problems

So far, we have considered initial value problems (IVPs), where the solution is constrained by the initial conditions. Another important class of ODE is the boundary value problem (BVP), where the solution is constrained by the boundary conditions.

Consider the ODE problem

$$\frac{d^2y}{dx^2} = f(x)$$

on the domain $0 \leq x \leq 1$ with boundary conditions

$$y(0) = 0 \text{ and } y(1) = 0.$$

Let us discretise on the domain $x_i = i\Delta x$ where have redefined i as $i \in \mathbb{Z}^+$. Evaluating the second derivative to second order in x, for interior grid points, gives

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{\Delta x^2} = f_i \quad i = 1, 2, \ldots N-2,$$

where $N-1 = 1/\Delta x$. Rearranging gives

$$y_{i+1} - 2y_i + y_{i-1} = \Delta x^2 f_i \quad i = 1, 2, \ldots N-2,$$

Combined with the boundary conditions, this is usefully represented in matrix form

$$T\mathbf{y} = \Delta x^2 \mathbf{f},$$

where all the elements y_i and f_i are contained in the vectors $\mathbf{y}$ and $\mathbf{f}$ respectively.

$$\overbrace{\begin{bmatrix} \mathbf{1} & & & & & & & & \\ & \ddots & & & & & & & \\ & & \ddots & & & & & & \\ & & 1 & -2 & 1 & & & & \\ & & & 1 & -2 & 1 & & & \\ & & & & 1 & -2 & 1 & & \\ & & & & & & \ddots & & \\ & & & & & & & \ddots & \\ & & & & & & & & \mathbf{1} \end{bmatrix}}^{T} \overbrace{\begin{bmatrix} y_0 \\ \vdots \\ \vdots \\ y_{i-1} \\ y_i \\ y_{i+1} \\ \vdots \\ \vdots \\ y_{N-1} \end{bmatrix}}^{\mathbf{y}} = \Delta x^2 \overbrace{\begin{bmatrix} \mathbf{0} \\ \vdots \\ \vdots \\ f_{i-1} \\ f_i \\ f_{i+1} \\ \vdots \\ \vdots \\ \mathbf{0} \end{bmatrix}}^{\mathbf{f}}$$

It is important to note the imposition of the boundary conditions via the entries marked in bold in T and $\mathbf{f}$. The python code **boundary_value.m** on the ele page is an example of this method. We look at the case

$$\frac{d^2y}{dx^2} = \sin x$$

on a domain $[0, \pi]$ with 'Dirichlet' boundary conditions

$$y(0) = y(\pi) = 0$$

where the value of the function is specified. The other type, Neumann boundary conditions, is where the first derivative is specified. The true solution is

$$y_* = -\sin x$$

Let's look at the code **boundary_value.py**. The problem is set up as $A\mathbf{x} = \mathbf{b}$. The vector $\mathbf{b}$ and matrix A are define here

```
b = (dx**2) * np.sin(x) # Right-hand side

# Initialize matrix A
A = np.zeros((nx, nx))

for i in range(1, nx - 1): # Fill A matrix with interior elements
    A[i, i + 1] = 1
    A[i, i] = -2
    A[i, i - 1] = 1
```

The right hand side is in b and the A matrix is defined row by row. The Dirichlet boundary conditions are defined here, and in the end elements of b (which are zero by definition)

```
# Define boundary conditions
A[0, 0] = 1.0
A[nx-1, nx-1] = 1.0 # this could also be done with A[-1,-1]
```

and finally $A\mathbf{x} = \mathbf{b}$ is solved

```
y = np.linalg.solve(A, b)
```

There will be more on matrix solvers in the next part of the module. The numerical solution can be compared with the true solution for two cases: a large number of points in x, nx=51, and a small number of points, nx=6. For the small number of points Δx is much larger and the truncation error is more significant; the solution is thus less accurate. The number of points is called the 'resolution' of a method.

Chapter 4

Numerical integration

Historical figures

This chapter's ideas span three generations. Around 1700, **Isaac Newton** (1643–1727) and his young collaborator **Roger Cotes** (1682–1716, who helped prepare the second edition of the *Principia*) developed the Newton–Cotes formulas, of which the trapezium rule and Simpson's rule are special cases. In 1748, **Maria Gaetana Agnesi** (1718–1799) published *Instituzioni Analitiche*, widely considered the first comprehensive calculus textbook and the first major mathematics book written by a woman: running to over a thousand pages, it systematically laid out algebra, analytic geometry, and differential and integral calculus for students, at a time when much mathematical writing was accessible only to specialists. Finally, in the early 1800s **Carl Friedrich Gauss** (1777–1855) showed that choosing the sample points optimally, rather than equally spaced, gives far higher accuracy for the same number of evaluations — Gaussian quadrature. Newton, Cotes, Agnesi, Gauss on Wikipedia.

In the previous chapter we discussed numerical methods for integrating equations forward in time, hence the name 'timestepping'. But in applied mathematics it is essential that we understand how to integrate with respect to any scalar variable, not just time – and, as we will see, this turns out to be somewhat of a different problem, albeit with some significant similarities.

The basic idea behind every method in this chapter is the same. By the fundamental theorem of calculus,

$$I = \int_a^b f(x)\,dx \tag{4.1}$$

is just the area under the curve $y = f(x)$ between $x = a$ and $x = b$. The essence of numerical integration is that we replace f by an interpolating polynomial p, whose area underneath is straightforward to compute exactly, and use that as our approximation to I. This process is called *numerical quadrature*. The name has old origins: *quadrature* comes from the Latin *quadratura* ('squaring'), since Greek geometers treated finding the area of a shape as the problem of constructing a *square* of that same area – 'squaring the circle' being the most famous instance of the problem. 'Numerical quadrature' is thus literally 'numerically finding the area', which is exactly what the trapezium rule and Simpson's rule do below, by replacing

the curve with a shape – a trapezoid, a parabolic segment – whose area is easy.

It is tempting to think of quadrature as simply 'the same idea as timestepping, but integrating over x instead of t'. It is not quite so simple. In timestepping, the right-hand side $F(x,t)$ depends on the solution x itself, which is *unknown* at future times until it has actually been computed. In quadrature, $f(x)$ is a completely *known* function, and its value can be found at any point in $[a,b]$ on demand, since nothing is unfolding in time and the whole domain is available upfront. We will return to this distinction properly, with a concrete example, in Section 4.1.4 below.

In this chapter we will explore the Newton-Cotes framework for describing numerical integration techniques, covering the trapezium rule and Simpson's rule, before turning to Gaussian quadrature and, finally, making precise how general numerical integration and timestepping relate to one another.

4.1 Newton-Cotes

Every quadrature rule developed in this chapter takes the same general form:

$$\int_a^b f(x)\,dx \approx \sum_{i=1}^{n} w_i f(x_i) \tag{4.2}$$

for some choice of *nodes* $x_1,\ldots,x_n$ and *weights* $w_1,\ldots,w_n$. The Newton-Cotes rules developed in this section – the trapezium rule and Simpson's rule – fix the nodes to be *equally spaced* across $[a,b]$, and solve only for the weights that make the rule as accurate as possible. Gaussian quadrature, developed in Section 4.2, takes a different approach: it solves for the nodes too, and does substantially better as a result.

4.1.1 Trapezium Rule

More than a thousand years before Newton and Cotes were born, Babylonian astronomers were already using the central idea of this section. Cuneiform tablets studied by historian Mathieu Ossendrijver show that, between roughly 350 and 50 BCE, Babylonian astronomers computed the distance travelled by Jupiter across the sky as the *area of a trapezoid* under a graph of its velocity against time – geometrically exactly the rule derived below, some 1400 years before it reappears in the work of Newton and Cotes (Ossendrijver, M., 2016, "Ancient Babylonian astronomers calculated Jupiter's position from the area under a time-velocity graph", *Science* 351(6272), 482–484).

Deriving the rule. Divide $[a,b]$ into n equal subintervals of width $h=(b-a)/n$, with $x_i = a+ih$ for $i=0,1,\ldots,n$. On each subinterval $[x_{i-1},x_i]$, approximate f by the straight line through its endpoints, as shown in Figure 4.1. The resulting trapezoid has area

$$A_i = \frac{h}{2}\left[f(x_{i-1})+f(x_i)\right]. \tag{4.3}$$

Summing over all n subintervals, each interior node x_i $(1\le i\le n-1)$ is shared by two neighbouring panels and so appears exactly twice; collecting terms gives the *composite trapezoidal*

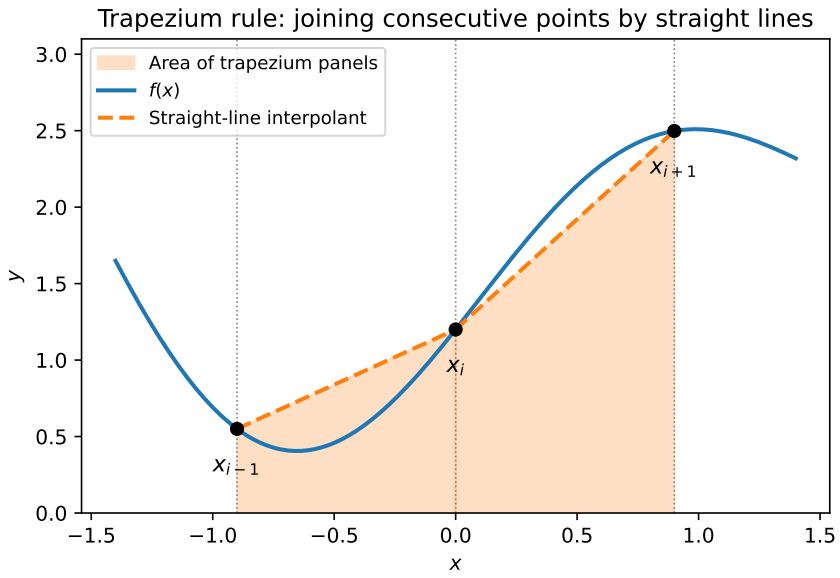


Figure 4.1: The trapezium rule joins each pair of consecutive points by a straight line (orange, dashed), rather than the parabola used by Simpson's rule (compare Figure 4.3, fitted to the same three points); the shaded areas are the two panel estimates for $\int_{x_{i-1}}^{x_i} f(x)\,dx$ and $\int_{x_i}^{x_{i+1}} f(x)\,dx$.

rule

$$T_n = \frac{h}{2}\left[f(x_0) + f(x_n)\right] + h\sum_{i=1}^{n-1} f(x_i). \tag{4.4}$$

Error analysis. By the fundamental theorem of calculus, the exact contribution of panel i is $I_i = \int_{x_{i-1}}^{x_i} f(x)\,dx = F(x_i) - F(x_{i-1})$, where $F' = f$. Taylor expanding $F(x_{i-1}) = F(x_i - h)$ about x_i and using $F' = f$, exactly as in Eq. (3.2), gives

$$I_i = hf(x_i) - \frac{h^2}{2}f'(x_i) + \frac{h^3}{6}f''(x_i) - \cdots.$$

Now recall the Backward Euler finite-difference formula, Eq. (3.4): writing out its leading correction term explicitly, rather than leaving it as $O(h)$, gives $f'(x_i) = \frac{1}{h}[f(x_i) - f(x_{i-1})] + \frac{h}{2}f''(x_i) - \cdots$. Substituting this into the expression above and simplifying,

$$I_i = \frac{h}{2}\left[f(x_{i-1}) + f(x_i)\right] - \frac{h^3}{12}f''(x_i) + \cdots.$$

so that, comparing with Eq. (4.3), the per-panel error is

$$E_i = I_i - A_i = -\frac{h^3}{12}f''(x_i) + \text{higher-order terms}. \tag{4.5}$$

If $|f''(x)| \le M$ on $[a, b]$, summing $|E_i|$ over all $n = (b-a)/h$ panels gives the total error bound

$$|E_T| \le n\frac{h^3}{12}M = \frac{(b-a)h^2}{12}M, \tag{4.6}$$

so the composite trapezoidal rule has *global* error $O(h^2)$. Since Eq. (4.5) vanishes identically when $f'' \equiv 0$, the rule is exact whenever f is linear (degree ≤ 1) – consistent with the fact that a straight line has zero error when approximated by... itself.

Worked example. We test the rule on $\int_0^1 e^x\,dx = e - 1$: an integral with a known exact value, but no special structure (it is neither linear nor, as we will see, cubic) that would make either this rule or Simpson's rule below exact.

```
import numpy as np

def f(x):
    return np.exp(x)

def trapezoidal(f, a, b, n):
    h = (b - a) / n
    x = a + h * np.arange(n + 1)
    fx = f(x)
    return h/2 * (fx[0] + fx[-1]) + h * np.sum(fx[1:-1])
```

Table 4.1 shows the result of halving h repeatedly. Figure 4.2(a) shows the same data on a

n	h	T_n	$\lvert T_n - I\rvert$
2	0.500000	1.7539310925	3.565×10^{-2}
4	0.250000	1.7272219046	8.940×10^{-3}
8	0.125000	1.7205185922	2.237×10^{-3}
16	0.062500	1.7188411286	5.593×10^{-4}
32	0.031250	1.7184216603	1.398×10^{-4}
64	0.015625	1.7183167869	3.496×10^{-5}
128	0.007812	1.7182905681	8.740×10^{-6}
256	0.003906	1.7182840134	2.185×10^{-6}

Table 4.1: Composite trapezoidal rule applied to $\int_0^1 e^x\,dx$. Each time h is halved, the error drops by a factor of almost exactly 4 – direct numerical confirmation of the $O(h^2)$ result in Eq. (4.6).

log-log plot: a straight line of slope 2, lying exactly on the predicted $O(h^2)$ reference line.

A practical caveat: round-off. Eq. (4.6) says the error keeps shrinking as $h \to 0$ – but only if every function evaluation and every addition is carried out exactly. In floating-point arithmetic this is not so: Kharab and Guenther note that "because of the round-off error, taking a large number of trapezoids will not result in general to a better approximation" (Kharab & Guenther, §10.2, p. 251), and recommend instead recomputing with increasing n and stopping once successive estimates agree to the desired tolerance, rather than trying to choose n in advance.

Figure 4.2(b) illustrates this directly, pushing the same worked example to n far beyond anything a practical computation would need: the error falls like $O(h^2)$ for a while, then

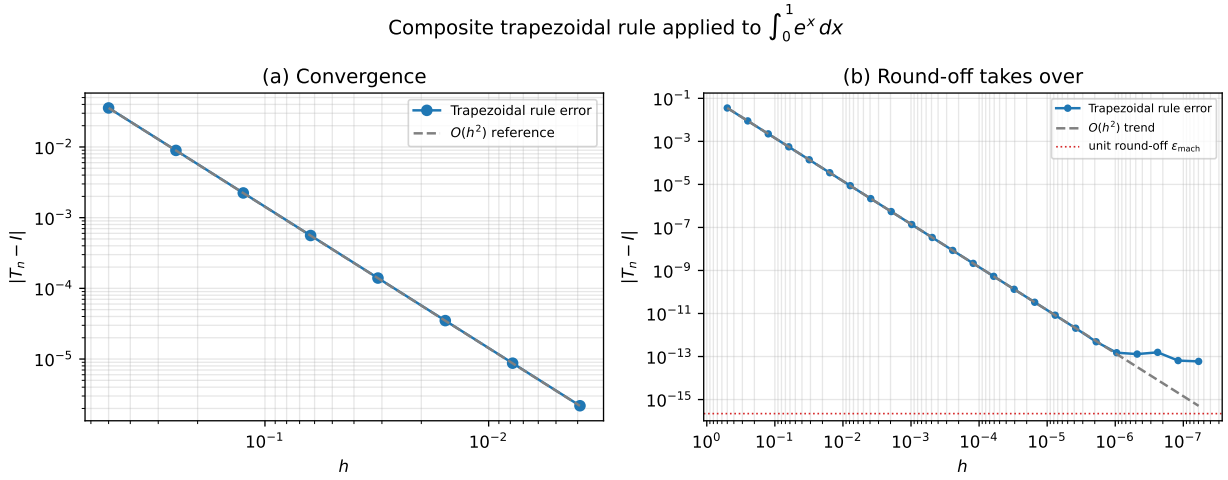


Figure 4.2: (a) Convergence of the composite trapezoidal rule applied to $\int_0^1 e^x\,dx$, on log-log axes: the computed error (blue) lies exactly on the theoretical $O(h^2)$ reference line (grey, dashed). (b) The same test extended to far smaller h: convergence tracks $O(h^2)$ until round-off error takes over, beyond which the error flattens off and becomes non-monotonic, settling several orders of magnitude above machine epsilon (red, dotted).

flattens off and even increases slightly from one n to the next, before settling several orders of magnitude above machine epsilon $\varepsilon_{\text{mach}} \approx 2.2\times10^{-16}$. The notebook `trapezium_roundoff.ipynb` provided with these notes walks through the full computation behind this panel, including exactly where the crossover happens and why.

4.1.2 Simpson's Rule

Deriving the rule. Divide $[a, b]$ into $n = 2m$ equal subintervals of width $h = (b-a)/n$, with $x_i = a + ih$ for $i = 0, 1, \ldots, n$, and take the subintervals two at a time. On each pair $[x_{i-1}, x_{i+1}]$, rather than joining $f(x_{i-1})$ and $f(x_{i+1})$ by a straight line as the trapezium rule does, approximate f by the unique parabola passing through all *three* points $(x_{i-1}, f(x_{i-1}))$, $(x_i, f(x_i))$, $(x_{i+1}, f(x_{i+1}))$, as shown in Figure 4.3. The area under this parabola works out to be

$$S_i = \frac{h}{3}\left[f(x_{i-1}) + 4f(x_i) + f(x_{i+1})\right]. \tag{4.7}$$

Summing over all $m = n/2$ non-overlapping pairs of subintervals and collecting terms – each interior node at an *odd* index appears in exactly one pair and so gets weight 4, each interior node at an *even* index is shared between two neighbouring pairs and so gets weight 2, and the two endpoints get weight 1 – gives the *composite Simpson's rule*

$$S_n = \frac{h}{3}\left[f(x_0) + 4\sum_{i=1}^{m} f(x_{2i-1}) + 2\sum_{i=1}^{m-1} f(x_{2i}) + f(x_n)\right]. \tag{4.8}$$

Error analysis. As with the trapezium rule, the exact contribution over a double panel is $I_i = \int_{x_{i-1}}^{x_{i+1}} f(x)\,dx = F(x_{i+1}) - F(x_{i-1})$, where $F' = f$. Taylor expanding $F(x_i + h)$ and

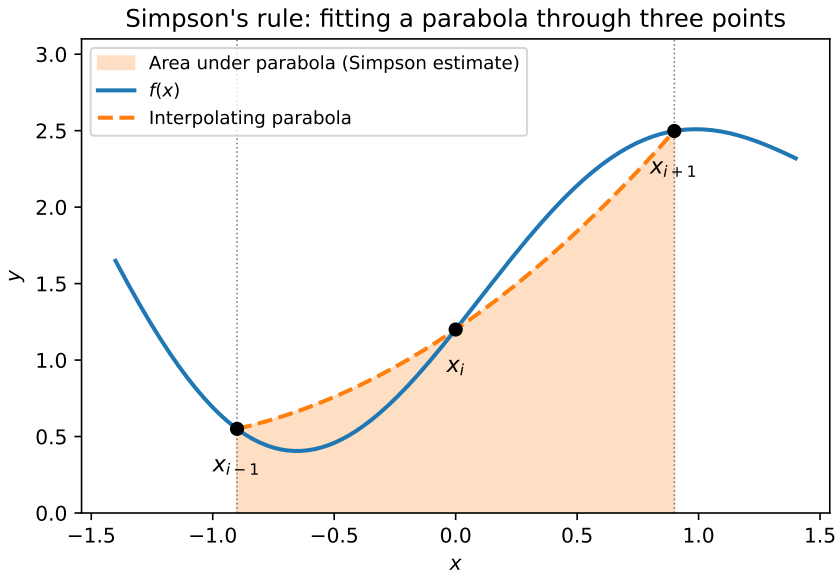


Figure 4.3: Simpson's rule fits a parabola (orange, dashed) through three consecutive points, rather than the two straight-line panels used by the trapezium rule (compare Figure 4.1, fitted to the same three points); the shaded area under the parabola is the estimate for $\int_{x_{i-1}}^{x_{i+1}} f(x)\,dx$.

$F(x_i - h)$ about x_i and subtracting – exactly the technique of Eqs. (3.1)–(3.2), but carried one order further so that the $f^{(4)}$ term survives – the odd-order terms double and the even-order terms cancel, giving

$$I_i = 2hf(x_i) + \frac{h^3}{3}f''(x_i) + \frac{h^5}{60}f^{(4)}(x_i) + \cdots .$$

Now recall the central-difference formula for the second derivative, Eq. (3.7): writing out its leading correction term explicitly, rather than leaving it as $O(h^2)$, gives

$$f''(x_i) = \frac{1}{h^2}\left[f(x_{i+1}) - 2f(x_i) + f(x_{i-1})\right] - \frac{h^2}{12}f^{(4)}(x_i) + \cdots .$$

Substituting this into the expression above and simplifying,

$$I_i = \frac{h}{3}\left[f(x_{i-1}) + 4f(x_i) + f(x_{i+1})\right] - \frac{h^5}{90}f^{(4)}(x_i) + \cdots .$$

so that, comparing with Eq. (4.7), the per-panel error is

$$E_i = I_i - S_i = -\frac{h^5}{90}f^{(4)}(x_i) + \text{higher-order terms.} \tag{4.9}$$

If $|f^{(4)}(x)| \le M$ on $[a, b]$, summing $|E_i|$ over all $m = n/2$ double panels gives the total error bound

$$|E_T| \le m\frac{h^5}{90}M = \frac{(b-a)h^4}{180}M, \tag{4.10}$$

so the composite Simpson's rule has *global* error $O(h^4)$ – two full orders better than the trapezium rule's $O(h^2)$ (Eq. (4.6)) for the same spacing h, though as with the trapezium rule this extra accuracy is only fully realised once round-off error is properly accounted for. Since Eq. (4.9) vanishes identically when $f^{(4)} \equiv 0$, the rule is exact whenever f is a polynomial of degree ≤ 3 – one degree *higher* than the quadratic actually being fitted, a pleasant surprise that follows because the leading error term happens to depend on $f^{(4)}$ rather than f'''.

4.1.3 Generalisation to higher order polynomials

Having gone from a straight line (two points, the trapezium rule) to a parabola (three points, Simpson's rule), the pattern seems obvious: fit a cubic through four points, a quartic through five, and so on, gaining two further orders of accuracy each time a degree is added. This is a perfectly sound idea in principle – but in practice it runs into a serious obstacle once the degree gets much beyond Simpson's rule.

The problem is that interpolating a function at *equally spaced* points with a high-degree polynomial does not, in general, get more accurate as the degree increases – it can get dramatically worse, particularly near the ends of the interval. This is known as *Runge's phenomenon*, after **Carl Runge** (1856–1927), the same Runge who – together with Martin Kutta – gave his name to the Runge-Kutta methods of Section 3.9. In 1901, while investigating exactly this kind of polynomial interpolation, Runge exhibited a function that gets worse under equally-spaced high-degree interpolation the higher the degree goes: $f(x) = \frac{1}{1+25x^2}$ on $[-1, 1]$, shown in Figure 4.4 together with three of its own equally-spaced polynomial interpolants. Runge on Wikipedia.

This has a well-known cousin in signal processing: the *Gibbs phenomenon*, the persistent overshoot that appears near a jump discontinuity when a Fourier series is truncated to finitely many terms. In both cases, adding more terms – higher polynomial degree here, more Fourier modes there – does not damp the oscillation near the problematic point (the interval's edges here, the discontinuity there), and can even make it worse; if you have already met the Gibbs phenomenon in a Fourier methods course, this is the same underlying difficulty showing up in a different setting. See the Runge's phenomenon Wikipedia article for more on the comparison. Because of this, raw high-degree Newton-Cotes rules are rarely used in practice beyond Simpson's rule. Far more reliable is exactly the strategy already used to get from a single trapezium to the composite trapezoidal rule, and from a single parabola to composite Simpson's rule: keep the polynomial degree fixed and low, and instead increase accuracy by applying the rule *compositely* over more, smaller subintervals.

An extension: Richardson extrapolation. There is, however, a genuinely different way to squeeze higher-order accuracy out of a low-order rule, without either raising the degree or simply shrinking h: combine two composite-trapezoidal estimates at different values of h so as to cancel their leading error term algebraically, producing a new estimate that is more accurate than either. This technique, called *Richardson extrapolation*, applied repeatedly to the composite trapezoidal rule, is known as *Romberg integration* (Kharab & Guenther, §10.3, pp. 270–276, building on the general Richardson extrapolation technique of their §9.2, pp.

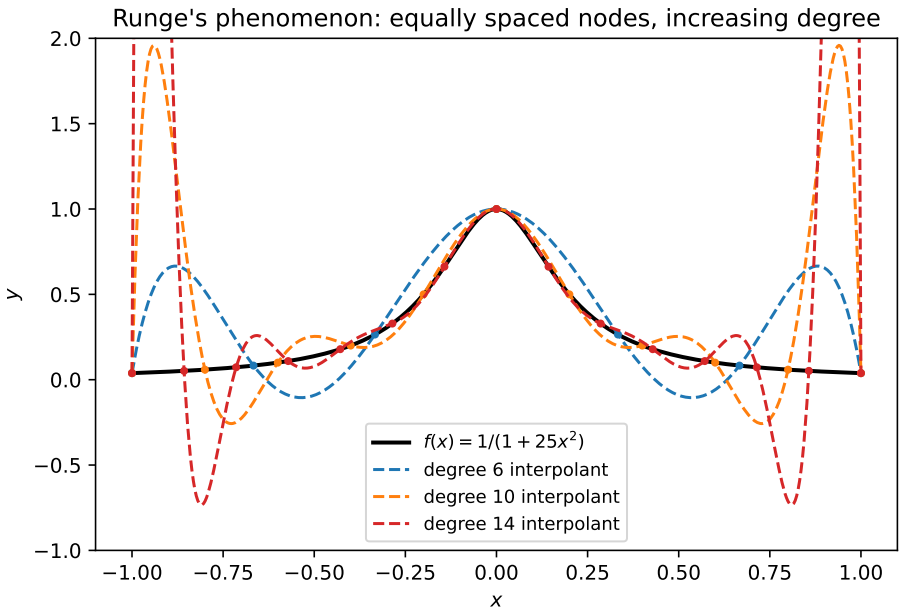


Figure 4.4: Runge's phenomenon: $f(x) = 1/(1 + 25x^2)$ (black) interpolated at equally spaced nodes by polynomials of degree 6, 10 and 14. Away from the edges the fit visibly improves with degree, but near $x = \pm 1$ it gets rapidly worse – the vertical axis here is deliberately clipped to $[-1, 2]$ to keep the central behaviour visible; left unclipped, the degree-14 interpolant actually peaks around $x \approx 0.96$ at a value of about 7.2, more than 170 times the true $f(0.96) \approx 0.04$.

239–241). It is not developed further in these notes, but is a natural next step for anyone who enjoyed this chapter and wants to see the idea through.

4.1.4 Comparison with timestepping

This chapter and the timestepping methods of Chapter 3 share an obvious family resemblance – both approximate something by a weighted sum of function values, Eq. (4.2) – and yet they are developed as entirely separate topics, with different tools and different concerns. Why?

The distinction is drawn explicitly by Iserles, right at the start of his own treatment of Runge-Kutta methods (§3.1–3.2, pp. 33–38). Quadrature integrates a *known* function: $f(x)$ can be evaluated at any $x \in [a, b]$ on demand, since nothing is unfolding in time and the whole domain of f is available upfront. Timestepping evolves an *unknown* state forward: the right-hand side $F(x, t)$ depends on the solution x itself, which is not known at t_{n+1} until it has actually been computed. Iserles makes the point by trying to write down the obvious quadrature-style formula for the solution of an ODE, then noting the snag: "we do not know the value of y at the nodes $t_n + c_1 h, \ldots$ We must resort to an approximation!" (p. 38). That one obstacle is the reason timestepping needs machinery – explicit staging, implicit equation-solving, predictor-corrector pairs – that pure quadrature never does.

We can make this concrete with a formula from each side of our own notes. The trapezium

rule, Eq. (4.3), applied over a single step $[t_n, t_{n+1}]$,

$$\int_{t_n}^{t_{n+1}} F(t)\,dt \approx \frac{h}{2}\left[F(t_n) + F(t_{n+1})\right], \tag{4.11}$$

is completely explicit: $F(t_n)$ and $F(t_{n+1})$ are both just known numbers, immediately computable. The *identical-looking* formula used as a timestepping rule for $\dot{x} = F(x,t)$ – the AM2 method, derived in Section 3.10.3 and shown there to be "identical to the trapezoidal implicit method,"

$$x_{n+1} = x_n + \frac{\Delta t}{2}\left(F_n + F_{n+1}\right), \tag{4.12}$$

is implicit, precisely because $F_{n+1} = F(x_{n+1}, t_{n+1})$ needs the not-yet-known x_{n+1}. Same formula, different character, entirely because of what it is being asked to do: one side has both function values in hand before the calculation starts, and the other has to solve for one of them.

This is also exactly why predictor-corrector methods (Section 3.10.4) exist for ODEs but have no quadrature analogue: quadrature never needs to *predict* a not-yet-known function value, because in quadrature there isn't one.

A second, independent distinction is *stability*. Composite quadrature, Eq. (4.4), sums independent panel contributions with no feedback between them – there is no mechanism for an error made on one panel to corrupt the estimate on another. Timestepping is inherently recursive, $x_{n+1} = g(x_n)$: each step's output becomes the next step's input, so a scheme's own error can compound across repeated steps. This is exactly why stability (Section 3.7) and stiffness (Section 3.11) are such central concerns for timestepping, with no real quadrature counterpart.

4.2 Gaussian Quadrature

Deriving the rule. Recall the general form of Eq. (4.2): every rule in this chapter approximates $\int_a^b f(x)\,dx$ by a weighted sum $\sum_i w_i f(x_i)$. The trapezium and Simpson's rules both fix the nodes x_i to be equally spaced across $[a,b]$ – including, awkwardly, forcing two of them to sit right at the endpoints a and b – and solve only for the weights w_i. Gaussian quadrature asks a more ambitious question: if we are also free to choose *where* the nodes go, rather than fixing them in advance, how much more accuracy can the same number of evaluations buy us? Figure 4.5 shows the resulting node placement for a 3-point rule alongside the trapezium rule's two fixed endpoint nodes, on the same curve.

As with the earlier rules, it is easiest to work on a reference interval and map onto it. Transforming $[a,b]$ to $[-1,1]$ via

$$x = a + \frac{b-a}{2}(z+1), \qquad g(z) = f\left(a + \frac{b-a}{2}(z+1)\right), \qquad -1 \le z \le 1, \tag{4.13}$$

gives $\int_a^b f(x)\,dx = \frac{b-a}{2}\int_{-1}^{1} g(z)\,dz$, so it suffices to find nodes $z_1, \ldots, z_n$ and weights $w_1, \ldots, w_n$ for $\int_{-1}^{1} g(z)\,dz \approx \sum_{i=1}^{n} w_i g(z_i)$.

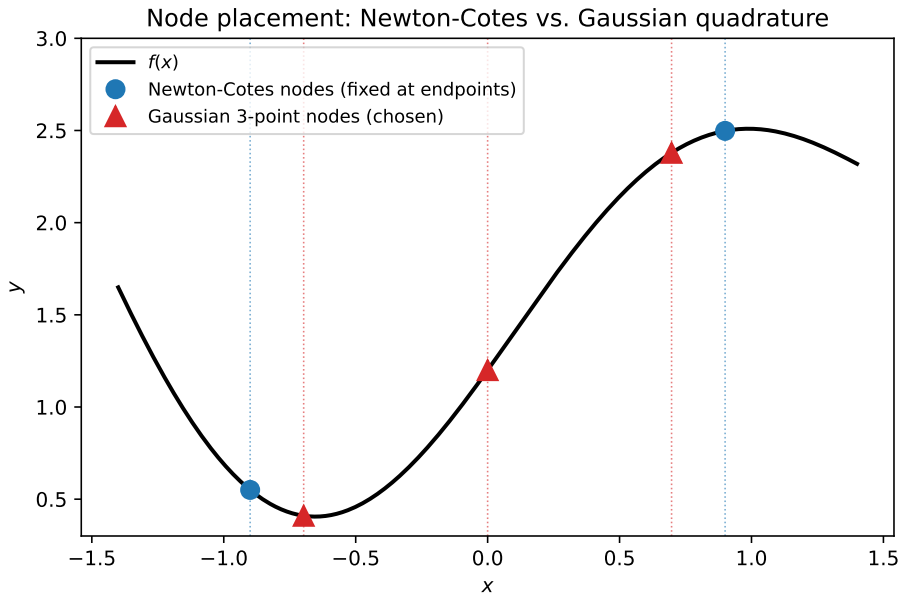


Figure 4.5: Node placement: the trapezium rule (Newton-Cotes) fixes its nodes at the interval's endpoints (blue circles), while the Gaussian 3-point rule instead chooses three interior nodes (red triangles) so as to maximise accuracy.

We derive the case $n = 3$ by the *method of undetermined coefficients*: with six unknowns $(z_1, z_2, z_3, w_1, w_2, w_3)$, we can hope to fix all six by requiring the rule to be exact for the six monomials $g(z) = 1, z, z^2, z^3, z^4, z^5$ – that is, for every polynomial up to degree 5. Integrating each monomial exactly over $[-1, 1]$ and demanding the quadrature sum match gives the nonlinear system

$$\begin{aligned} w_1 + w_2 + w_3 = 2, \quad w_1 z_1 + w_2 z_2 + w_3 z_3 = 0, \quad w_1 z_1^2 + w_2 z_2^2 + w_3 z_3^2 = \tfrac{2}{3}, \\ w_1 z_1^3 + w_2 z_2^3 + w_3 z_3^3 = 0, \quad w_1 z_1^4 + w_2 z_2^4 + w_3 z_3^4 = \tfrac{2}{5}, \quad w_1 z_1^5 + w_2 z_2^5 + w_3 z_3^5 = 0. \end{aligned} \tag{4.14}$$

Solving a nonlinear system of this kind directly is well beyond the method of undetermined coefficients as used for the trapezium and Simpson's rules; Gauss himself proved that Eq. (4.14) has a unique solution, given by

$$z_1 = -\sqrt{\tfrac{3}{5}}, \quad z_2 = 0, \quad z_3 = \sqrt{\tfrac{3}{5}}, \qquad w_1 = w_3 = \tfrac{5}{9}, \quad w_2 = \tfrac{8}{9}, \tag{4.15}$$

giving the *Gaussian three-point quadrature formula*

$$\int_{-1}^{1} g(z)\, dz \approx \frac{5}{9} g\left(-\sqrt{\tfrac{3}{5}}\right) + \frac{8}{9} g(0) + \frac{5}{9} g\left(\sqrt{\tfrac{3}{5}}\right). \tag{4.16}$$

The same method extends to any number of points n, though the resulting nonlinear systems are solved once and tabulated rather than re-derived each time. The general result is that n-point Gaussian quadrature is exact for every polynomial of degree $\leq 2n - 1$ – roughly *double* the degree an n-point Newton-Cotes rule achieves for the same number of evaluations (Kharab & Guenther, §10.4, pp. 277–281, Table 10.5).

Worked example. We test the three-point rule, Eq. (4.16), on the same integral used throughout this chapter, $\int_0^1 e^x\,dx = e - 1$.

```
import numpy as np

def f(x):
    return np.exp(x)

# 3-point Gauss-Legendre nodes/weights on [-1, 1] (Table 10.5)
nodes = np.array([-np.sqrt(3/5), 0.0, np.sqrt(3/5)])
weights = np.array([5/9, 8/9, 5/9])

def gauss3(f, a, b):
    x = a + (b - a)/2 * (nodes + 1)
    return (b - a)/2 * np.sum(weights * f(x))
```

Table 4.2 compares the result against composite Simpson's rule at $n = 4$ and $n = 6$.

Method	Evaluations	Estimate	Error
Gauss (3-point)	3	1.7182810044	8.241×10^{-7}
Simpson ($n = 4$)	5	1.7183188419	3.701×10^{-5}
Simpson ($n = 6$)	7	1.7182891699	7.341×10^{-6}

Table 4.2: Gaussian 3-point quadrature against composite Simpson's rule, $\int_0^1 e^x\,dx$. Kharab and Guenther remark (§10.4, p. 281) that three-point Gaussian quadrature is generally comparable in accuracy to seven-point Simpson's rule; here it does even better, beating seven-point Simpson's rule outright while using fewer than half the evaluations.

Chapter 5

Matrix problems

Historical figures

The classical methods in this chapter are 19th-century German: Gaussian elimination is named after **Carl Friedrich Gauss** (1777–1855), though the technique predates him by centuries, while **Carl Gustav Jacob Jacobi** (1804–1851) and **Philipp Ludwig von Seidel** (1821–1896) developed the two classical iterative alternatives that bear their names. The chapter's other central ideas — LU decomposition and the matrix *condition number* — came from a wartime codebreaker turned numerical analyst: in 1948, working at the National Physical Laboratory in London, **Alan Turing** (1912–1954) published *Rounding-off errors in matrix processes*, reformulating Gaussian elimination in these terms. A colleague of his there, **Olga Taussky-Todd** (1906–1995), had arrived at matrix theory — then a minor topic — via analysing aircraft flutter problems, and went on to turn it into an essential tool for applied mathematics, publishing over 300 papers and helping establish numerical linear algebra as a field in its own right. Gauss, Jacobi, von Seidel, Turing, Taussky-Todd on Wikipedia.

Many linear numerical problems in science and engineering can be expressed as a matrix inversion problem

$$A\mathbf{x} = \mathbf{b}, \tag{5.1}$$

where A is a matrix and $\mathbf{x}$ and $\mathbf{b}$ are vectors. Another common problem using matrices is the eigenvalue problem

$$A\mathbf{v}_i = \lambda_i \mathbf{v}_i, \tag{5.2}$$

where λ_i are the eigenvalues and $\mathbf{v}_i$ are the eigenvectors. We restrict ourselves to real eigenvalues, $\lambda_i \in \mathbb{R}$. The aim of this section is to explore iterative methods for solving inversion and eigenvalue problems. These are useful when the problem is too large to be solved directly. We will explore the following numerical methods for matrix problems:

- A matrix decomposition to simplify $A\mathbf{x} = \mathbf{b}$, LU decomposition.
- Iterative methods for matrix inversion: Jacobi and Gauss-Seidel.
- The power method for determining eigenvalues.

5.1 Example application

When you use the Google search engine, it requires a ranking of web pages. Part of the PageRank algorithm solves an eigenvalue problem in order to do this. The matrices involved are massive, so the power method, studied later, is used to find the leading eigenvalues.

5.2 Revision of simultaneous equations

N linear equations for $(x_1, x_2 \dots x_N)$ can be written as

$$\begin{aligned} a_{11}x_1 + \dots\dots + a_{1N}x_N &= b_1 \\ \vdots & \vdots \\ \vdots & \vdots \\ a_{N1}x_1 + \dots\dots + a_{NN}x_N &= b_N \end{aligned}$$

where a_{ij} are constant coefficients and b_i form the right hand side. These can be written in the form $A\mathbf{x} = \mathbf{b}$

$$\overbrace{\begin{pmatrix} a_{11} & \dots & \dots & a_{1N} \\ \vdots & & & \vdots \\ \vdots & & & \vdots \\ a_{N1} & \dots & \dots & a_{NN} \end{pmatrix}}^{A} \overbrace{\begin{pmatrix} x_1 \\ \vdots \\ \vdots \\ x_N \end{pmatrix}}^{\mathbf{x}} = \overbrace{\begin{pmatrix} b_1 \\ \vdots \\ \vdots \\ b_N \end{pmatrix}}^{\mathbf{b}}.$$

In order to save space $A\mathbf{x} = \mathbf{b}$ can be written in the augmented form $(A|\mathbf{b})$

$$\left(\begin{array}{cccc|c} a_{11} & \dots & \dots & a_{1N} & b_1 \\ \vdots & & & \vdots & \vdots \\ \vdots & & & \vdots & \vdots \\ a_{N1} & \dots & \dots & a_{NN} & b_N \end{array}\right).$$

5.3 Gaussian elimination revisited $\Diamond$

We are going to revisit Gaussian elimination and show that it also corresponds to one type of matrix decomposition.

$$\left(\begin{array}{cccc|c} \boxed{1} & 1 & 1 & 1 & 10 \\ 2 & 3 & 1 & 5 & 31 \\ -1 & 1 & -5 & 3 & -2 \\ 3 & 1 & 7 & -2 & 18 \end{array}\right)$$

We identify a 'pivot' indicated by the boxed element whose row will remain unchanged throughout row operations. The method we will use here is called Gaussian elimination

'without pivoting', meaning that, whilst pivots are used, we do not swap rows. R_j denotes row j and m_{rc} the elements (with row r and column c) of a matrix of multipliers. We apply Gaussian elimination to zero the column below the pivot. The operations are

$$R_2 - 2R_1 \ (m_{21} = 2), \quad R_3 + R_1 \ (m_{31} = -1), \quad R_4 - 3R_1 \ (m_{41} = 3).$$

Notice that, by convention, the m_{rc} elements are *minus* the multiplier used. Now identify a new pivot.

$$\left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 10 \\ 0 & \boxed{1} & -1 & 3 & 11 \\ 0 & 2 & -4 & 4 & 8 \\ 0 & -2 & 4 & -5 & -12 \end{array}\right)$$

With the new pivot,

$$R_3 - 2R_2 \ (m_{32} = 2), \quad R_4 + 2R_2 \ (m_{42} = -2)$$

A third pivot is identified

$$\left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 10 \\ 0 & 1 & -1 & 3 & 11 \\ 0 & 0 & \boxed{-2} & -2 & -14 \\ 0 & 0 & 2 & 1 & 10 \end{array}\right)$$

Finally

$$R_4 + R_3 \ (m_{43} = -1)$$

$$\left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 10 \\ 0 & 1 & -1 & 3 & 11 \\ 0 & 0 & -2 & -2 & -14 \\ 0 & 0 & 0 & -1 & -4 \end{array}\right)$$

Now that we have cleared the lower diagonal we can easily solve for

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix}$$

by 'sweeping up' through the matrix or back substituting. By inspection of the matrix we can see

$$\begin{aligned} x_4 &= 4, \\ -2x_3 - 2x_4 = -2x_3 - 8 = -14 \quad x_3 &= 3, \\ \text{and in a similar way,} & \\ x_2 &= 2, \\ x_1 &= 1. \end{aligned}$$

5.4 LU decomposition

It turns out that the Gaussian Elimination process can now be interpretred as an LU decomposition

$$A = LU$$

where L is a lower triangular matrix with a non-zero diagonal and U is upper triangular with a non-zero diagonal.

$$\overbrace{\begin{pmatrix} 1 & 1 & 1 & 1 \\ 2 & 3 & 1 & 5 \\ -1 & 1 & -5 & 3 \\ 3 & 1 & 7 & -2 \end{pmatrix}}^{A} = \overbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ -1 & 2 & 1 & 0 \\ 3 & -2 & -1 & 1 \end{pmatrix}}^{L} \overbrace{\begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & -1 & 3 \\ 0 & 0 & -2 & -2 \\ 0 & 0 & 0 & -1 \end{pmatrix}}^{U}$$

You can verify this in python. We note that L is the same as the matrix of multipliers

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ m_{21} & 1 & 0 & 0 \\ m_{31} & m_{32} & 1 & 0 \\ m_{41} & m_{42} & m_{43} & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ -1 & 2 & 1 & 0 \\ 3 & -2 & -1 & 1 \end{pmatrix}$$

Once A has been decomposed using LU, we then have a much simpler method of solving $A\mathbf{x} = \mathbf{b}$.

$$A\mathbf{x} = LU\mathbf{x} = L(U\mathbf{x}) = \mathbf{b}$$

This can thus be solved in two parts by defining $\mathbf{y} = U\mathbf{x}$ such that

$$\begin{aligned} L\mathbf{y} &= \mathbf{b} \\ U\mathbf{x} &= \mathbf{y} \end{aligned} \tag{5.3}$$

Both L and U are much easier to invert than A as they simply require forward or backward substitution. Thus the speed of the calculation has been enhanced by the LU decomposition. LU is one of several other decompositions, so we are just scratching the surface!

Let us see how the LU method works for a 3x3 problem.

$$\overbrace{\begin{pmatrix} 1 & 1 & -1 \\ 2 & -1 & 1 \\ 3 & 2 & -4 \end{pmatrix}}^{A} = \overbrace{\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 1/3 & 1 \end{pmatrix}}^{L} \overbrace{\begin{pmatrix} 1 & 1 & -1 \\ 0 & -3 & 3 \\ 0 & 0 & -2 \end{pmatrix}}^{U}$$

Let us now invert A using $\mathbf{b} = (0, 6, -4)^T$

The stage $L\mathbf{y} = \mathbf{b}$ is

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 1/3 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 6 \\ -4 \end{pmatrix}$$

We solve for $\mathbf{y}$ by sweeping down, using forward substitution

$$\begin{aligned} y_1 &= 0 \\ 2y_1 + y_2 = y_2 &= 6 \\ 3y_1 + y_2/3 + y_3 = 2 + y_3 = -4 &\Rightarrow y_3 = -6 \end{aligned}$$

so that

$$\mathbf{y} = \begin{pmatrix} 0 \\ 6 \\ -6 \end{pmatrix}$$

The second stage of Eq. 5.3 is to invert U

$$\begin{pmatrix} 1 & 1 & -1 \\ 0 & -3 & 3 \\ 0 & 0 & -2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \mathbf{y} = \begin{pmatrix} 0 \\ 6 \\ -6 \end{pmatrix}$$

And then by sweeping up (backward substitution) we have

$$\begin{aligned} -2x_3 = -6 &\Rightarrow x_3 = 3 \\ -3x_2 + 3x_3 = 6 &\Rightarrow x_2 = 1 \\ x_1 + x_2 - x_3 = 0 &\Rightarrow x_1 = 2 \end{aligned}$$

so that the solution is

$$\mathbf{x} = \begin{pmatrix} 2 \\ 1 \\ 3 \end{pmatrix}.$$

5.4.1 The cost of Gaussian elimination versus LU

For N by N matrices the number of operations for each method becomes critical as N gets larger. We define two types of operation:

1. evaluation of each multiplier element m_{cr}.
2. multiplication-subtraction of each row element from the pivot row.

For the clearing the first column, there are $N-1$ evaluations of the multiplier elements, and there are N multiplication-subtraction operations along each row. Thus to clear the column below the first pivot there are

$$N(N-1) = N^2 - N$$

operations. The approximate total number of operations for all pivots is evaluated using standard summation formulae

$$\sum_{k=1}^{N} k^2 - k = \frac{1}{3}(N^3 - N) = O(N^3).$$

In contrast, the LU decomposition simply uses forward and backward substitution. Backward substitution starts with one operation and ends with N. Forward substitution has a similar number of steps giving $O(N^2)$ steps. Thus, when N is large, LU decomposition is significantly faster than than Gaussian elimination.

5.5 Iterative matrix methods

Just as we used iterative methods for finding roots of equations in part 1, we can also formulate matrix inversion in an iterative way. Such formulations are very useful for large matrix problems.

5.5.1 Jacobi method

The first method, called Jacobi iteration, involves first rewriting the inversion problem as

$$A\mathbf{x} = \mathbf{b} \Leftrightarrow \mathbf{x} = \mathbf{c} - B\mathbf{x}.$$

This is analogous to the approach of fixed-point iteration, i.e. rewriting the problem in a form more appropriate for iteration. The Jacobi iteration is then

$$\mathbf{x}_{n+1} = \mathbf{c} - B\mathbf{x}_n. \tag{5.4}$$

5.5.2 Example of rearrangement

Consider solving $A\mathbf{x} = \mathbf{b}$ with

$$A = \begin{pmatrix} 7 & -2 & 1 \\ 1 & -9 & 3 \\ 2 & 1 & 10 \end{pmatrix} \quad \mathbf{b} = \begin{pmatrix} 17 \\ 13 \\ 15 \end{pmatrix}$$

The following is the long-winded way of rearranging it into the Jacobi form but provides insight into how the method works. We will learn later of a more efficient method. If we expand $A\mathbf{x} = \mathbf{b}$ as simultaneous equations by writing $\mathbf{x} = \begin{pmatrix} x & y & z \end{pmatrix}^T$ we get

$$7x - 2y + z = 17, \tag{5.5}$$
$$x - 9y + 3z = 13, \tag{5.6}$$
$$2x + y + 10z = 15. \tag{5.7}$$

Dividing through using the diagonal elements of A: (5.5)/7, -(5.6)/9 and (5.7)/10 to get

$$x = \frac{17}{7} + \frac{2}{7}y - \frac{1}{7}z$$
$$y = -\frac{13}{9} + \frac{1}{9}x + \frac{1}{3}z$$
$$z = \frac{3}{2} - \frac{1}{5}x - \frac{1}{10}y$$

Reverting to iterative matrix form gives

$$\mathbf{x}_{n+1} = \overbrace{\begin{pmatrix} \frac{17}{7} \\ -\frac{13}{9} \\ \frac{3}{2} \end{pmatrix}}^{\mathbf{c}} - \overbrace{\begin{pmatrix} 0 & -\frac{2}{7} & \frac{1}{7} \\ -\frac{1}{9} & 0 & -\frac{1}{3} \\ \frac{1}{5} & \frac{1}{10} & 0 \end{pmatrix}}^{B} \mathbf{x}_n$$

Not that the B matrix has zeros on the diagonal.

5.5.3 Python example

We can test the Jacobi method in python.

```
import numpy as np

# Set print precision
np.set_printoptions(precision=15, suppress=True)

# Define matrix A and vector b
A = np.array([[7, -2, 1],
              [1, -9, 3],
              [2, 1, 10]])

b = np.array([[17],
            [13],
            [15]])

# Solve for the true solution using numpy.linalg.solve
x_star = np.linalg.solve(A, b)
print("x_star =\n", x_star)

# Define matrix B and vector c for the iterative method
B = np.array([[0, -2/7, 1/7],
              [-1/9, 0, -1/3],
              [1/5, 1/10, 0]])

c = np.array([[17/7],
              [-13/9],
              [3/2]])

# Optional: check spectral radius of B
eigvals = np.linalg.eigvals(B)
spectral_radius = max(abs(eigvals))
print("\nSpectral radius of B:", spectral_radius)
```

Initialise $\mathbf{x}$ with an arbitrary starting point

```
# Initial value of x (column vector of zeros)
x = np.array([[0],
              [0],
              [0]]) # column vector
```

Now apply the Jacobi iteration 10 times

```
# Jacobi iteration: x = c - np.matmul(B,x)
for n in range(10):
```

```
    x = c - np.matmul(B,x)
    # or equivalently:x= c - np.dot(B, x)
```

The result for the final iteration is

```
x =

   2.024028797279636
  -0.826914551848934
   1.177881191449034
```

which is close to the true solution, The absolute error ($|x - x_*|$) is

```
# Absolute error after 10 Jacobi iterations
error = np.abs(x - x_star)
print("Absolute error:\n", error)

Absolute error:
 [[0.000009664258826]
 [0.000008525074143]
 [0.000003423935582]]
```

which is of order 10^{-5} the size of the solution.

5.5.4 Gauss-Seidel method

The Jacobi iteration in iterative form is

$$x_{n+1} = \frac{17}{7} + \frac{2}{7}y_n - \frac{1}{7}z_n \tag{5.8}$$

$$y_{n+1} = -\frac{13}{9} + \frac{1}{9}x_n + \frac{1}{3}z_n \tag{5.9}$$

$$z_{n+1} = \frac{3}{2} - \frac{1}{5}x_n - \frac{1}{10}y_n \tag{5.10}$$

The Gauss-Seidel method is a modification of Jacobi to accelerate convergence. After the evaluation of x_{n+1} in Equation 5.8, the Gauss-Seidel method used the more up-to-date value in Equation 5.9. Equally, in Equation 5.10 the values x_{n+1} and y_{n+1} are used instead on the right hand side

$$x_{n+1} = \frac{17}{7} + \frac{2}{7}y_n - \frac{1}{7}z_n \tag{5.11}$$

$$y_{n+1} = -\frac{13}{9} + \overbrace{\frac{1}{9}x_{n+1}}^{\text{Modified}} + \frac{1}{3}z_n \tag{5.12}$$

$$z_{n+1} = \frac{3}{2} - \overbrace{\frac{1}{5}x_{n+1}}^{\text{Modified}} - \overbrace{\frac{1}{10}y_{n+1}}^{\text{Modified}} \tag{5.13}$$

where the terms modified from the Jacobi method are annotated. The use of the more up to date values of x and y leads to faster convergence to the solution. We now need to develop a methodology for analysing the convergence of the Jacobi and Gauss-Seidel methods.

5.6 Decomposition for analysis of Jacobi and Gauss-Seidel

We decompose A as

$$A = L + D + U,$$

where L and U are now *strictly* (zero diagonal) lower and upper triangular matrices and D is a diagonal matrix. This decomposition will clarify the relationship betwen the B and A matrices. It will also lead to a more rapid calculation of the B matrix. For example

$$\overbrace{\begin{pmatrix} 7 & -2 & 1 \\ 1 & -9 & 3 \\ 2 & 1 & 10 \end{pmatrix}}^{A} = \overbrace{\begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 2 & 1 & 0 \end{pmatrix}}^{L} + \overbrace{\begin{pmatrix} 7 & 0 & 0 \\ 0 & -9 & 0 \\ 0 & 0 & 10 \end{pmatrix}}^{D} + \overbrace{\begin{pmatrix} 0 & -2 & 1 \\ 0 & 0 & 3 \\ 0 & 0 & 0 \end{pmatrix}}^{U}$$

is simply a moving of the elements of A into the appropriate matrices. Note that the L and U here are not identical to those in the LU decomposition.

5.6.1 Jacobi method

Now the Jacobi method can be analysed.

$$\begin{aligned} &A\mathbf{x} = \mathbf{b} \Leftrightarrow (L + D + U)\mathbf{x} = \mathbf{b} \\ &\Rightarrow (L + U)\mathbf{x} + D\mathbf{x} = \mathbf{b} \Rightarrow D\mathbf{x} = \mathbf{b} - (L + U)\mathbf{x} \\ &\Rightarrow \mathbf{x} = D^{-1}\mathbf{b} - D^{-1}(L + U)\mathbf{x} \Rightarrow \mathbf{x}_{n+1} = D^{-1}\mathbf{b} - D^{-1}(L + U)\mathbf{x}_n \\ &\Rightarrow \mathbf{c} = D^{-1}\mathbf{b}, \quad \boxed{B = D^{-1}(L + U)} \end{aligned}$$

Thus the B matrix can be compiled from the L, U and D matrices. This method should be faster than the simultaneous equation method we used previously.

5.6.2 Gauss-Seidel method

We can also analyse the Gauss-Seidel method in a similar way. We modify the Jacobi method

$$\begin{aligned} \mathbf{x}_{n+1} &= D^{-1}\mathbf{b} - D^{-1}(L + U)\mathbf{x}_n \\ &= D^{-1}\mathbf{b} - D^{-1}U\mathbf{x}_n - D^{-1}L\mathbf{x}_n \end{aligned}$$

to

$$\mathbf{x}_{n+1} = D^{-1}\mathbf{b} - D^{-1}U\mathbf{x}_n - D^{-1}L\overbrace{\mathbf{x}_{n+1}}^{\text{Modified}}$$

for the Gauss-Seidel method. Here we have modified the last term in the Jacobi method with variables from previous steps, which corresponds to the multiplication of $\mathbf{x}_{n+1}$ by L. We can see this decomposition in Eqs. (5.11)-(5.13) by expressing them in vector-matrix form

$$\overbrace{\begin{pmatrix} x_{n+1} \\ y_{n+1} \\ z_{n+1} \end{pmatrix}}^{\mathbf{x}_{n+1}} = \overbrace{\begin{pmatrix} \frac{17}{7} \\ -\frac{13}{9} \\ \frac{3}{2} \end{pmatrix}}^{D^{-1}\mathbf{b}} - \overbrace{\begin{pmatrix} 0 & -\frac{2}{7} & \frac{1}{7} \\ 0 & 0 & -\frac{1}{3} \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} x_n \\ y_n \\ z_n \end{pmatrix}}^{D^{-1}U\mathbf{x}_n} - \overbrace{\begin{pmatrix} 0 & 0 & 0 \\ -\frac{1}{9} & 0 & 0 \\ \frac{1}{5} & \frac{1}{10} & 0 \end{pmatrix} \begin{pmatrix} x_{n+1} \\ y_{n+1} \\ z_{n+1} \end{pmatrix}}^{D^{-1}L\mathbf{x}_{n+1}}$$

Rearranging gives

$$\begin{aligned} \Rightarrow (D+L)\mathbf{x}_{n+1} &= \mathbf{b} - U\mathbf{x}_n \\ \Rightarrow \mathbf{x}_{n+1} &= (D+L)^{-1}\mathbf{b} - (D+L)^{-1}U\mathbf{x}_n \\ \Rightarrow \mathbf{c} &= (D+L)^{-1}\mathbf{b}, \quad \boxed{B = (D+L)^{-1}U} \end{aligned}$$

5.7 Norms: tools for convergence analysis

In order to analyse the convergence properties of iterative matrix methods, we require the ability to quantify the magnitude of a matrix or a vector.

5.7.1 Vector norms

Put simply, norms are a way of quantifying the 'length' of a vector.

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix}$$

There are three common norm definitions.

1. **Euclidian or l_2 norm**

$$\|\mathbf{x}\|_2 = \left(\sum_{i=1}^{N} x_i^2 \right)^{1/2}$$

   The l_2 norm is essentially applying Pythagorus' theorem to determine the length.

2. **Manhattan or l_1 norm**

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{N} |x_i|$$

   The l_1 norm is sometimes called the "Taxicab" norm and measures the summation of the distances along each axis.

3. **Infinity (l_∞) norm**

$$\|\mathbf{x}\|_\infty = \max_{i=1\ldots N}(|x_i|)$$

Also called the supremum or sup norm. Compared to the l_1 norm, it only uses the maximum absolute value. It will be used more in our evaluation of the Jacobi and Gauss-Seidel methods.

Let's consider some examples of these norms to understand more how they work. Consider the vector

$$\mathbf{x} = \begin{pmatrix} 2 \\ -3 \\ 6 \end{pmatrix}.$$

The different norms for this vector are:

$$\|\mathbf{x}\|_2 = \sqrt{4+9+36} = \sqrt{49} = 7,$$

$$\|\mathbf{x}\|_1 = |2| + |-3| + |6| = 11,$$

$$\|\mathbf{x}\|_\infty = \max(|2|, |-3|, |6|) = 6.$$

There are several useful identities associated with norms. One useful property for our purposes is called homogeneity. When the vector is multiplied by a positive constant α we have

$$\|\alpha\mathbf{x}\| = \alpha\|\mathbf{x}\|.$$

The definition of a vector norm allows us to define the convergence problem more precisely. Convergence means that the solution ($\mathbf{x}$) converges to the true solution ($\mathbf{x}_*$) as $n \to \infty$. This can now be stated more rigorously as

$$\|\mathbf{x}_n - \mathbf{x}_*\| \to 0 \quad \text{as } n \to \infty.$$

Let us consider an artificial example of a converging numerical method

$$\mathbf{x}_n = \left(\frac{1}{n}, 1 + \frac{1}{n^2}, 0\right)^T$$

so the true solution is

$$\mathbf{x}_n \to \mathbf{x}_* = (0, 1, 0)^T \quad \text{as } n \to \infty.$$

Let's now use the norms to quantify convergence

$$\mathbf{x}_n - \mathbf{x}_* = \left(\frac{1}{n}, \frac{1}{n^2}, 0\right)^T$$

First consider the l_∞ norm

$$\|\mathbf{x}_n - \mathbf{x}_*\|_\infty = \max\left(\frac{1}{n}, \frac{1}{n^2}, 0\right) = \frac{1}{n}$$

and we can clearly see that

$$\|\mathbf{x}_n - \mathbf{x}_*\|_\infty \to 0 \quad \text{as } n \to \infty$$

Now lets try the l_2 norm

$$\|\mathbf{x}_n - \mathbf{x}_*\|_2 = \left(\frac{1}{n^2} + \frac{1}{n^4}\right)^{1/2}$$

We can also see using the l_2 norm that

$$\|\mathbf{x}_n - \mathbf{x}_*\|_2 \to 0 \quad \text{as } n \to \infty$$

5.7.2 Matrix norms

For our numerical analysis, it will also prove useful to define a norm of a matrix A. The l_∞ norm of a matrix will prove useful later and is defined as

$$\|A\|_\infty = \max_{i=1\ldots N} \left(\sum_{j=1}^{N} |a_{ij}|\right)$$

where a_{ij} are the matrix elements in row-column format. The method involves summing absolute values along each row (the sum over j) to generate a vector, and then applying the infinity norm (the maximum over i) to the resulting vector. For a 2x2 matrix, the absolute values are summed across each row, forming a vector, and the maximum of these elements is found. Consider the matrix

$$A = \begin{pmatrix} 1 & -2 \\ 3 & 4 \end{pmatrix}$$

The l_∞ norm is then

$$\begin{aligned} \|A\|_\infty &= \max(|1| + |-2|, |3| + |4|) \\ &= \max(3, 7) = 7 \end{aligned}$$

A very useful identity with matrix norms is called *submultiplicativity* (a bit of mouthful!). For two matrices A and B, the matrix norm of the multiplication is such that

$$\|AB\| \leq \|A\|\|B\| \tag{5.14}$$

Recall, that *multiplicativity* applies to absolute values ($|ab| \equiv |a||b|$) so the term *sub*multiplicativity makes sense for Eq. (5.14). A similar submultiplicativity relation exists for multiplication of matrices by vectors

$$\|A\mathbf{x}\| \leq \|A\|\|\mathbf{x}\| \tag{5.15}$$

5.8 Convergence analysis of Jacobi method: method of norms

Definition 3. *A is strictly diagonally dominant if $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for any value of i where a_{ij} are the elements of A.*

Theorem 5. *The Jacobi method will converge to the solution of* $A\mathbf{x} = \mathbf{b}$ *if* A *is strictly diagonally dominant.*

Proof The Jacobi algorithm is

$$\mathbf{x}_{n+1} = \mathbf{c} - B\mathbf{x}_n.$$

The true solution is when the iteration has converged and

$$\mathbf{x}_{n+1} = \mathbf{x}_n = \mathbf{x}_*$$

so that the true solution can be stated as

$$\mathbf{x}_* = \mathbf{c} - B\mathbf{x}_*.$$

The error ($\mathbf{e}_n = \mathbf{x}_n - \mathbf{x}_*$) is

$$\mathbf{e}_{n+1} = \mathbf{x}_{n+1} - \mathbf{x}_* = -B\mathbf{e}_n,$$

showing that it is the matrix B that controls convergence. Taking the l_∞ norm gives

$$\|\mathbf{e}_{n+1}\|_\infty = \|B\mathbf{e}_n\|_\infty.$$

Applying the submultiplicativity identity gives

$$\|\mathbf{e}_{n+1}\|_\infty \leq \|B\|_\infty \|\mathbf{e}_n\|_\infty.$$

For convergence we thus require the norm of the error to decrease with each step giving

$$\|B\|_\infty < 1. \tag{5.16}$$

We now need to relate Eq. (5.16) to the structure of the matrix A. Decomposing A as

$$A = L + D + U,$$

transforms the convergence condition to

$$\|B\|_\infty = \|D^{-1}(L+U)\|_\infty < 1.$$

Expanding the matrices in terms of the elements of A gives

$$\|D^{-1}(L+U)\|_\infty = \max_{i=1\ldots N} \sum_{j\neq i} |a_{ii}^{-1} a_{ij}| = \max_{i=1\ldots N} |a_{ii}^{-1}| \sum_{j\neq i} |a_{ij}| < 1.$$

There is a lot of information in this equation. Let's unpack it. Recall that both L and U have zero diagonals, so the sum $L+U$ also has a zero diagonal and contains all the non-diagonal parts of A. The summation of a_{ij} over $j \neq i$ accounts for multiplication by $L+U$. The a_{ii}^{-1} are the elements of D^{-1}, and since it is independent of j, it can be moved outside the summation. The maximum over i of absolute values is the infinity norm operation. Therefore for convergence, the $|a_{ii}|$ must be greater than $\sum_{j\neq i} |a_{ij}|$ for $i = 1\ldots N$. so that

$$|a_{ii}| > \sum_{j\neq i} |a_{ij}|$$

for any value of i. Therefore A is *strictly diagonally dominant*. Strict diagonal dominance is a *sufficient* condition for the convergence of the Jacobi method; it is not a *necessary* condition as there are some cases that are not strictly diagonally dominant that still converge (see next section).

5.8.1 Examples of diagonal dominance

The matrix

$$\begin{pmatrix} 5 & 2 & 1 \\ 0 & 4 & -3 \\ -4 & 2 & 7 \end{pmatrix}$$

is strictly diagonally dominant as for row 1, $5 > 2 + 1$, row 2, $4 > 3$ and row 3, $7 > 4 + 2$. However, the tridiagonal matrix

$$\begin{pmatrix} -2 & 1 & 0 \\ 1 & -2 & 1 \\ 0 & 1 & -2 \end{pmatrix}$$

is not strictly diagonally dominant as for row 2, $2 = 1 + 1$. Thus the proof of convergence based on norms cannot be applied to this matrix. However, this doesn't mean Jacobi will not converge. It is useful in these situations to try an alternative method, given in the next section.

5.9 Convergence analysis using eigenvalues

The strictly diagonally dominant condition is a little limited, as we know there are some matrices that don't satisfy this condition yet still converge, such as the tridiagonal matrix. Another way of approaching the problem is by using the eigenvalues of B. Recall the error equation

$$\mathbf{e}_{n+1} = -B\mathbf{e}_n.$$

The eigenvalues (λ_i) and normal (unit length) eigenvectors ($\mathbf{v}_i$) of $-B$ (note the minus sign)are defined as

$$-B\mathbf{v}_i = \lambda_i \mathbf{v}_i \tag{5.17}$$

If we split the initial error into its J eigenvectors

$$\mathbf{e}_0 = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \ldots + c_J\mathbf{v}_J$$

where c_i are constants determined by the initial condition. Strictly speaking, for $\mathbf{v}_i$ to be orthogonal and thus form a complete basis, $-B$ should be symmetric. If we multiply by $-B$ and use equation 5.17 we have

$$\begin{aligned} -B\mathbf{e}_0 &= -c_1B\mathbf{v}_1 - c_2B\mathbf{v}_2 + \ldots - c_JB\mathbf{v}_J \\ \mathbf{e}_1 &= c_1\lambda_1\mathbf{v}_1 + c_2\lambda_2\mathbf{v}_2 + \ldots + c_J\lambda_J\mathbf{v}_J \end{aligned}$$

Iterating n times gives

$$\begin{aligned} \mathbf{e}_n &= c_1\lambda_1^n\mathbf{v}_1 + c_2\lambda_2^n\mathbf{v}_2 + \ldots + c_J\lambda_J^n\mathbf{v}_J \\ \|\mathbf{e}_n\| &= c_1|\lambda_1|^n + c_2|\lambda_2|^n + \ldots + c_J|\lambda_J|^n \end{aligned}$$

For large n, the largest absolute eigenvalue in the series will dominate. The largest absolute eigenvalue will thus dominate the convergence of the absolute error to zero. The iterative method will only converge if $|\lambda_i| < 1$ for all i. We thus define the spectral radius (S)

$$S = \max(|\lambda_i|)$$

If $S < 1$ we can be assured of convergence. The rate of convergence will then be determined by the size of S: the smaller S, the faster the convergence. This provides a relatively straightforward way of assessing how fast Jacobi or Gauss-Seidel converges (or any other iteration that can be formulated in a similar way). Note that λ_i can be eigenvalues of either $-B$ or B, as it is the absolute value of the eigenvalues that is important.

5.9.1 Worked example of spectral radius

Consider solving $A\mathbf{x} = \mathbf{b}$ by the Jacobi method with

$$A = \begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix}$$

By inspection

$$L = \begin{pmatrix} 0 & 0 \\ -1 & 0 \end{pmatrix} \quad D = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix} \quad U = \begin{pmatrix} 0 & -1 \\ 0 & 0 \end{pmatrix}$$

and so

$$-B = -D^{-1}(L+U) = -\begin{pmatrix} \frac{1}{3} & 0 \\ 0 & \frac{1}{3} \end{pmatrix}\begin{pmatrix} 0 & -1 \\ -1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & \frac{1}{3} \\ \frac{1}{3} & 0 \end{pmatrix}$$

The eigenvalues of $-B$ are

$$\begin{aligned} |-B-\lambda I| &= 0 \\ \begin{vmatrix} -\lambda & \frac{1}{3} \\ \frac{1}{3} & -\lambda \end{vmatrix} &= 0 \\ \lambda^2 = \frac{1}{9} \Rightarrow \lambda = \pm\frac{1}{3} \Rightarrow S &= \max(|\lambda|) = \frac{1}{3} \end{aligned}$$

So $S < 1$, indicating that the Jacobi method will converge. We will now compute S for the Gauss-Seidel method.

$$-B = -(L+D)^{-1}U = -\begin{pmatrix} 3 & 0 \\ -1 & 3 \end{pmatrix}^{-1}\begin{pmatrix} 0 & -1 \\ 0 & 0 \end{pmatrix} = -\frac{1}{9}\begin{pmatrix} 3 & 0 \\ 1 & 3 \end{pmatrix}^{-1}\begin{pmatrix} 0 & -1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & \frac{1}{3} \\ 0 & \frac{1}{9} \end{pmatrix}$$

The eigenvalues of $-B$ are

$$\begin{aligned} \begin{vmatrix} -\lambda & \frac{1}{3} \\ 0 & \frac{1}{9} - \lambda \end{vmatrix} &= 0 \\ \lambda\left(\lambda - \frac{1}{9}\right) \Rightarrow \lambda = 0, \frac{1}{9} \Rightarrow S &= \max(|\lambda|) = \frac{1}{9} \end{aligned}$$

So the spectral radius for Gauss-Seidel is smaller than Jacobi, so Gauss-Seidel should converge faster.

5.10 Calculation of eigenvalues: power method

The power method is a way of calculating the maximum absolute eigenvalue of a very large matrix. The method forms part of the the Google PageRank algorithm. The power method involves expanding the vector $\mathbf{x}$ in terms of the normal eigenvectors of A

$$\mathbf{x} = c_1\mathbf{v}_1 + \ldots + c_N\mathbf{v}_N$$

where we have also ordered the expansion in terms of the eigenvector with the largest absolute eigenvalue first, the next largest second etc.

$$|\lambda_1| > |\lambda_2| \ldots \geq |\lambda_N|$$

Multiplying through by A gives

$$A\mathbf{x} = c_1A\mathbf{v}_1 + \ldots + c_NA\mathbf{v}_N$$

By definition, $A\mathbf{v}_i = \lambda_i\mathbf{v}_i$ and so

$$A\mathbf{x} = c_1\lambda_1\mathbf{v}_1 + \ldots + c_N\lambda_N\mathbf{v}_N$$

Iterating k times, we accumulate powers of A (hence the name power method)

$$A^k\mathbf{x} = c_1\lambda_1^k\mathbf{v}_1 + \ldots + c_N\lambda_N^k\mathbf{v}_N$$

As $k \to \infty$

$$A^k\mathbf{x} \to c_1\lambda_1^k\mathbf{v}_1 \quad \text{as } k \to \infty.$$

The power method is implemented in the following way. We first define an iterative form of the eigenvalue equation, using k as the iteration step.

$$A\mathbf{x}_{k-1} = m_k\mathbf{x}_k$$

and, following the power method derivation above, if the method converges then

$$\mathbf{x}_k \to \mathbf{x}_{k-1} \to \mathbf{v}_1 \quad \text{as } k \to \infty.$$

$$m_k \to \lambda_1 \quad \text{as } k \to \infty.$$

For practical computation, the power method is split in the following way:

- **Matrix multiplication**

$$\mathbf{z}_{k-1} = A\mathbf{x}_{k-1}, \quad m_k = \max(\mathbf{z}_{k-1}),$$

- **Normalisation**

$$\mathbf{x}_k = \frac{\mathbf{z}_{k-1}}{m_k}.$$

  where m_k is the largest element of $\mathbf{z}_{k-1}$. The normalisation step prevents the numerical problem of underflow or overflow of the norm of $\mathbf{x}_k$. We can see that eliminating $\mathbf{z}_{k-1}$ from the two steps gives $m_k\mathbf{x}_k = A\mathbf{x}_{k-1}$ which is our iterative eigenvalue problem.

It is important to note that the power method works best when absolute values of the eigenvalues are significantly separated. When the eigenvalues are close, convergence of the method will be slow.

5.10.1 Python example

Lets find the largest absolute eigenvalue of the 3x3 matrix

$$A = \begin{pmatrix} -9 & 14 & 4 \\ -7 & 12 & 4 \\ 0 & 0 & 1 \end{pmatrix}$$

In python this is

```
import numpy as np

A = np.array([
    [-9, 14, 4],
    [-7, 12, 4],
    [ 0, 0, 1]
])
```

We define an arbitrary initial condition for $\mathbf{x}$

```
x = np.array([[1],
              [1],
              [1]])
```

We then apply the multiplication and normalisation steps as single lines of code. We have placed this algorithm in a loop of 10 iterations.

```
for n in range(10):
    # Matrix-vector multiplication
    z = np.matmul(A,x)

    # Normalize by the maximum value in z
    x = z / np.max(z)

print("Maximum eigenvalue:\n", np.max(z))
```

```
Maximum eigenvalue:
 5.000001024000262
```

We can compare with the value determined by the python intrinsic eigenvalue routine.

```
# Calculate eigenvalues of A using python's numpy linalg module.
eigvals = np.linalg.eigvals(A)

print("Maximum absolute eigenvalue of A:", np.max(np.abs(eigvals)))
```

```
Maximum absolute eigenvalue of A: 5.0
```

5.10.2 Power method for smallest absolute eigenvalue

The smallest absolute eigenvalue can also be determined by using the inverse in the place of A, as $1/\lambda_i$ are the eigenvalues of A^{-1}. The algorithm should be changed to

```
# Reset x
x = np.array([[1],
              [1],
              [1]])

for n in range(10):
    # Matrix-vector multiplication by inverse of A
    z = np.matmul(np.linalg.inv(A),x)

    # Normalize by the maximum value in z
    x = z / np.max(z)

print("Minimum eigenvalue of A:\n", 1/np.max(z))

print("Minimum value using python routines:", np.min(np.abs(eigvals)))
```

giving the smallest absolute eigenvalue, 1.

5.11 The condition number

On a computer, all the numbers represented in a matrix or a vector will suffer from round-off error. In some situations, these round-off errors have a negligible effect on the solution; these matrices are termed "well conditioned". When the solution is sensitive to round-off error, then they are termed "ill conditioned".

Let us formalise these scenarios by considering a small change to $A\mathbf{x} = \mathbf{b}$, through changes to the right-hand side, $\mathbf{b} \to \mathbf{b} + \Delta\mathbf{b}$, and the solution, $\mathbf{x} \to \mathbf{x} + \Delta\mathbf{x}$.

$$A(\mathbf{x} + \Delta\mathbf{x}) = \mathbf{b} + \Delta\mathbf{b}$$

Subtracting $A\mathbf{x} = \mathbf{b}$ gives

$$A\Delta\mathbf{x} = \Delta\mathbf{b}, \quad \Delta\mathbf{x} = A^{-1}\Delta\mathbf{b}$$

So, the impact of the change $\Delta\mathbf{b}$ on $\Delta\mathbf{x}$ depends on A^{-1}, and this is especially the case in the direction of the eigenvectors with the largest eigenvalues of A^{-1} (corresponding to the smallest eigenvalues of A). This effect is captured by the condition number c

$$c = \frac{\lambda_{max}}{\lambda_{min}},$$

where λ_{min} and λ_{max} are the minimum and maximum eigenvalues of A. An ill conditioned A will have a relatively small minimum eigenvalue (the relative part is determined by comparing it to λ_{max}) and thus a large c. The opposite is true of a well conditioned matrix. Consider the following example

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 1.0001 \end{pmatrix}$$

We might have a hunch that this matrix will be problematic as its determinant is close to zero. Let's see what the condition number shows us in this case. The eigenvalues of A are 2 and 0.5×10^{-4} giving a condition number of

$$c = \frac{2}{0.5 \times 10^{-4}} = 4 \times 10^4,$$

which is large and thus A is ill conditioned. In python, the condition number can be determined from the command

```
A=np.array([[1, 1],
            [1,1.0001]])

print(np.linalg.cond(A))
```

5.12 Matrices and partial differential equations

We can now combine our knowledge of ODEs and matrices. Let us consider the diffusion partial differential equation (PDE) for the variable $\phi(x,t)$

$$\frac{\partial \phi}{\partial t} = K \frac{\partial^2 \phi}{\partial x^2}, \tag{5.18}$$

where K is a constant diffusion. We discretise Eq. (5.18) on the domain $x_i = i\Delta x$ and $t_n = n\Delta t$ where $i \in \mathbb{Z}^+$ and $n \in \mathbb{Z}^+$. The variable ϕ is discretised as

$$\phi \to \phi_i^{(n)},$$

where the indices i and n are in the subscript and superscript respectively. Note the braces around n to distinguish it from a power. This problem is such that often an implicit (Backward-Euler in time) discretisation is required for numerical stability. The second-order approximation for the spatial derivatives is also used giving

$$\frac{\phi_i^{(n+1)} - \phi_i^{(n)}}{\Delta t} = K \frac{\phi_{i+1}^{(n+1)} - 2\phi_i^{(n+1)} + \phi_{i-1}^{(n+1)}}{\Delta x^2}.$$

Rearranging so that $n+1$ terms are the left hand side gives

$$-\nu \phi_{i-1}^{(n+1)} + (1+2\nu)\phi_i^{(n+1)} - \nu \phi_{i+1}^{(n+1)} = \phi_i^{(n)},$$

where

$$\nu = \frac{K\Delta t}{\Delta x^2}.$$

This can be solved in the tridiagonal matrix form

$$T\mathbf{y}^{(n+1)} = \mathbf{y}^{(n)},$$

where all the elements $\phi_i^{(n)}$ are contained in the vector $\mathbf{y}^{(n)}$. Expanding the matrix equation gives

$$\overbrace{\begin{bmatrix}
\ddots & & & & & & & & & \\
& \ddots & & & & & & & & \\
& & \ddots & & & & & & & \\
& & -\nu & 1+2\nu & -\nu & & & & & \\
& & & -\nu & 1+2\nu & -\nu & & & & \\
& & & & -\nu & 1+2\nu & -\nu & & & \\
& & & & & & \ddots & & & \\
& & & & & & & \ddots & & \\
& & & & & & & & \ddots &
\end{bmatrix}}^{T}
\overbrace{\begin{bmatrix}
\phi_0^{(n+1)} \\ \vdots \\ \vdots \\ \phi_{i-1}^{(n+1)} \\ \phi_i^{(n+1)} \\ \phi_{i+1}^{(n+1)} \\ \vdots \\ \vdots \\ \phi_{N-1}^{(n+1)}
\end{bmatrix}}^{\mathbf{y}^{(n+1)}}
=
\overbrace{\begin{bmatrix}
\phi_0^{(n)} \\ \vdots \\ \vdots \\ \phi_{i-1}^{(n)} \\ \phi_i^{(n)} \\ \phi_{i+1}^{(n)} \\ \vdots \\ \vdots \\ \phi_{N-1}^{(n)}
\end{bmatrix}}^{\mathbf{y}^{(n)}}.$$

Chapter 6

Topics in mathematical modelling

Historical figures

Kirchhoff's circuit laws (1845), used throughout this chapter's electronic network example, generalised Ohm's law to arbitrary circuits and are the oldest idea here; their author, **Gustav Kirchhoff** (1824–1887), could hardly have anticipated where numerical simulation on a computer would eventually lead. In 1955, **Mary Tsingou** (1928–2026) coded and ran the numerical experiment at Los Alamos — simulating a chain of nonlinearly coupled oscillators on the MANIAC computer — that became known as the Fermi–Pasta–Ulam problem, one of the first uses of a computer to explore nonlinear dynamics; her contribution went uncredited for over 50 years until the problem was renamed Fermi–Pasta–Ulam–Tsingou (FPUT) in 2008. Her experiment is now recognised, alongside **Edward Norton Lorenz**'s (1917–2008) independent and more famous accidental discovery of deterministic chaos eight years later, as one of the two foundational moments in which a computer revealed unexpected behaviour in a nonlinear system. Lorenz's came in 1961, while re-running a truncated, rounded version of a numerical weather model and finding wildly different results — leading to the Lorenz equations used in this chapter. Kirchhoff, Tsingou, Lorenz on Wikipedia.

In this chapter, we will look at some applications of the numerical methods that you have previously encountered. It is hoped that this process will increase your appreciation of how wide ranging numerical modelling techniques are. The emphasis here is on the mathematical and numerical issues and should not require expertise in any of these science areas beyond the details provided. The following sections will give the mathematical outline and scientific background of five topics. Through the lectures, practicals and coursework, we will investigate them further.

6.1 Electronic networks: a matrix problem

The two fundamental laws of electronic circuits are:

1. **Ohm's law**

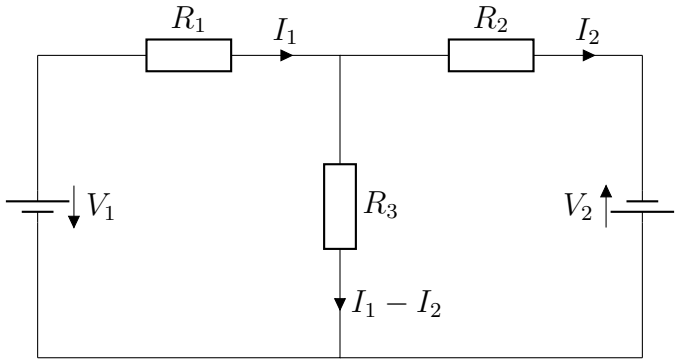
$$V = IR,$$

where V is the voltage, I the current and R the resistance. Put very simply, the voltage can be viewed as *pushing* the current through the resistance. Rewriting Ohm's law as $I = V/R$, we see that, for a given voltage, the current is inversely proportional to the resistance.

2. **Kirchhoff's laws**

   - Kirchhoff's current law states that sum of the current into and out of a node is zero.
   - Kirchhoff's voltage law states that the sum of voltages (including sources such as batteries) is zero around a closed loop.

Consider the following electronic network of voltage sources from batteries (V_1, V_2) and resistors (R_1, R_2 and R_3).



The currents (I_1, I_2 and $I_1 - I_2$) are constructed such that Kirchhoff's current law is satisfied at the node at the top. If Ohm's and Kirchhoff's voltage law are applied to the two loops we have

$$V_1 = I_1 R_1 + (I_1 - I_2) R_3$$
$$V_2 = I_2 R_2 + (I_2 - I_1) R_3$$

which can be written in matrix form as

$$\begin{pmatrix} V_1 \\ V_2 \end{pmatrix} = \begin{pmatrix} R_1 + R_3 & -R_3 \\ -R_3 & R_2 + R_3 \end{pmatrix} \begin{pmatrix} I_1 \\ I_2 \end{pmatrix}$$

Thus, for a given voltage vector, we can calculate the current by inverting the resistance matrix. The most general form of this type of network also includes sources of current, increasing the size of the characteristic matrices.

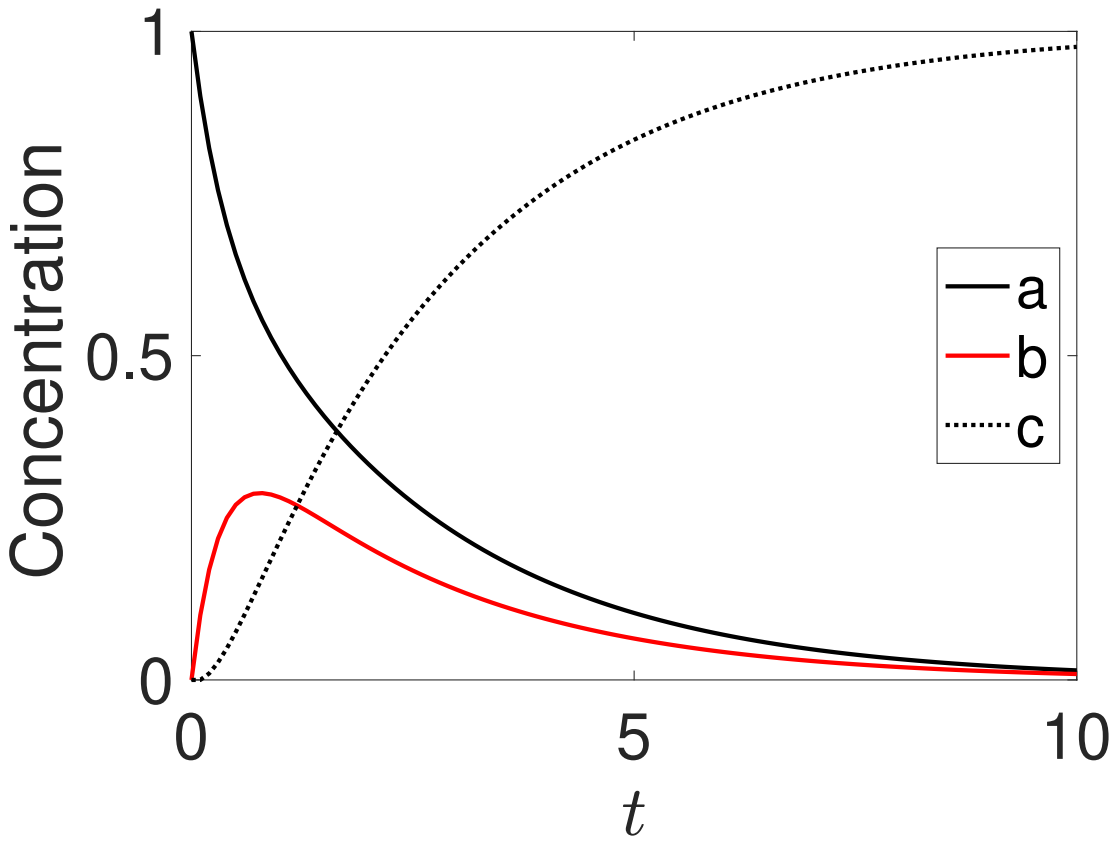


Figure 6.1: Evolution of concentrations a, b, c with time, for initial conditions $a(0) = 1$, $b(0) = c(0) = 0$, and rate constants are $r_1 = r_2 = r_3 = 1$.

6.2 Chemical kinetics: an ODE/matrix problem

Consider chemical reactions between species A, B, C with concentrations a, b, c respectively

$$\begin{aligned}
\frac{da}{dt} &= -r_1 a + r_2 b, \\
\frac{db}{dt} &= r_1 a - (r_2 + r_3) b, \\
\frac{dc}{dt} &= r_3 b,
\end{aligned}$$

where r_1, r_2 and r_3 are rate constants with time scales as small as 0.1 s. The transfer between the species is constructed so that the total concentration is conserved

$$\begin{aligned}
&\frac{da}{dt} + \frac{db}{dt} + \frac{dc}{dt} = 0, \\
&\Rightarrow a + b + c = \text{constant} = a(0) + b(0) + c(0),
\end{aligned}$$

and set by the initial conditions.

Given the size of the rate constants, an implicit method is appropriate

$$
\begin{aligned}
a_{n+1} &= a_n - r_1 \Delta t a_{n+1} + r_2 \Delta t b_{n+1}, \\
b_{n+1} &= b_n + r_1 \Delta t a_{n+1} - (r_2 + r_3) \Delta t b_{n+1}, \\
c_{n+1} &= c_n + r_3 \Delta t b_{n+1},
\end{aligned}
$$

where n is the timestep. Rearranging and writing as a matrix problem gives.

$$
\begin{pmatrix} 1 + r_1 \Delta t & -r_2 \Delta t & 0 \\ -r_1 \Delta t & 1 + (r_2 + r_3) \Delta t & 0 \\ 0 & -r_3 \Delta t & 1 \end{pmatrix} \begin{pmatrix} a_{n+1} \\ b_{n+1} \\ c_{n+1} \end{pmatrix} = \begin{pmatrix} a_n \\ b_n \\ c_n \end{pmatrix}
$$

So solving for the values at the new timestep now involves a matrix inversion at each step. The extra computational expense of the matrix inversion is the trade-off for additional numerical stability.

6.3 Predictability and chaos: nonlinear ODEs

The Lorenz equations were invented by a meteorologist called Edward Lorenz from MIT, USA in 1963 (see the *Journal of Atmospheric Sciences*, volume 20, page 130). He was interested in formulating a simple model to explain why weather forecasts were sometimes unpredictable. For example: why do weather forecasts with very similar starting points sometimes have such different outcomes? This feature is termed, 'sensitivity to initial conditions'. In devising these equations and their numerical solutions, he also made a foundational contribution to the mathematical science of chaos.

The Lorenz equations consist of three time-dependent ordinary differential equations (ODEs) for the three variables X, Y, Z:

$$
\frac{dX}{dt} = \sigma(Y - X), \tag{6.1}
$$

$$
\frac{dY}{dt} = -XZ + rX - Y, \tag{6.2}
$$

$$
\frac{dZ}{dt} = XY - bZ, \tag{6.3}
$$

where σ, r and b are parameters. The key feature of these equations is the presence of *non-linearity* via the term $-XZ$ in equation (6.2) and XY in equation (6.3).

6.3.1 Predictability ideas

The Lorenz equations are an example of a model that exhibits chaos and unpredictability. The key aspects of chaos are:

- **Sensitivity to initial conditions** This is where two close initial conditions can diverge and lead to completely different trajectories. In the example of weather forecasting, this could be small changes in the Western Atlantic giving rise to bigger changes in the weather in the UK.

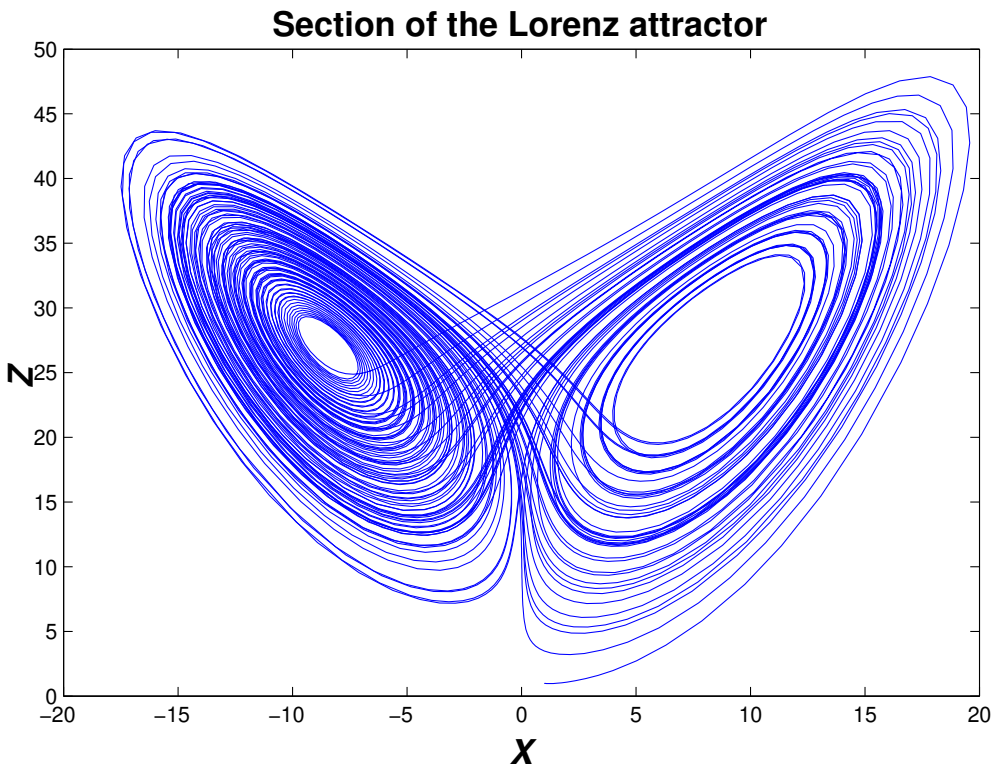


Figure 6.2: An X-Z section of the Lorenz attractor

- **Sensitivity to model parameters** This is where the evolution of the system is changed by the alteration of a key parameter in the system. For example, a weather prediction model has many different parameters. If one of these is changed, it could lead to a different behaviour of the forecast.
- **Aperiodic behaviour in time** This is where the variation of a variable in time never repeats itself.

6.3.2 Parameters for chaos

The parameter values

$$\sigma = 10,$$
$$r = 28,$$
$$b = 8/3,$$

are the values that produce the chaotic behaviour described previously. The X, Y, Z plot of the trajectory forms the 'attractor' of the system, and its distinctive two-lobe structure has fascinated mathematicians for years. As the parameter r is decreased to zero the system will 'bifurcate', and the attractor will no longer exist. Bifurcations and attractors form part of the theory of dynamical systems. We are going to focus on how the numerical method and timestep combine to preserve the intricate structure of the attactor.

6.3.3 Runge-Kutta schemes

A common choice for solution of this system is the fourth order Runga Kutta scheme. The right hand side Eqs. 6.1 - 6.3 allows for relatively straighforward evalutation of values within the time step. It also has the advantage that only values from the previous timestep are required. In the ODEs part of this module, this scheme was written for a single scalar variable x_n. This can be changed for a three-component ODE by changing $x_n \to \mathbf{x}_n$ where

$$\mathbf{x}_n = \begin{pmatrix} X_n \\ Y_n \\ Z_n \end{pmatrix}.$$

The scalar function $F(x,t)$ changes to a vector function with a vector argument. Also, since the right hand side is not explicitly a function of time, we can drop the time argument, so $F(x_n,t_n) \to \mathbf{F}(\mathbf{x}_n,t_n) \to \mathbf{F}(\mathbf{x}_n)$ where the right-hand side of Eqs. 6.1 - 6.3 form its element.

$$\mathbf{F}(\mathbf{x}_n) = \begin{pmatrix} \sigma(Y_n - X_n), \\ -X_n Z_n + rX_n - Y_n, \\ X_n Y_n - bZ_n, \end{pmatrix}.$$

Here scalar coefficients change to vectors

$$\begin{aligned} \mathbf{k}_1 &= \mathbf{F}(\mathbf{x}_n), \\ \mathbf{k}_2 &= \mathbf{F}\left(\mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_1\right), \\ \mathbf{k}_3 &= \mathbf{F}\left(\mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_2\right), \\ \mathbf{k}_4 &= \mathbf{F}\left(\mathbf{x}_n + \Delta t\mathbf{k}_3\right) \end{aligned}$$

giving the timestepping formula

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4).$$

6.4 Image compression: a matrix decomposition problem

We now consider a very powerful matrix decomposition termed 'singular value decomposition' (SVD) and apply it to the problem of image compression.

$$A = USV^T \tag{6.4}$$

The decomposition is analogous to diagonalisation that uses the eigenvalues of A, but broader in application in that A might also be non-invertible (singular). S is a diagonal matrix containing the positive singular values σ_i. Furthermore, the convention is that the singular

values are in decreasing order with large values first. The columns of U and V are called the left and right singular vectors respectively. The matrices U and V are also orthogonal:

$$U^T U = V^T V = I \tag{6.5}$$

Taking the transpose of Eq. 6.4 and multiplying by A gives

$$\begin{aligned} A^T A &= (USV^T)^T USV^T \\ &= VS^T U^T USV^T \\ &= VS^T SV^T \end{aligned}$$

where we have applied Eq. 6.5 in the last stage. Whilst A is not diagonalisable in general, $A^T A$ is and so the above is exactly of the form for diagonalisation with the diagonal having elements of $S^T S$ i.e σ_i^2. Returning to Eq. 6.4, if we code a 2D image as a matrix A, we can further decompose Eq. 6.4 into a summation over singular values using *outer* products.

$$\begin{aligned} A &= USV^T \\ &= \mathbf{u}_1 \sigma_1 \mathbf{v}_1^T + \mathbf{u}_2 \sigma_2 \mathbf{v}_2^T + \ldots \end{aligned} \tag{6.6}$$

Where $\mathbf{u}_i$ and $\mathbf{v}_i$ are the left and right eigenvectors. Whilst an inner product (dot product) produces a single scalar value, the outer product produces a matrix from two vectors.

Let's consider two example vectors in python. Note here we are defining row vectors to fit with the numpy product routines.

```
import numpy as np

u_1=np.array([1, 2])
v_1=np.array([3, 4])
print(u_1,v_1)
```

```
[1 2] [3 4]
```

The inner product produces a single scalar number

```
print(np.inner(u_1,v_1))
```

```
11
```

whilst the outer product produces a matrix.

```
print(np.outer(u_1,v_1))
```

```
[[3 4]
 [6 8]]
```

Thus, Eq. 6.6 is a summation of separate matrices (images) weighted by the singular values.

The following python code uses the scikit-image image processing library. It decomposes an image and reconstitutes it using the SVD. It first transforms the image into a 2D array

```
# import skikit-imaga libraries
from skimage import io, img_as_float
from skimage.color import rgb2gray

# Read the image
A = io.imread('img.jpg')

# Convert to floating point values
B = img_as_float(A)

# Convert to 2D array, gray scale
C = rgb2gray(B)
```

A SVD is peformed on this array using the python SVD routine.

```
U, S, V = np.linalg.svd(C,full_matrices=True)
```

The code then sums up a number (here 4) of outer product series

```
import matplotlib.pyplot as plt
# sum over first N singular values
N=4

AN=U[:, :N] @ np.diag(S[:N]) @ V[:N, :]

plt.imshow(AN, cmap='gray')
```

where the @ shorthand performs an outer product given the ordering of the indicies for U, V and diagonal elements of S. Try increasing N to 20 and beyond.

Chapter 7

Partial differential equations

Historical figures

Joseph Fourier (1768–1830) derived and solved the diffusion (heat) equation in the early 1800s, introducing the Fourier series along the way to do so. Over a century later, his tool was turned into a tool for checking whether a numerical scheme is trustworthy: **John von Neumann** (1903–1957) developed the stability analysis used throughout this chapter, decomposing numerical solutions into Fourier modes to determine whether a finite-difference scheme amplifies or damps them. That analysis was put into practice by his wife, **Klára Dán von Neumann** (1911–1963), a self-taught programmer who wrote the code for the 1950 ENIAC experiment that produced the first successful numerical weather prediction, using finite-difference methods to timestep a simplified atmospheric PDE forward in time — putting exactly this chapter's ideas to work for the first time on a real computer. Fourier, J. von Neumann, K. von Neumann on Wikipedia.

In this chapter we consider the linear diffusion equation as an example of discretising partial differential equations.

7.1 The linear diffusion equation

The linear diffusion equation is

$$\frac{\partial \phi}{\partial t} = K \frac{\partial^2 \phi}{\partial x^2}$$

with K a constant. Assume the domain is periodic in x. If the initial condition is wavelike

$$\phi(x, 0) = e^{ikx}$$

then the exact solution can be found. Let

$$\phi(x, t) = \Phi(t) e^{ikx}.$$

Then

$$\frac{d\Phi}{dt} = -k^2 K \Phi,$$

so

$$\Phi = \Phi(0)e^{-k^2Kt}; \quad \Phi(0) = 1$$
$$\phi(x,t) = e^{-k^2Kt}e^{ikx}.$$

The solution is a stationary wave decreasing in amplitude. The damping becomes faster as the wavelength decreases.

7.2 von Neumann stability analysis

Divide the periodic space domain into N equal intervals of size Δx so that $x_j = j\Delta x$, $j = 0,\ 1,\ 2, \ldots,\ N$. Choose a time step Δt and let $t^{(n)} = n\Delta t$, $n = 0,\ 1,\ 2,\ \ldots$. Write ϕ_j^n for the numerical approximation to $\phi(x_j, t^{(n)})$.

Consider first a centred difference scheme:

$$\frac{\phi_j^{(n+1)} - \phi_j^{(n-1)}}{2\Delta t} = K\left(\frac{\phi_{j+1}^{(n)} - 2\phi_j^{(n)} + \phi_{j-1}^{(n)}}{\Delta x^2}\right).$$

This scheme is second order in space and second order in time, meaning that if we substitute the true solution into the finite difference scheme then the dominant terms in the residual are a term proportional to Δx^2 and a term proportional to Δt^2.

Von Neumann stability analysis. On a *periodic domain* any function can be decomposed into Fourier components. If the evolution of the function is governed by a *linear equation* with *constant coefficients* then its behaviour can be determined by looking at the behaviour of each Fourier component. Similarly, the stability of a linear numerical method can be found be considering each Fourier component and checking whether it grows in time. More generally, for nonlinear equations or equations with non-constant coefficients or non-periodic boundary conditions, the stability of individual Fourier components gives a useful guide but does not provide a sufficient condition for the stability of the full problem.

Look for numerical solutions of the form

$$\phi_j^{(n)} = A^n e^{ikj\Delta x}$$

where A is a (possibly complex) constant—the *amplification factor*. If we find $|A| > 1$ then the numerical solution is growing with time.

Substitute into the numerical scheme to obtain

$$A^{n+1}e^{ikj\Delta x} - A^{n-1}e^{ikj\Delta x} = \frac{2K\Delta t}{\Delta x^2}\left(A^n e^{ik(j+1)\Delta x} - 2A^n e^{ikj\Delta x} + A^n e^{ik(j-1)\Delta x}\right).$$

Cancelling powers of A and $e^{ikj\Delta x}$ leaves a quadratic equation for A:

$$A^2 + \frac{2K\Delta t}{\Delta x^2}\left(2 - 2\cos k\Delta x\right)A - 1 = 0.$$

There are two roots for A, corresponding to the physical mode and the computational mode.

$$A = -\frac{2K\Delta t}{\Delta x^2}\left(1 - \cos k\Delta x\right) \pm \left[\left(\frac{2K\Delta t}{\Delta x^2}\right)^2 \left(1 - \cos k\Delta x\right)^2 + 1\right]^{1/2}.$$

Both roots A^+ and A^- are real and their product is $A^+A^- = -1$. A^+ lies between 0 and 1 and this corresponds to the physical mode ($A^+ \to 1$ as $\Delta x,\ \Delta t/\Delta x \to 0$). On the other hand, $A^- < -1$ and this corresponds to the computational mode. The computational mode is unconditionally unstable. Therefore, this centred difference scheme is a poor choice for the diffusion equation.

An alternative scheme uses a forward Euler rather than a centred time step:

$$\frac{\phi_j^{(n+1)} - \phi_j^{(n)}}{\Delta t} = K\left(\frac{\phi_{j+1}^{(n)} - 2\phi_j^{(n)} + \phi_{j-1}^{(n)}}{\Delta x^2}\right).$$

This is first-order accurate in time and second-order accurate in space.

Von Neumann analysis leads to the following expression for the amplification factor:

$$A = 1 - \frac{2K\Delta t}{\Delta x^2}\left(1 - \cos k\Delta x\right).$$

Now $0 \leq (1 - \cos k\Delta x) \leq 2$, so $|A| \leq 1$ if $2K\Delta t/\Delta x^2 \leq 1$. This scheme is conditionally stable. Note that, because of the quadratic dependence on Δx, a moderate increase in resolution might require a much smaller time step.

Another alternative scheme uses a backward Euler time step:

$$\frac{\phi_j^{(n+1)} - \phi_j^{(n)}}{\Delta t} = K\left(\frac{\phi_{j+1}^{(n+1)} - 2\phi_j^{(n+1)} + \phi_{j-1}^{(n+1)}}{\Delta x^2}\right).$$

Again, this is first-order accurate in time and second-order accurate in space.

Von Neumann analysis leads to

$$A = \left(1 + \frac{2K\Delta t}{\Delta x^2}\left(1 - \cos k\Delta x\right)\right)^{-1}.$$

For this scheme $|A| \leq 1$ for any value of Δt, so it is unconditionally stable. However, the unknown $\phi^{(n+1)}$ appears on both sides of the equation defining the scheme, so it is non-trivial to solve for $\phi^{(n+1)}$. In this linear one-dimensional example $\phi^{(n+1)}$ can be found by solving a tridiagonal system of simultaneous equations, which can be done efficiently using Gaussian elimination. See the end of the matrices chapter for more details.